Web Programming

Rodrigo Martins Pagliares

Last update: 10/19/2025

Web Programming

Rodrigo Martins Pagliares

CSS building blocks

Don't want to see ads?

Filter

CSS building blocks
- CSS building blocks
- CSS selectors
- Type, class, and ID selectors
- Attribute selectors
- Pseudo-classes and pseudo-elements
- Combinators
- Cascade, specificity, and inheritance
- Cascade layers
- The box model
- Backgrounds and borders
- Handling different text directions
- Overflowing content
- CSS values and units
- Sizing items in CSS
- Images, media, and form elements
- Styling tables
- Debugging CSS
- Organizing your CSS

Learn web development

Welcome to the MDN learning area. This set of articles aims to guide complete beginners to web development with all that they need to start coding websites.

The aim of this area of MDN is not to take you from "beginner" to "expert" but to take you from "beginner" to "comfortable." From there, you should be able to start making your way, learning from the rest of MDN, and other intermediate to advanced resources that assume a lot of previous knowledge.

If you are a complete beginner, web development can be challenging — we will hold your hand and provide enough detail for you to feel comfortable and learn the topics properly. You should feel at home whether you are a student learning web development (on your own or as part of a class), a teacher looking for class materials, a hobbyist, or someone who just wants to understand more about how web technologies work.

Looking to become a front-end web developer?

We have put together a course that includes all the essential information you need to work towards your goal.

Get started

In this article

- Where to start
- Topics covered
- Getting our code examples
- Contact us
- See also

Master React 18 and build scalable apps with a complete React learning path.

Mozilla ads

Don't want to see ads?

https://tinyurl.com/zrffw4xb

The box model

Lecture: The box model
Module: CSS building blocks

Introduction

Introduction

- **Everything** in **CSS has** a **box around** it.
- **Understanding** these **boxes** is **key** to **create layouts** or to **align items** with other **items**.
- In this **lecture**, we will take a **look** at the **CSS *Box Model*** so that you can build **more complex layouts**.

Objective of the lecture

To **learn** about the **CSS Box Model**, what makes up the **box model** and **how to switch** to the **alternate model**.

Block & inline boxes

Block & inline boxes

- In **CSS** we broadly have **two types** of **boxes**:
  1. **Block boxes**
  2. **Inline boxes**

- The **type** of **box** refers to **how** the **box behaves** in **terms** of **page flow**, and in **relation** to other **boxes** on the **page**.

Box block

- **Break** onto a **new line**.
- Extend in the **inline direction** to **fill** the **space** available in its **container**.
- The `width` and `height` **properties** ARE **applied**.
- `padding`, `margin` and `border` will cause other **elements** to be **pushed away** from the **box**.

Unless we decide to **change** the `display` **type** to `inline`, **HTML block elements** such as `<h1>` and `<p>` use, by default, **block** as their **outer display type**.

Inline box

- The **box** will **NOT break** onto a **new line**.
- The `width` and `height` **properties ARE NOT** applied.
- **Vertical** `padding`, `margins` and `borders` **ARE** applied but **DO NOT cause** other **inline boxes** to **move away** from the **box**.
- **Horizontal** `padding`, `margins`, and `borders` ARE applied **apply** and **DO cause** other **inline boxes** to **move away** from the **box**.

The **HTML inline elements** **`<a>`**, **`<span>`**, **`<em>`** and **`<strong>`** are examples of **elements** that **display** as **inline block** by **default** in **CSS**.

The **type** of **box applied** to an **element** is **defined** by **`display` property values** such as **`block`** and **`inline`**, and **relates** to the **outer value** of **`display`**.

Aside: Inner & outer display types

Inner & outer display types

- **Boxes** have **outer** and **inner display** types**.**
- **Outer display**:
  - ✓Details **whether** the **box** is **block** or **inline**.
- **Inner display**:
  - ✓Dictates **how elements** inside a **box** are **laid out**.

Inner display type and normal flow

- The **elements** in the **inner display type** of a **box** are **laid out** in **normal flow**, by default.
  - ✓The **normal flow** is the **system** by which **elements** are **placed** inside the **browser's viewport**.
- **Block-level elements** appears on their own **new line** and are **separated** by any `margin` that is **set** on **them**.
- **Inline-level elements are placed next to each other as** long as they can **fit** on **the same line**.

Block and inline layout in normal flow

On this Page

Elements participating in a block formatting context
Elements participating in an inline formatting context
The display property and flow layout
Summary
See Also

Related Topics

CSS

CSS Reference

CSS Flow Layout

▾ Guides

Block and inline layout in normal flow
Flow Layout and Overflow
Flow Layout and Writing Modes
In Flow and Out of Flow
Introduction to formatting contexts

In this guide, we will explore the basics of how Block and Inline elements behave when they are part of the normal flow.

Normal Flow is defined in the CSS 2.1 specification, which explains that any boxes in normal flow will be part of a *formatting context*. They can be either block or inline, but not both at once. We describe block-level boxes as participating in a *block formatting context*, and inline-level boxes as participating in an *inline formatting context*.

The behaviour of elements which have a block or inline formatting context is also defined in this specification. For elements with a block formatting context, the spec says:

"In a block formatting context, boxes are laid out one after the other, vertically, beginning at the top of a containing block. The vertical distance between two sibling boxes is determined by the 'margin' properties. Vertical margins between adjacent block-level boxes in a block formatting context collapse.

In a block formatting context, each box's left outer
left edge of the containing block (for right-to-left fo
edges touch)." - 9.4.1

https://mzl.la/3iBcDON

Changing the inner display type

- We can **change** the **inner display type** of a **box** by using **`display` values** like **`flex`**.

- `display:flex;`
  - ✓The **outer display type** is **block**,
  - ✓The **inner display type** is changed to **flex**.

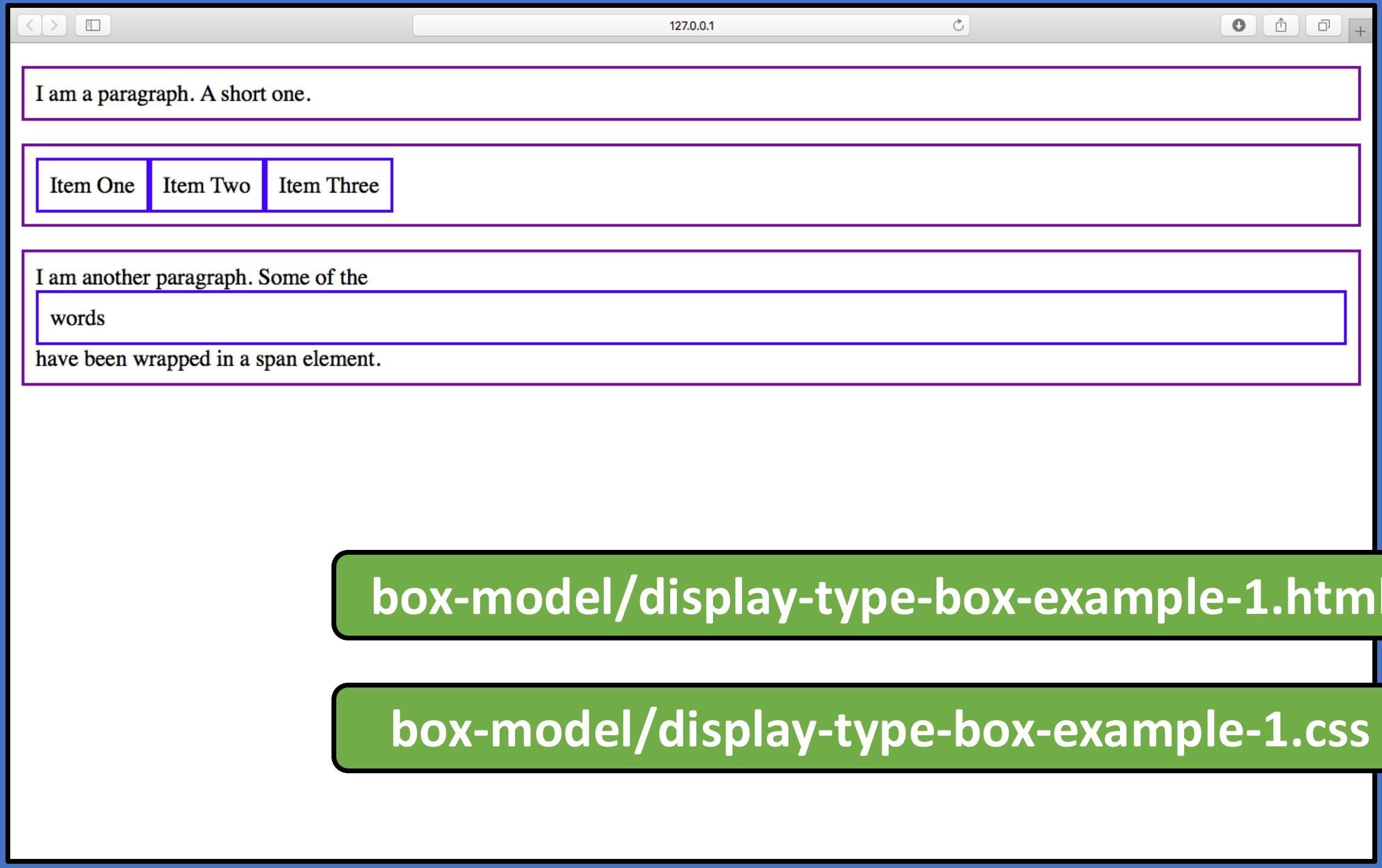
- Any **direct children** of this **box** will **become flex items**.
  - ✓ **Laid out** according to the **rules** set out in the **Flexbox specification**.
  - ✓**NOT** laid out with **normal flow**.

Note: `display` property

Note: To read more about the **values** of `display`, and **how boxes work** in **block** and **inline layout**, take a look at the **MDN guide** to **Block** and **Inline Layout**.

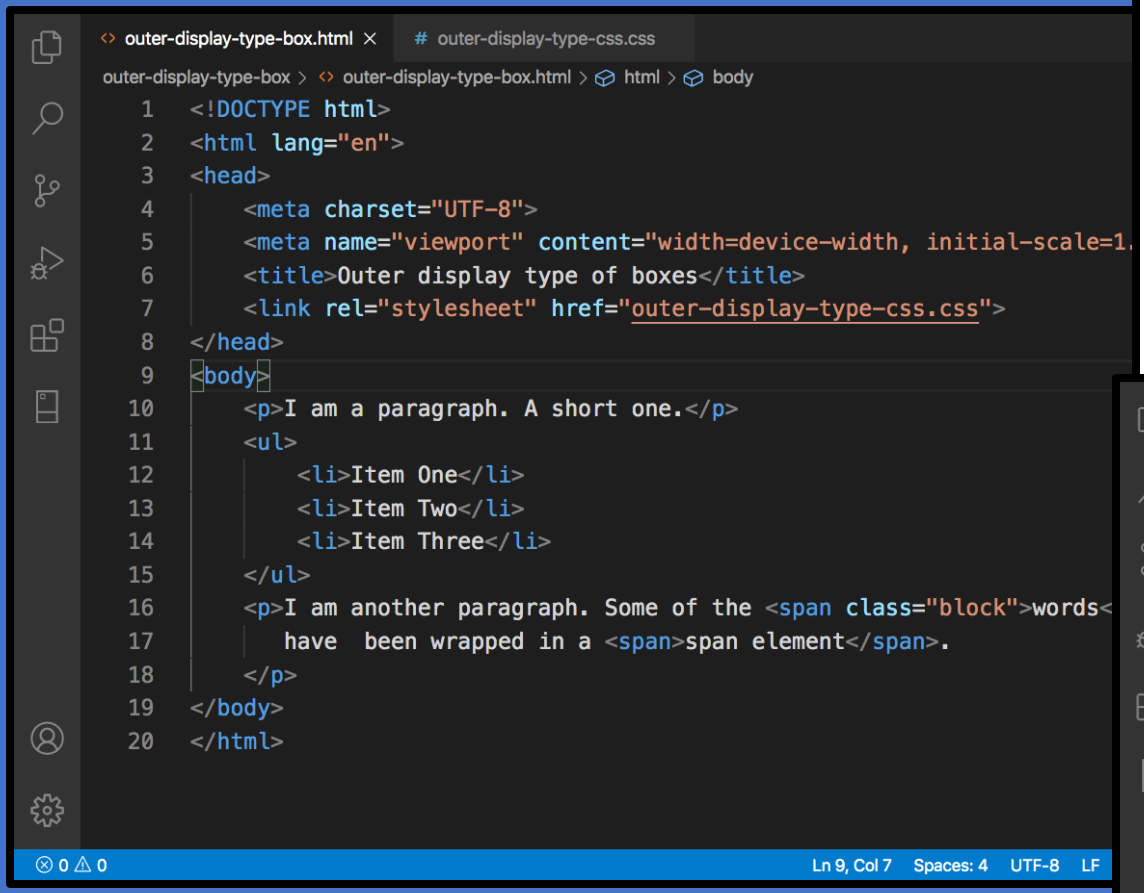
Examples of different `display` types

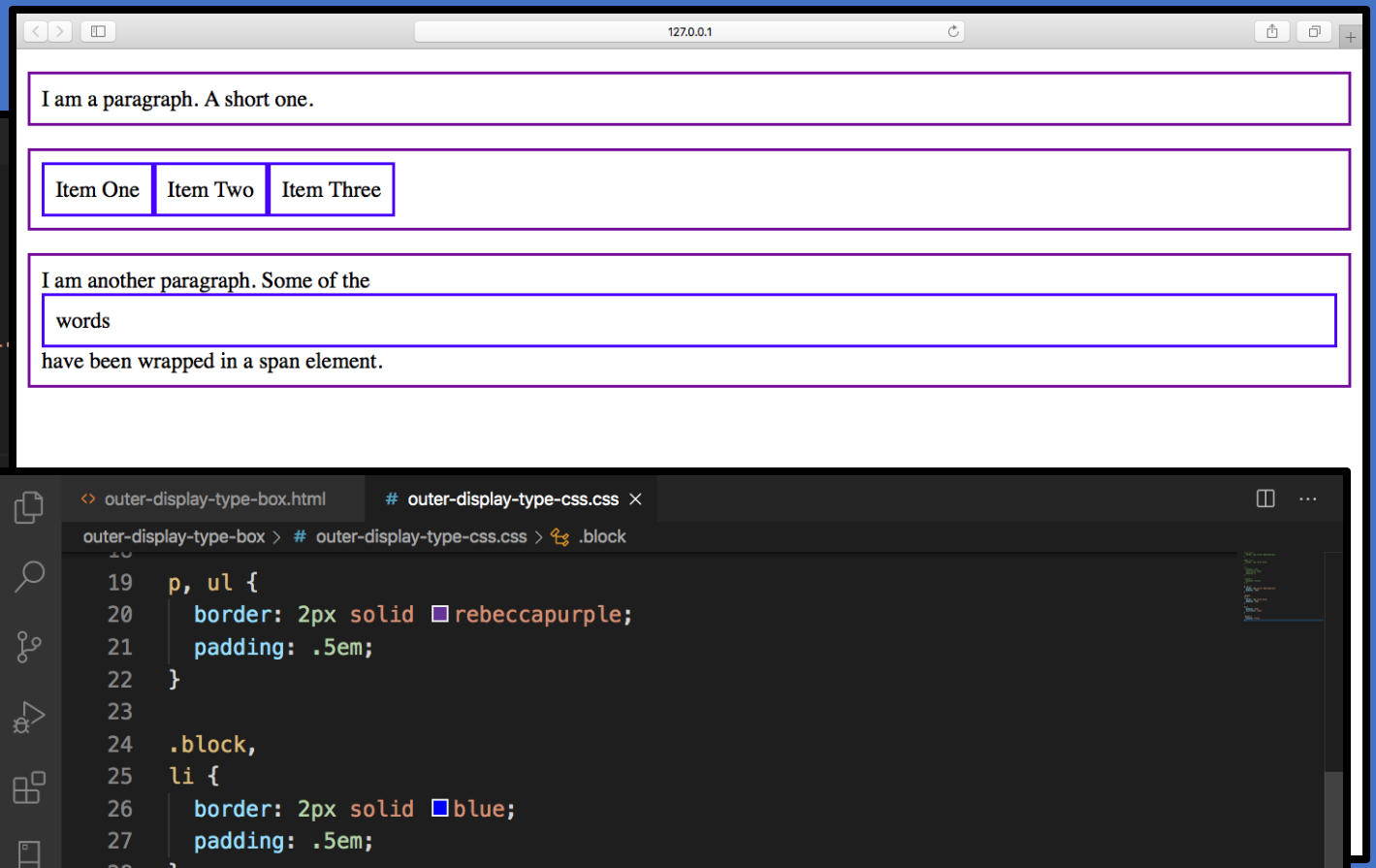
Example #1
Changing the `display` type

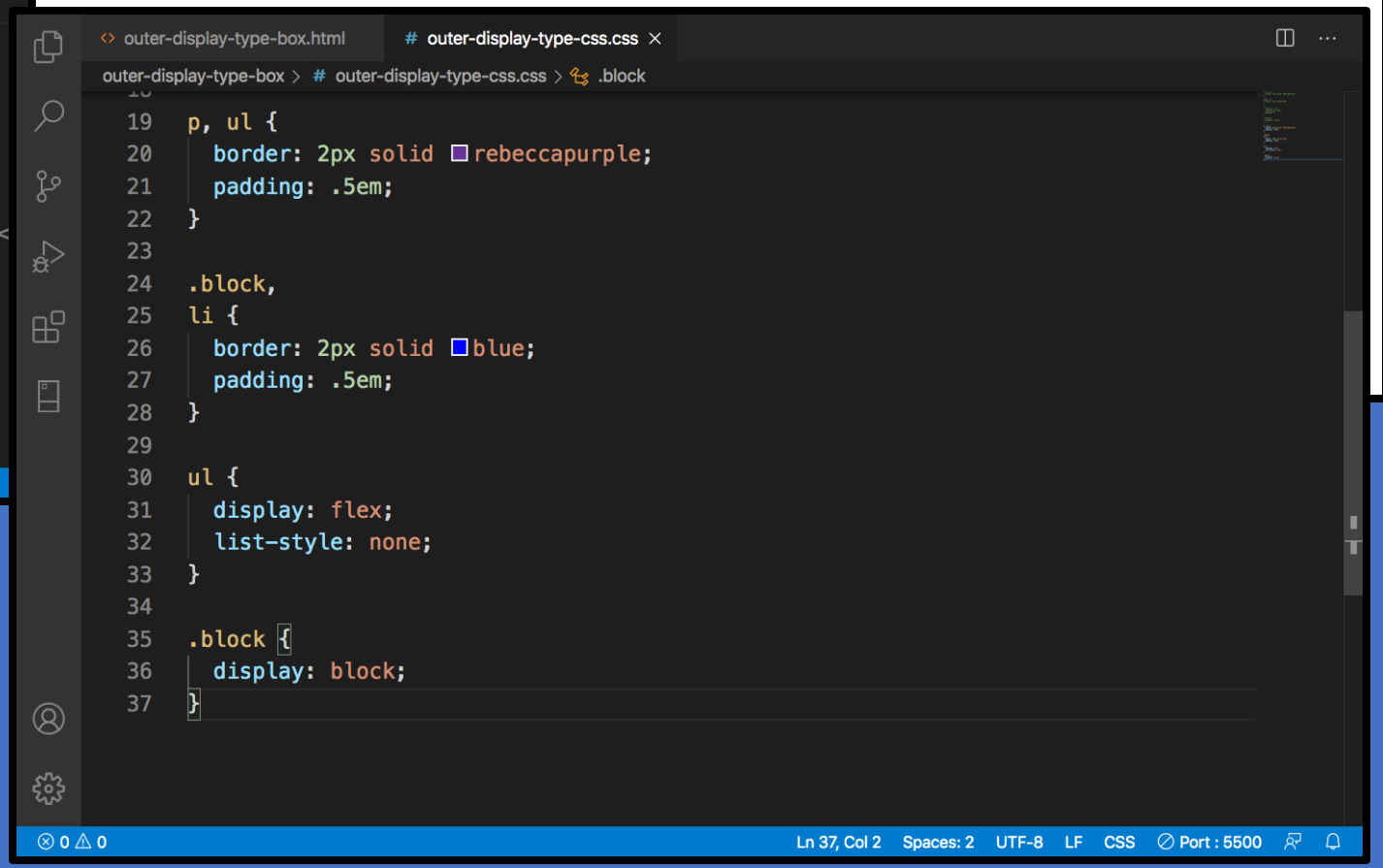


box-model/display-type-box-example-1.html

box-model/display-type-box-example-1.css


outer-display-type-box.html
outer-display-type-css.css
outer-display-type-box > outer-display-type-box.html > html > body
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.
6 <title>Outer display type of boxes</title>
7 <link rel="stylesheet" href="outer-display-type-css.css">
8 </head>
9 <body>
10 I am a paragraph. A short one.
11
12 Item One
13 Item Two
14 Item Three
15
16 I am another paragraph. Some of the words<
17 have been wrapped in a span element.
18
19 </body>
20 </html>
⊗ 0 ⚠ 0
Ln 9, Col 7 Spaces: 4 UTF-8 LF


127.0.0.1
I am a paragraph. A short one.
Item One
Item Two
Item Three
I am another paragraph. Some of the
words
have been wrapped in a span element.

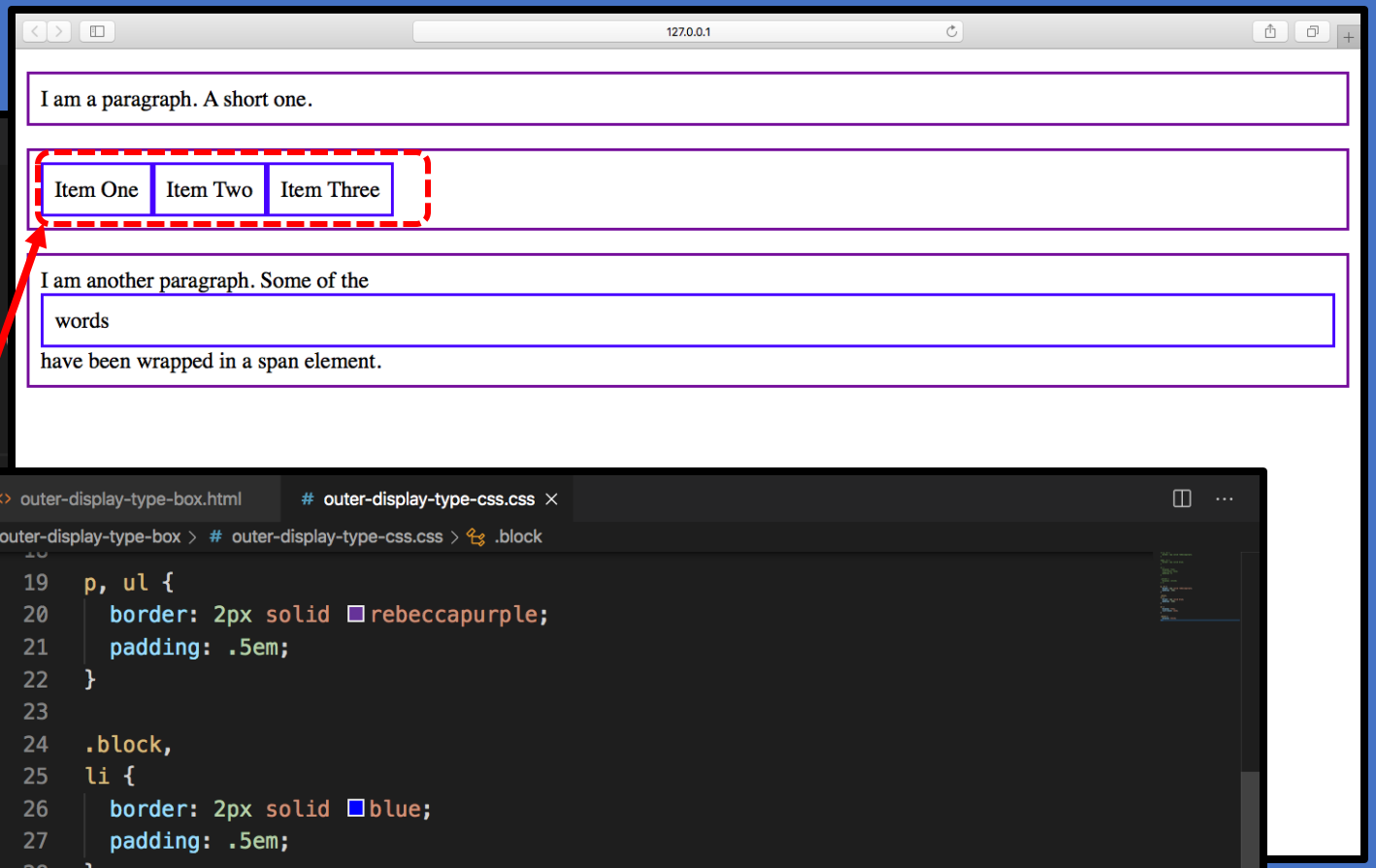

outer-display-type-box.html
outer-display-type-css.css
outer-display-type-box > outer-display-type-css.css > .block
19 p, ul {
20 border: 2px solid rebeccapurple;
21 padding: .5em;
22 }
23
24 .block,
25 li {
26 border: 2px solid blue;
27 padding: .5em;
28 }
29
30 ul {
31 display: flex;
32 list-style: none;
33 }
34
35 .block {
36 display: block;
37 }
⊗ 0 ⚠ 0
Ln 37, Col 2 Spaces: 2 UTF-8 LF CSS Port : 5500

Changing the inner display type

```
outer-display-type-box.html
outer-display-type-box > outer-display-type-box.html > html > body
```

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1
    <title>Outer display type of boxes</title>
    <link rel="stylesheet" href="outer-display-type-css.css">
</head>
<body>
    <p>I am a paragraph. A short one.</p>
    <ul>
        <li>Item One</li>
        <li>Item Two</li>
        <li>Item Three</li>
    </ul>
    <p>I am another paragraph. Some of the <span class="block">
      have  been wrapped in a <span>span element</span>.
    </p>
</body>
</html>
```

Ln 9, Col 7




```
outer-display-type-box.html
outer-display-type-box > outer-display-type-css.css > .block
```

```css
p, ul {
  border: 2px solid rebeccapurple;
  padding: .5em;
}

.block,
li {
  border: 2px solid blue;
  padding: .5em;
}

ul {
  display: flex;
  list-style: none;
}

.block {
  display: block;
}
```

Ln 37, Col 2 Spaces: 2 UTF-8 LF CSS Port : 5500

This example establishes **flex layout** for the **list items inside** the **container**, however, the **list** itself is a **block box** and — like the **paragraph** — **expands** to the **full container width** and **breaks** onto a **new line**.

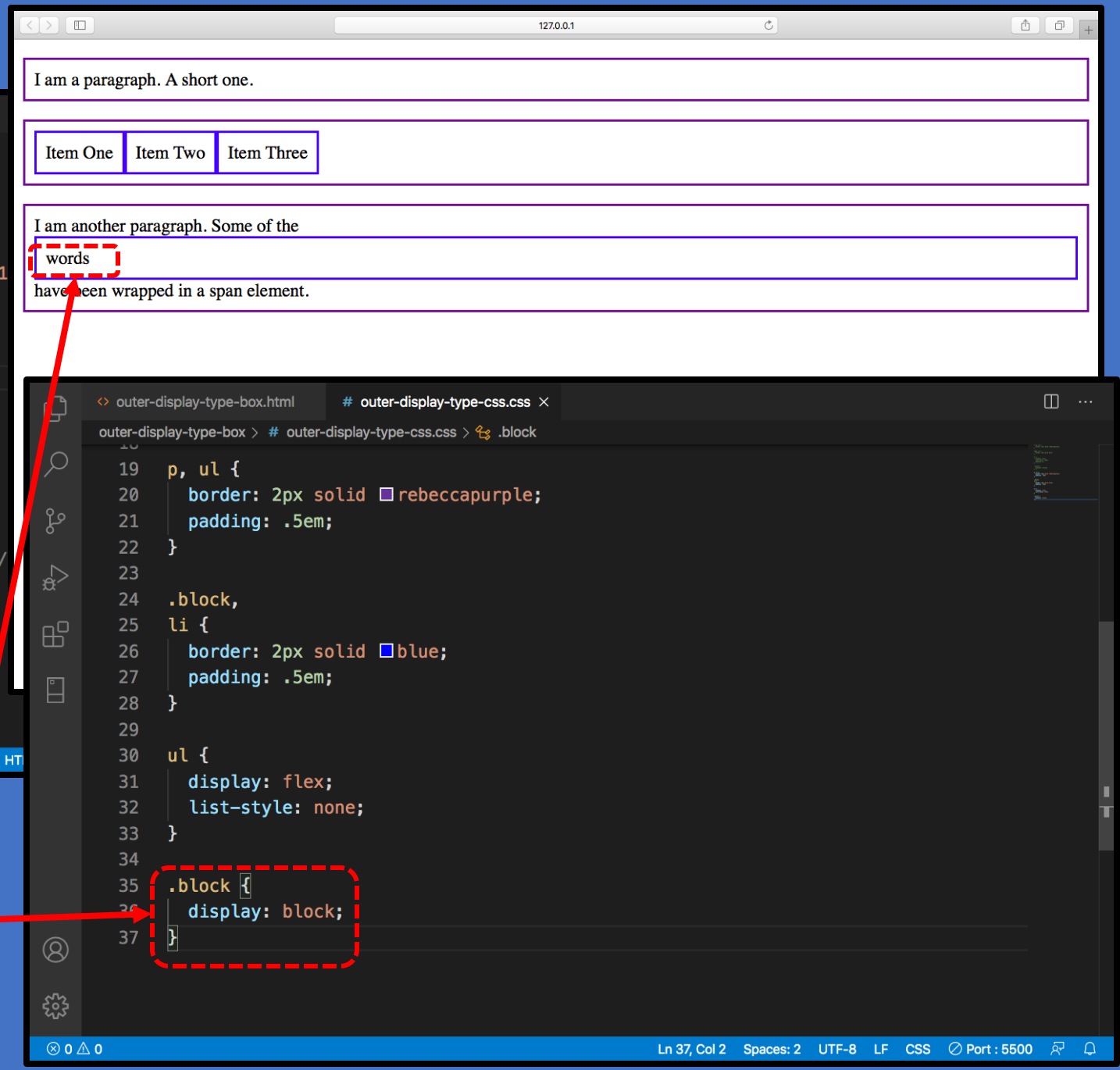
Changing the inner display type

outer-display-type-box.html × # outer-display-type-css.css

outer-display-type-box > outer-display-type-box.html > html > body

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1
6      <title>Outer display type of boxes</title>
7      <link rel="stylesheet" href="outer-display-type-css.css">
8  </head>
9  <body>
10     <p>I am a paragraph. A short one.</p>
11     <ul>
12         <li>Item One</li>
13         <li>Item Two</li>
14         <li>Item Three</li>
15     </ul>
16     <p>I am another paragraph. Some of the <span class="block">words</
17        have  been wrapped in a <span>span element</span>.
18     </p>
19 </body>
20 </html>
```

⊗ 0 ⚠ 0 Ln 9, Col 7 Spaces: 4 UTF-8 LF HT



outer-display-type-box.html # outer-display-type-css.css ×

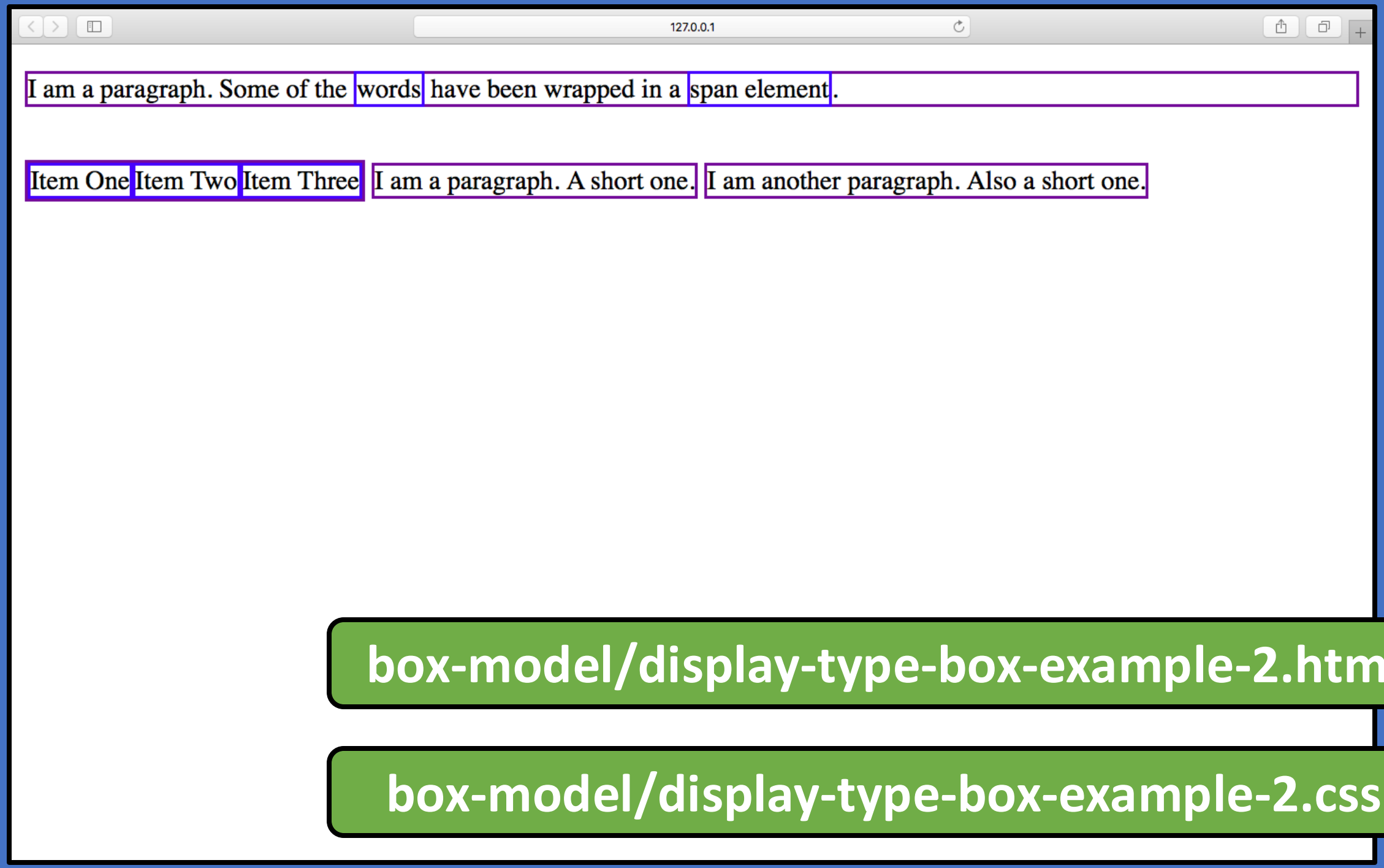
outer-display-type-box > # outer-display-type-css.css > .block

```
19 p, ul {
20   border: 2px solid rebeccapurple;
21   padding: .5em;
22 }
23
24 .block,
25 li {
26   border: 2px solid blue;
27   padding: .5em;
28 }
29
30 ul {
31   display: flex;
32   list-style: none;
33 }
34
35 .block {
36   display: block;
37 }
```

⊗ 0 ⚠ 0 Ln 37, Col 2 Spaces: 2 UTF-8 LF CSS Port : 5500

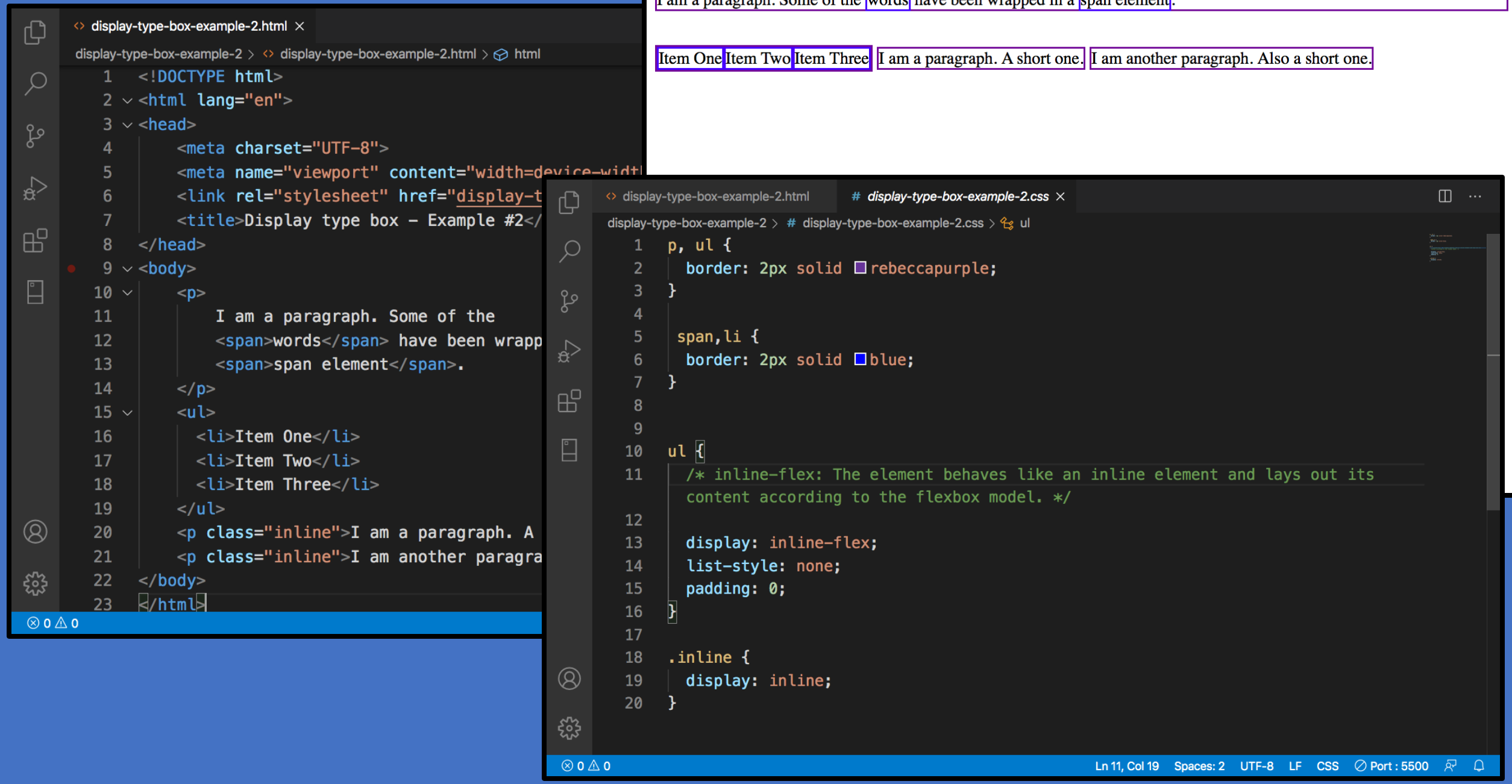
 is an **inline element**, however, one of them has a `class` of **block**, and we have **set** it to **inner display type** to `display: block`.

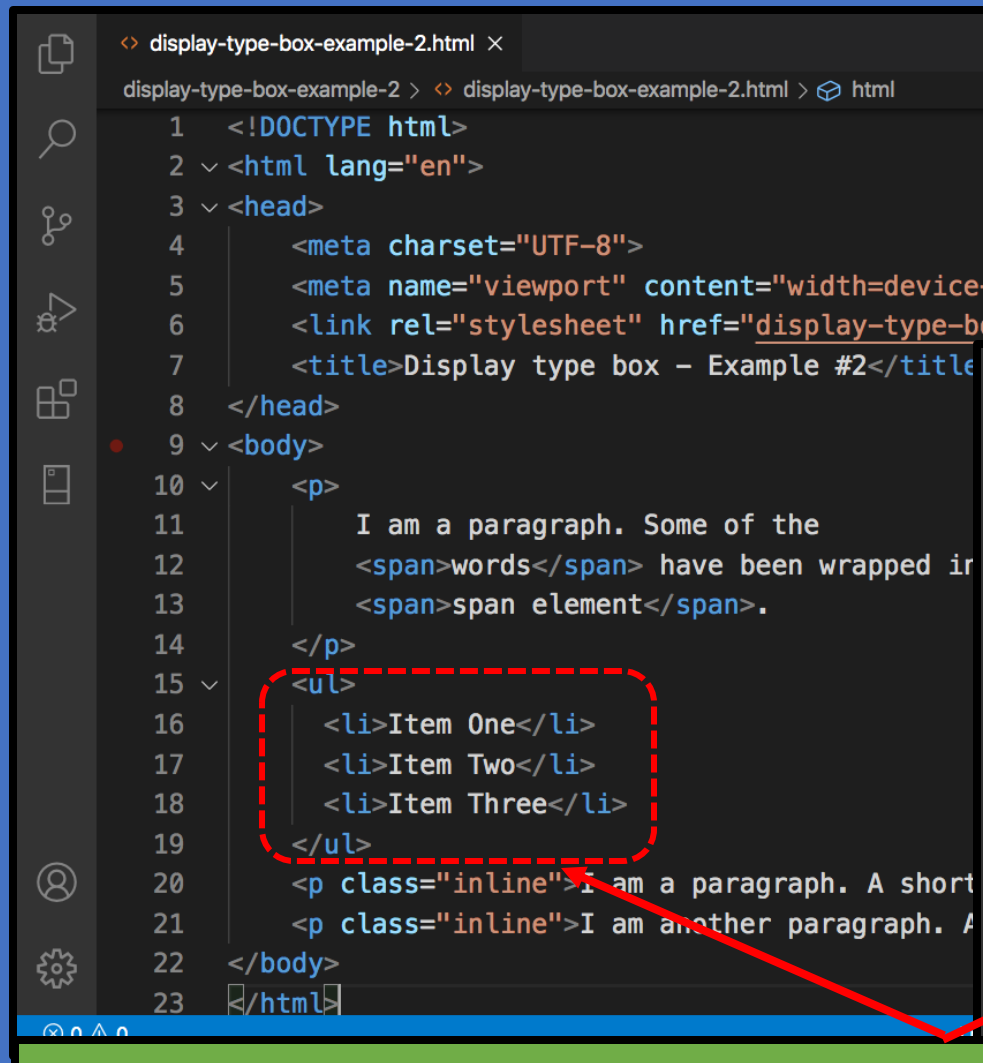
Example #2
Changing the `display` type

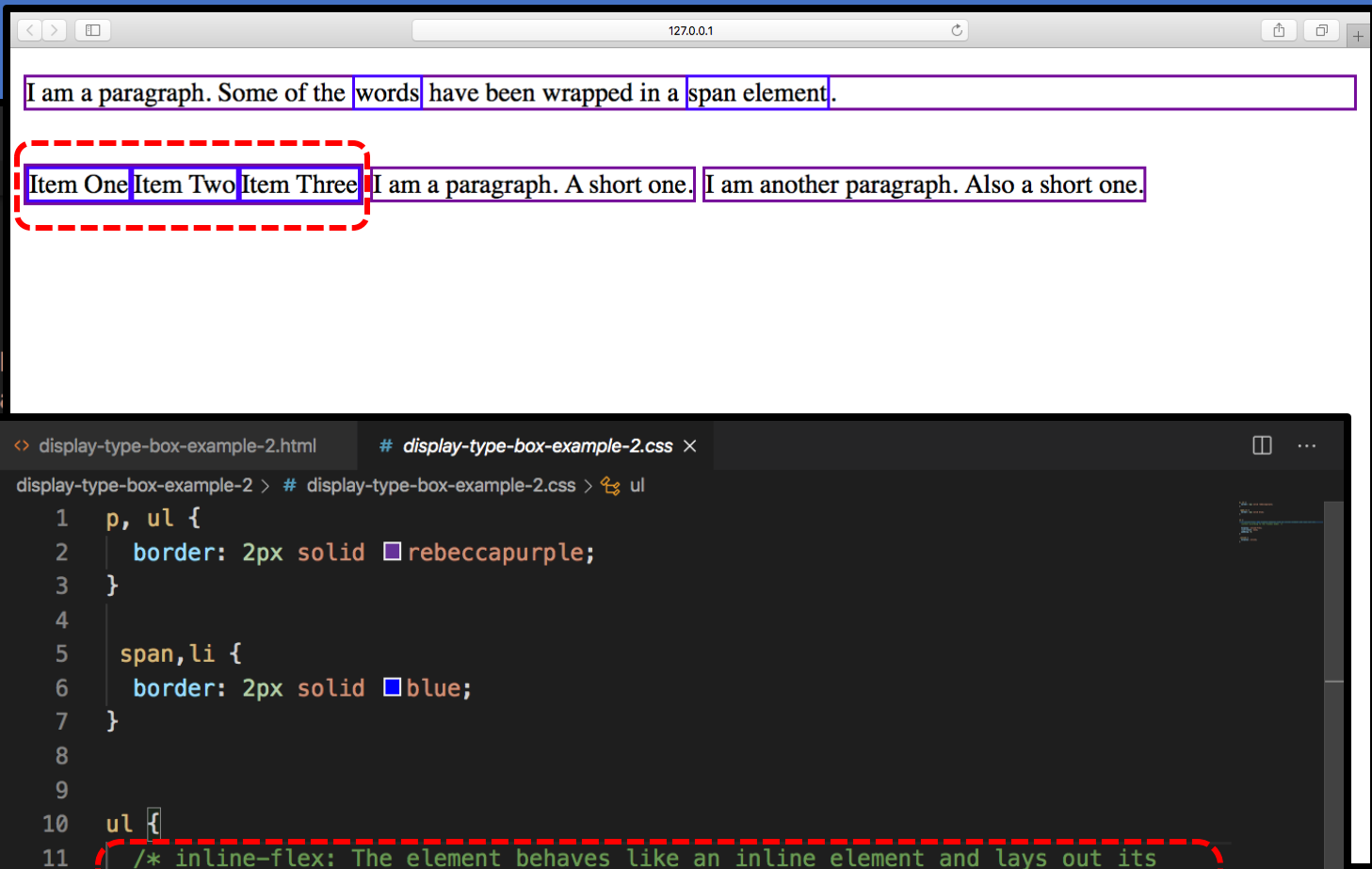


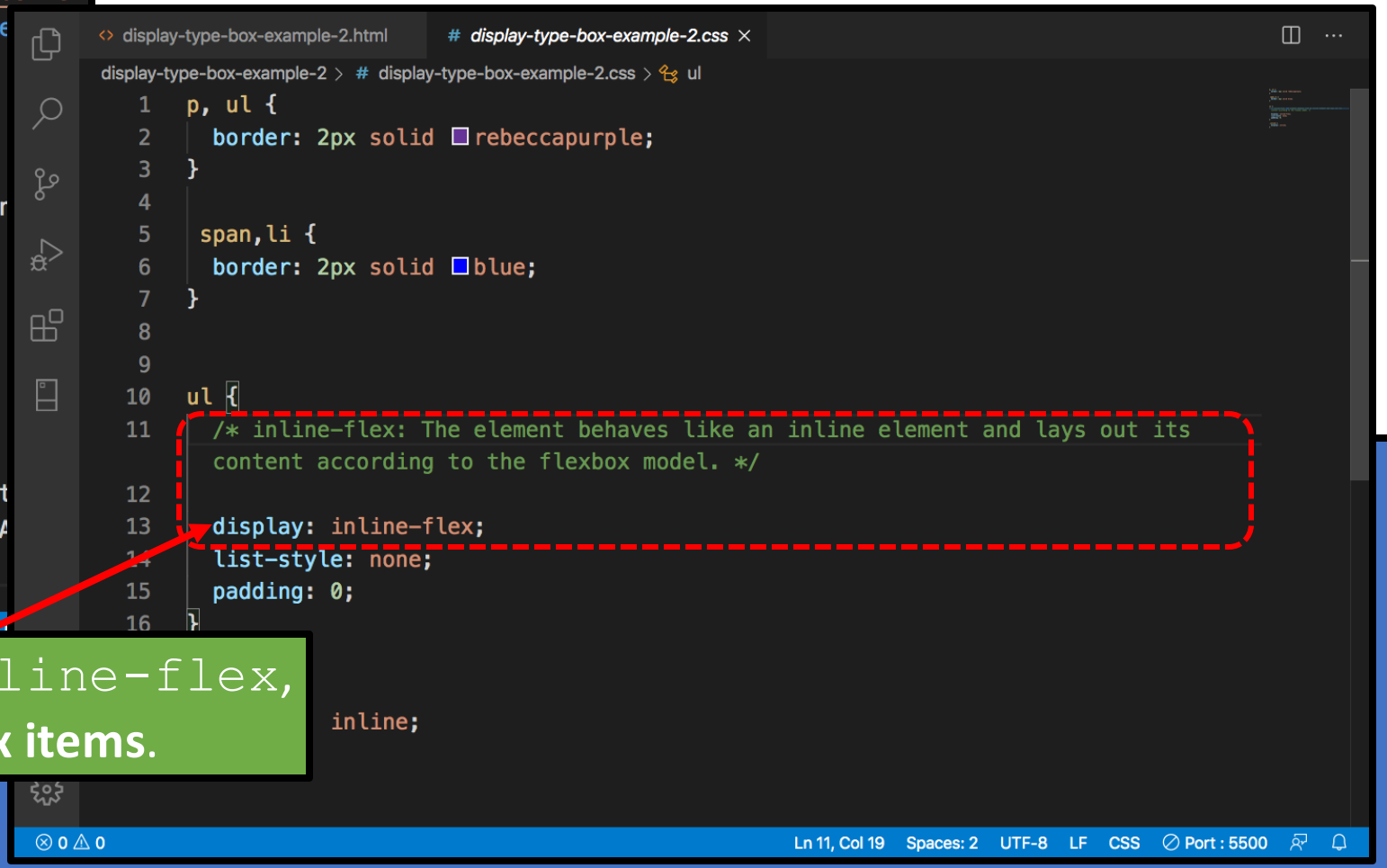
box-model/display-type-box-example-2.html

box-model/display-type-box-example-2.css

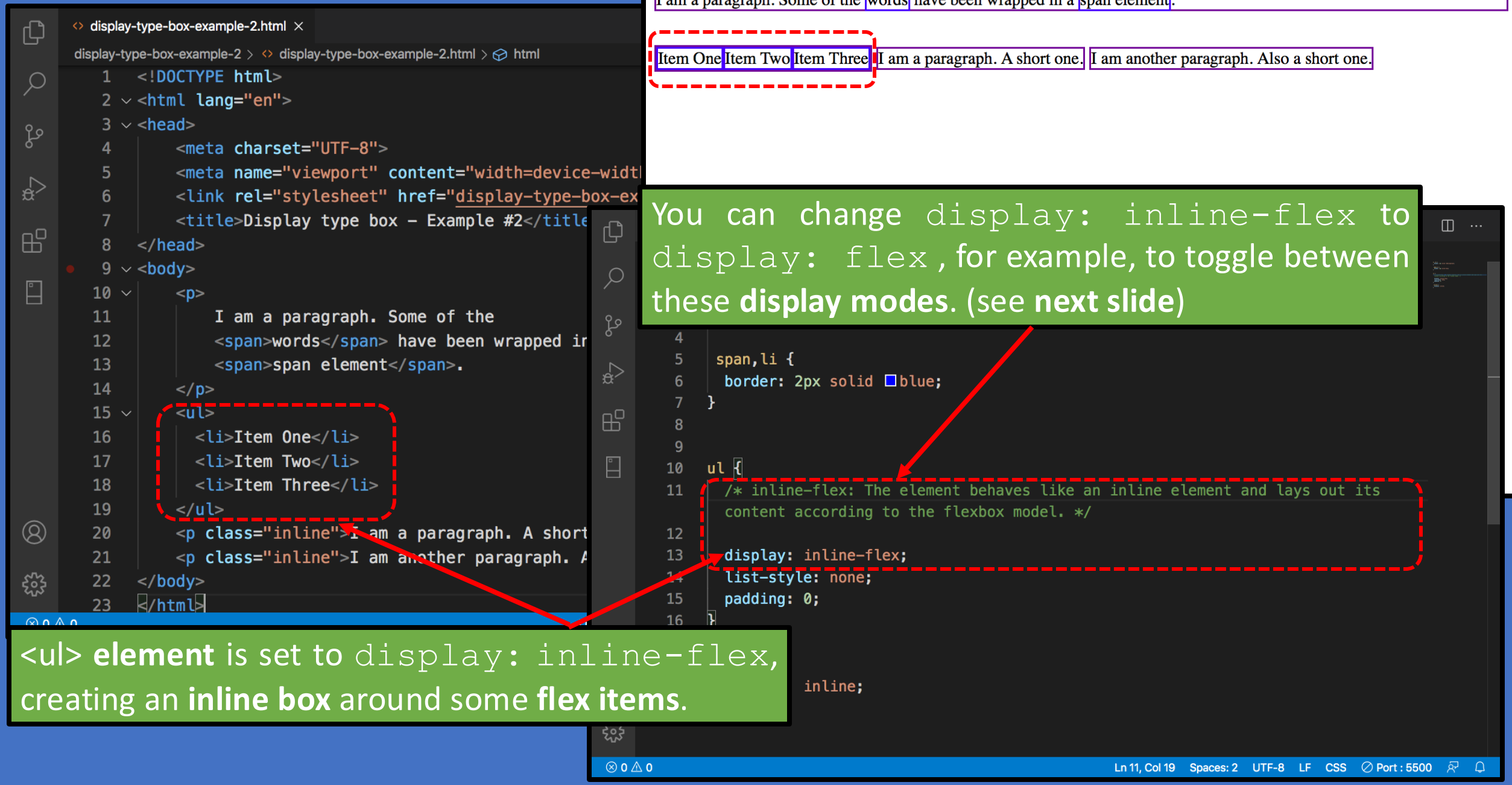

display-type-box-example-2.html
display-type-box-example-2 > display-type-box-example-2.html > html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-widt
6 <link rel="stylesheet" href="display-t
7 <title>Display type box - Example #2</
8 </head>
9 <body>
10
11 I am a paragraph. Some of the
12 words have been wrapp
13 span element.
14
15
16 Item One
17 Item Two
18 Item Three
19
20 I am a paragraph. A
21 I am another paragra
22 </body>
23 </html>
⊗ 0 ⚠ 0
127.0.0.1
I am a paragraph. Some of the words have been wrapped in a span element.
Item One Item Two Item Three I am a paragraph. A short one. I am another paragraph. Also a short one.
display-type-box-example-2.html
display-type-box-example-2.css
display-type-box-example-2 > display-type-box-example-2.css > ul
1 p, ul {
2 border: 2px solid rebeccapurple;
3 }
4
5 span,li {
6 border: 2px solid blue;
7 }
8
9
10 ul {
11 /* inline-flex: The element behaves like an inline element and lays out its
content according to the flexbox model. */
12
13 display: inline-flex;
14 list-style: none;
15 padding: 0;
16 }
17
18 .inline {
19 display: inline;
20 }
⊗ 0 ⚠ 0
Ln 11, Col 19
Spaces: 2
UTF-8
LF
CSS
Port : 5500







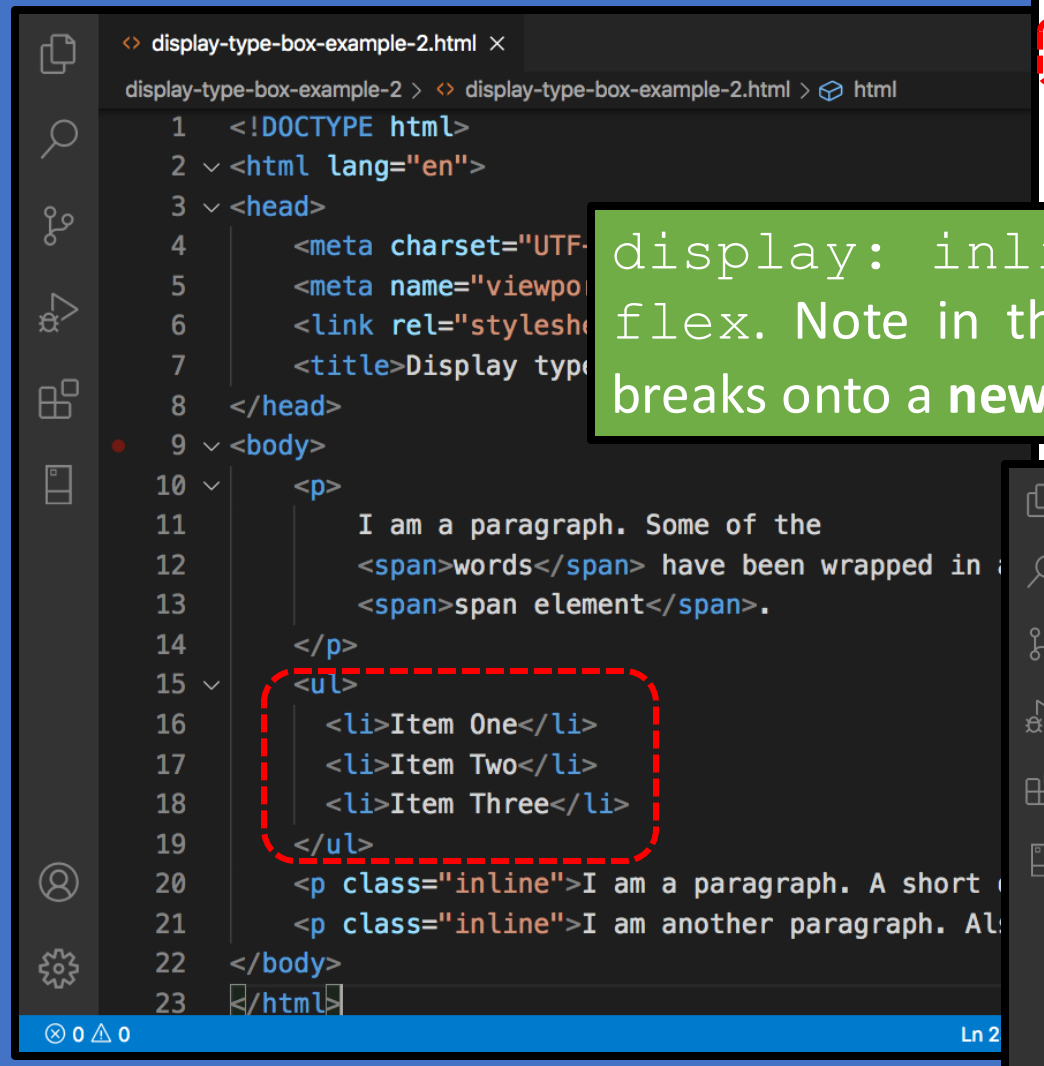
 element is set to `display: inline-flex`, creating an **inline box** around some **flex items**.

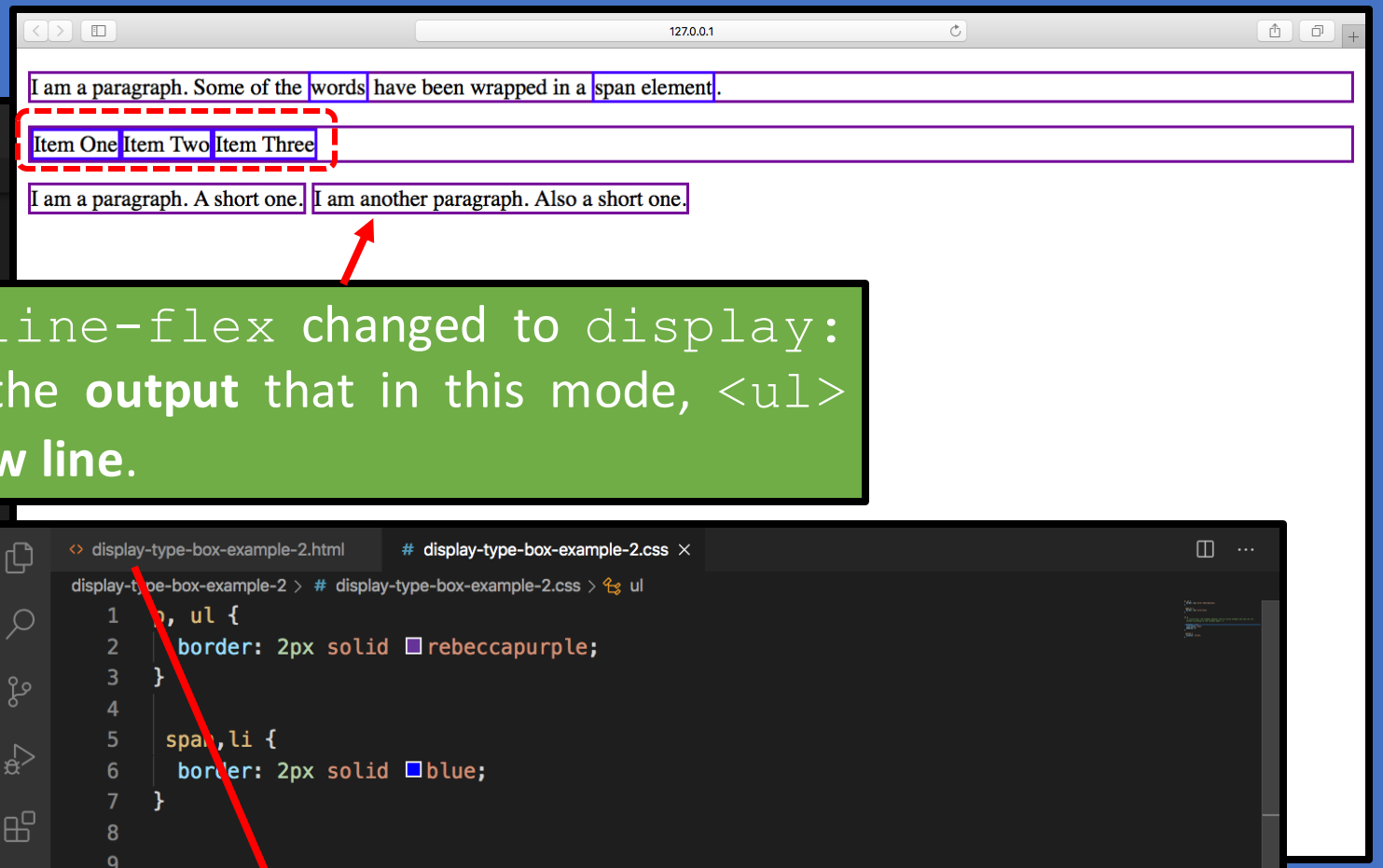

display-type-box-example-2.html
display-type-box-example-2 > display-type-box-example-2.html > html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width
<link rel="stylesheet" href="display-type-box-ex
<title>Display type box - Example #2</title
</head>
<body>

I am a paragraph. Some of the
words have been wrapped in
span element.

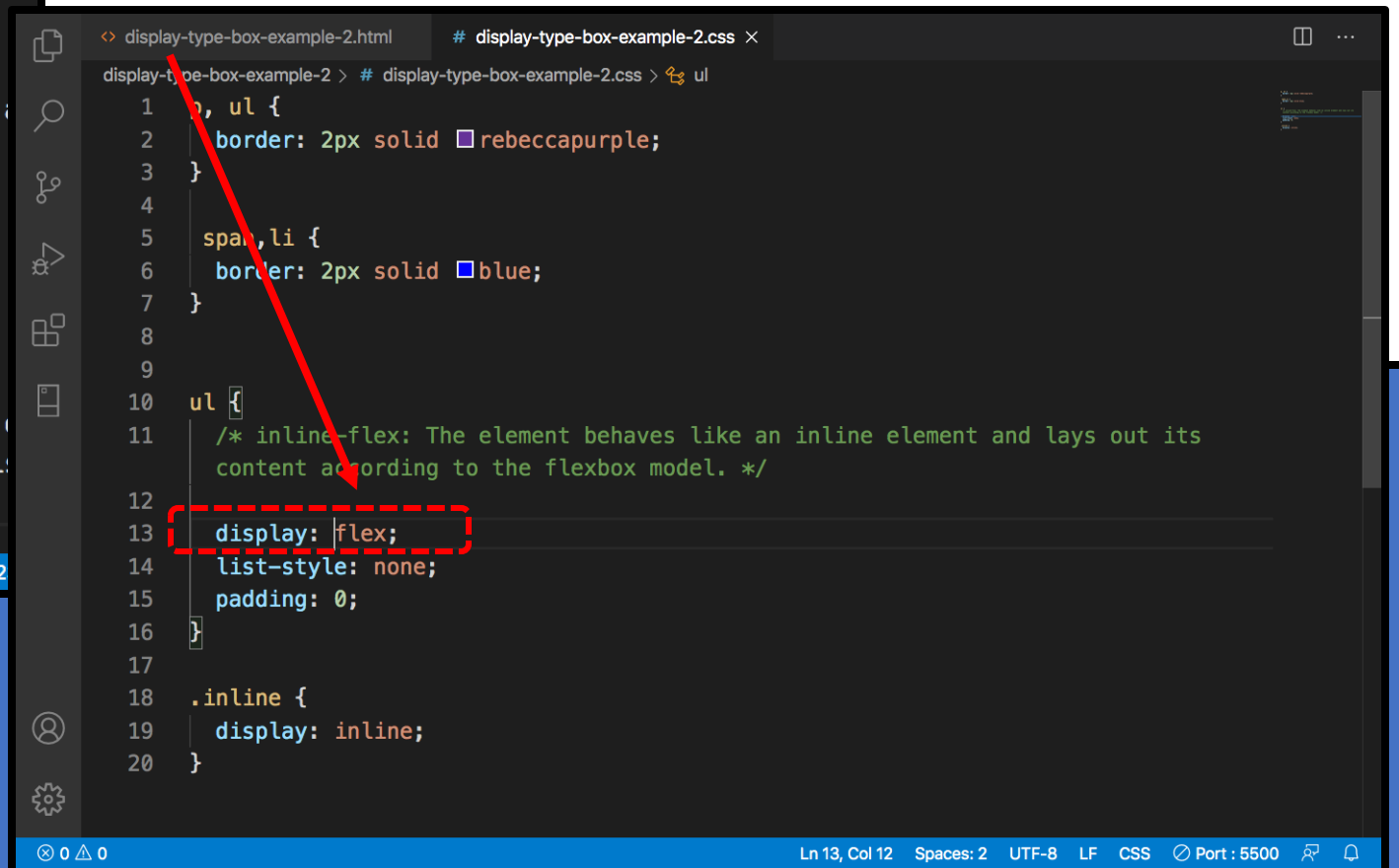
Item One
Item Two
Item Three

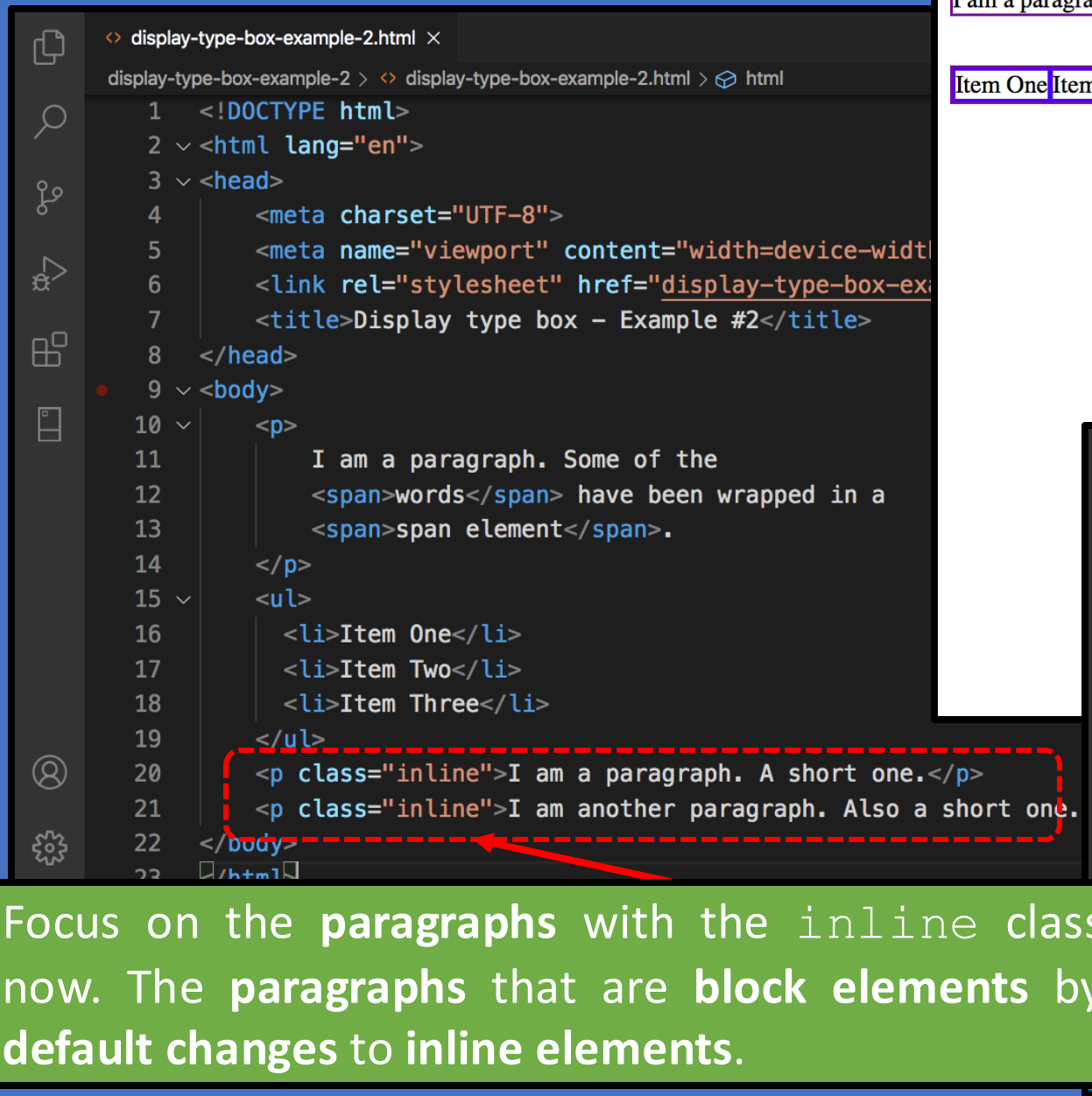
I am a paragraph. A short
I am another paragraph. A
</body>
</html>
127.0.0.1
I am a paragraph. Some of the words have been wrapped in a span element.
Item One Item Two Item Three I am a paragraph. A short one. I am another paragraph. Also a short one.
You can change display: inline-flex to display: flex, for example, to toggle between these display modes. (see next slide)
span,li {
border: 2px solid blue;
}
ul {
/* inline-flex: The element behaves like an inline element and lays out its content according to the flexbox model. */
display: inline-flex;
list-style: none;
padding: 0;
}
inline;
 element is set to display: inline-flex, creating an inline box around some flex items.
Ln 11, Col 19 Spaces: 2 UTF-8 LF CSS Port : 5500

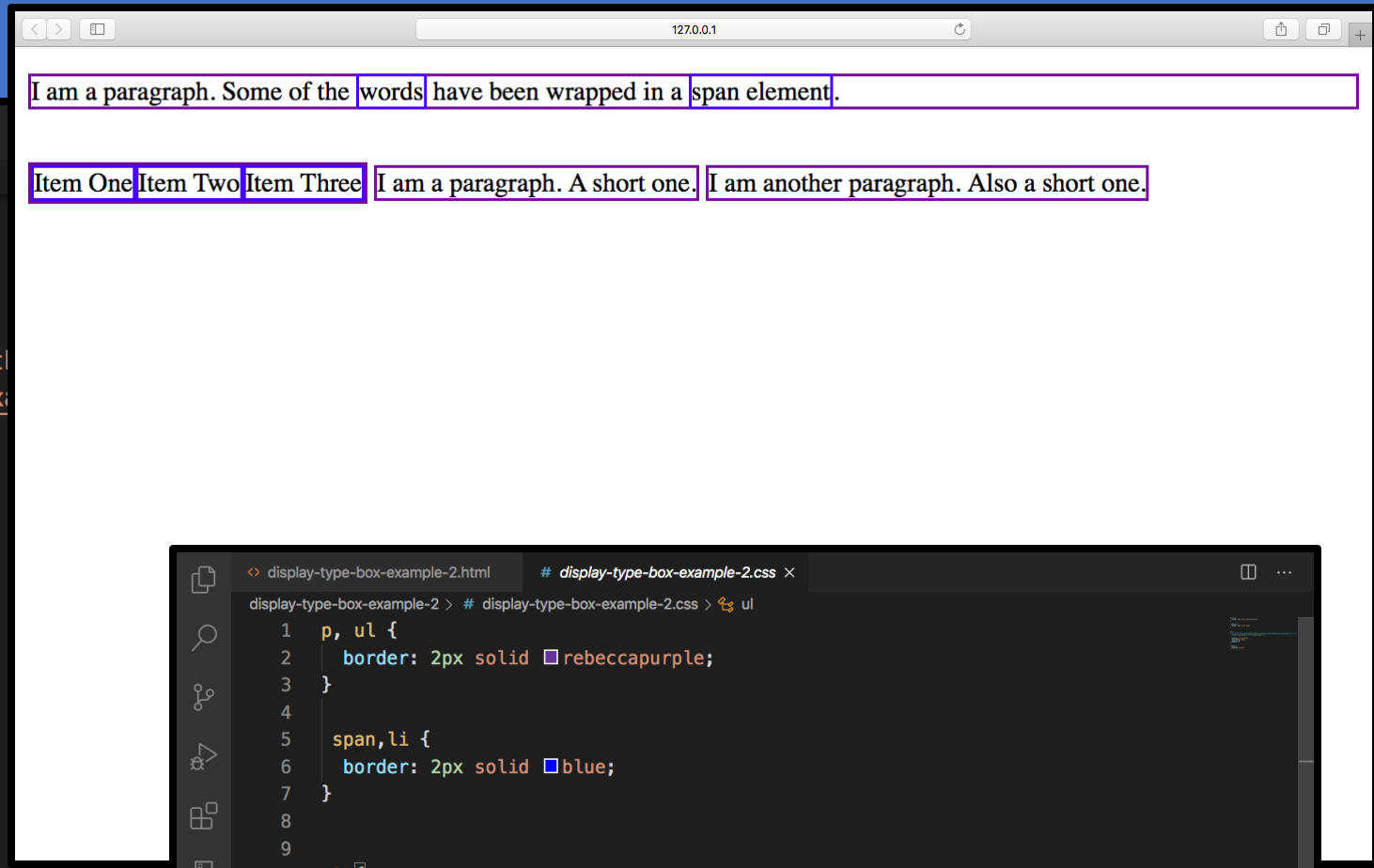


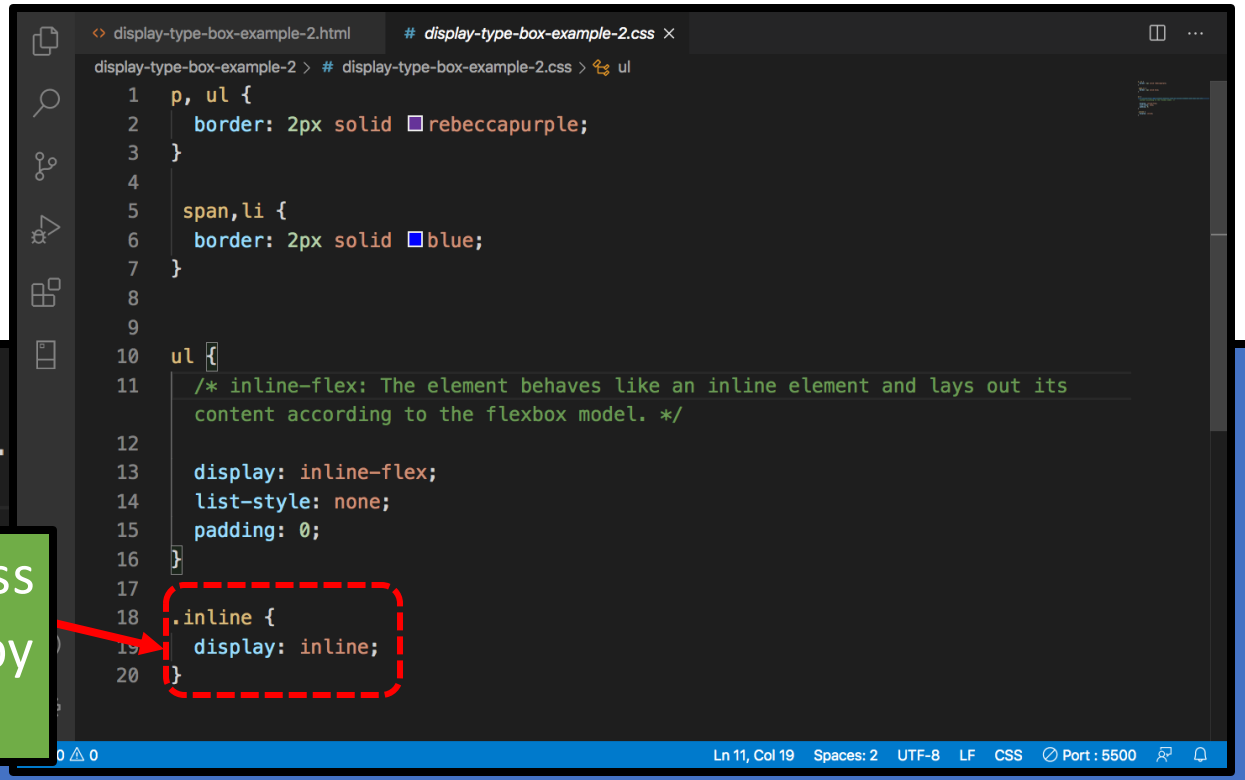


`display: inline-flex` changed to `display: flex`. Note in the **output** that in this mode, `<ul>` breaks onto a **new line**.

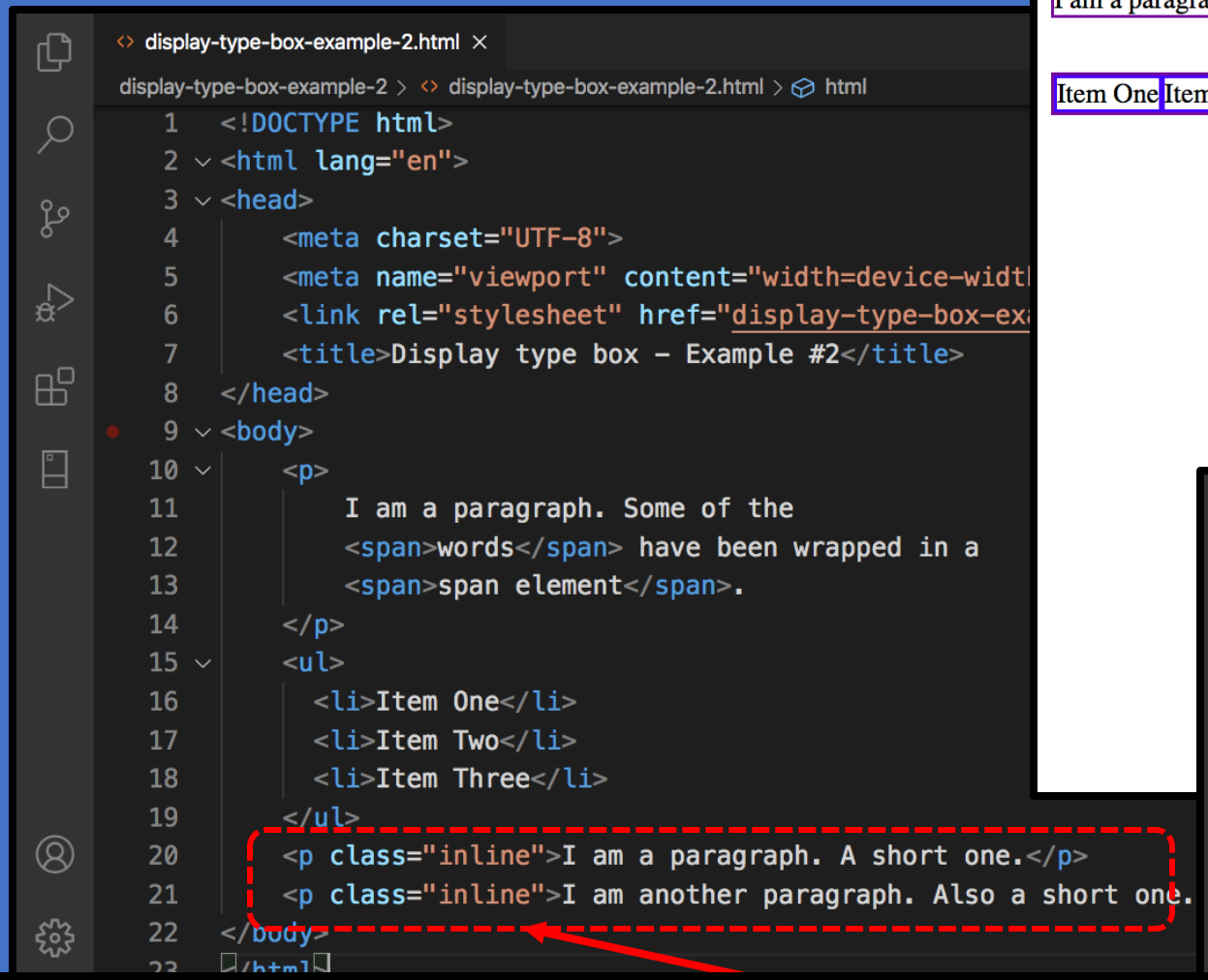


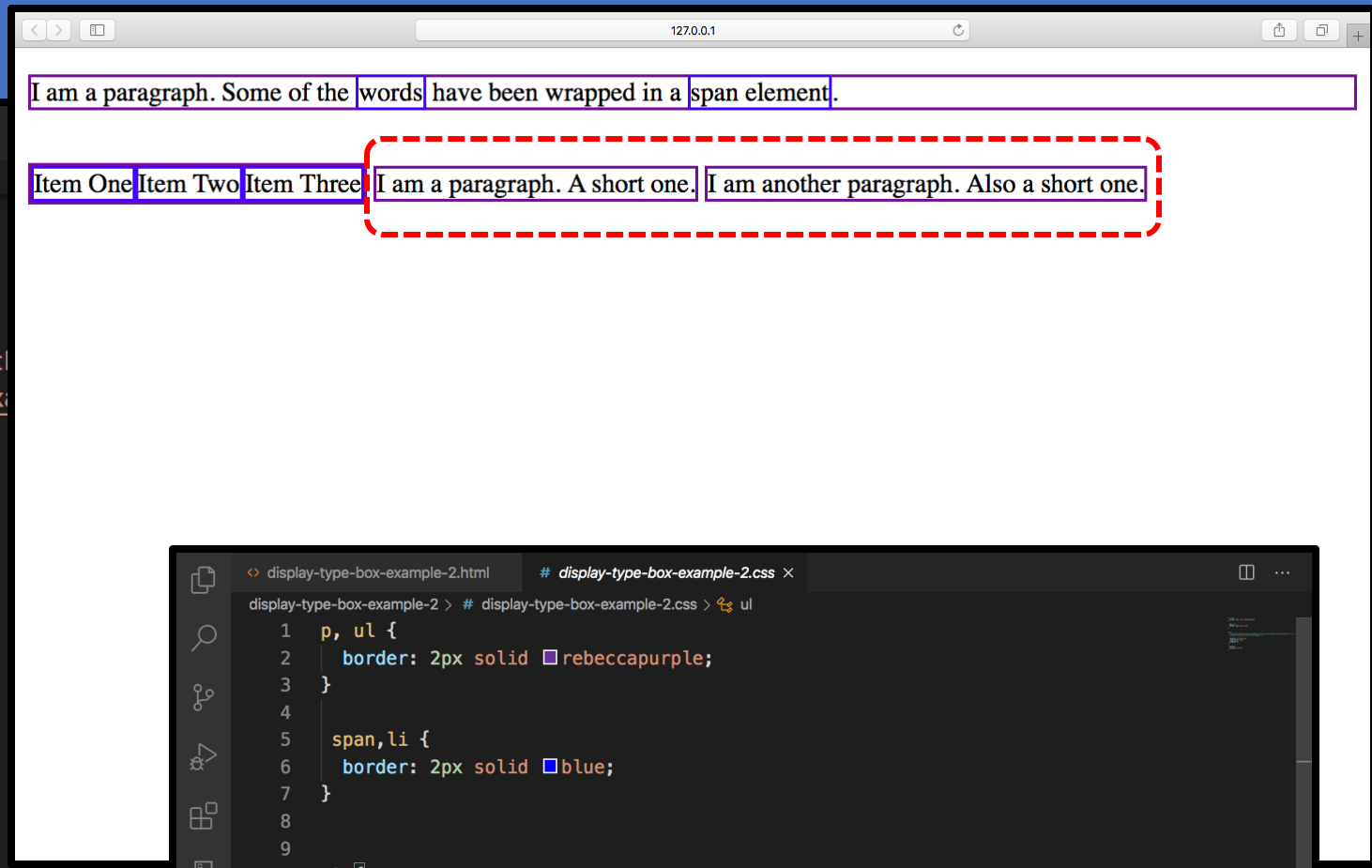


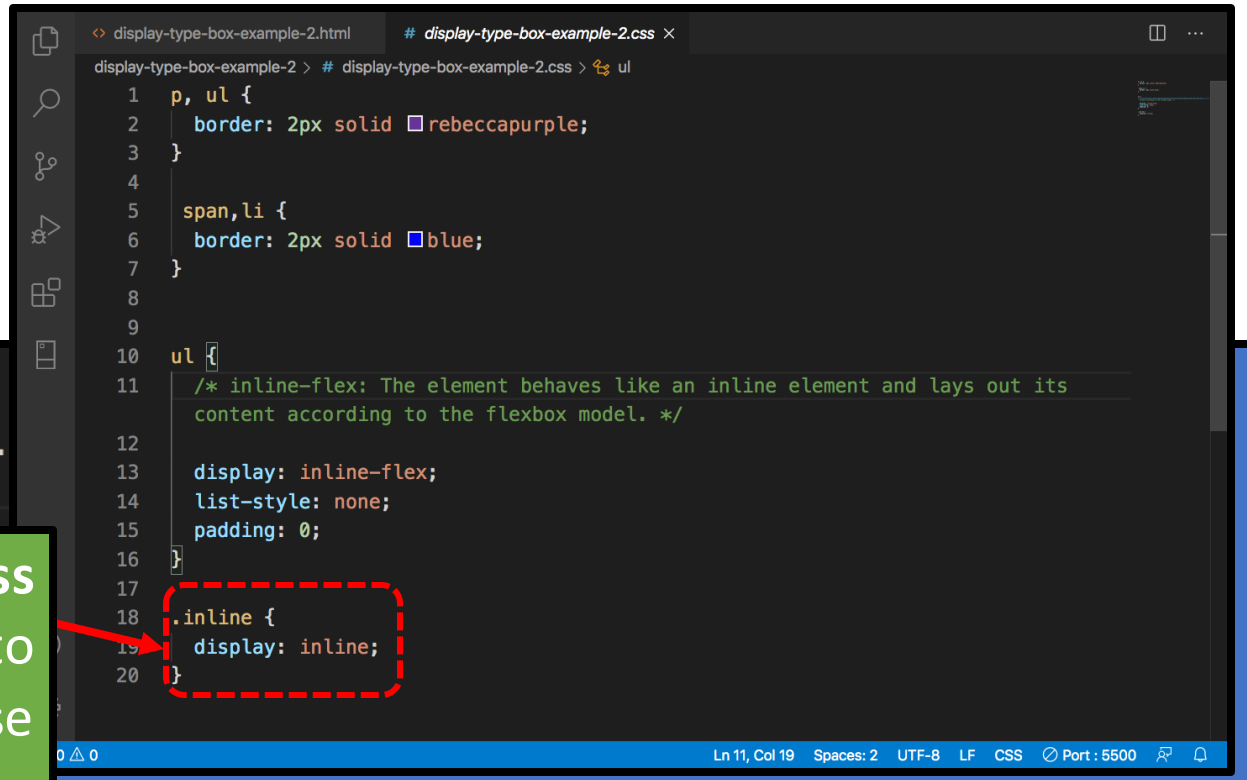




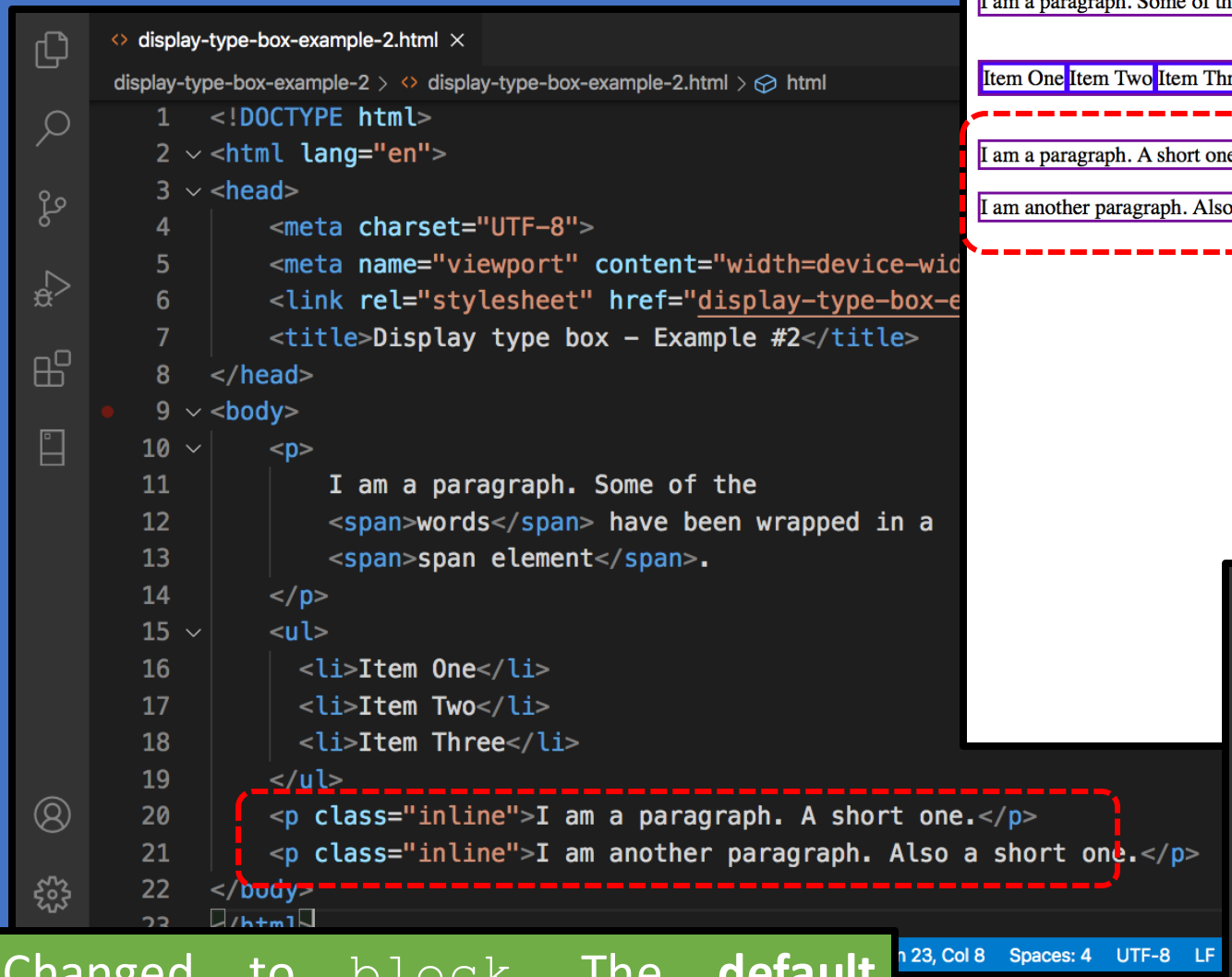
Focus on the **paragraphs** with the `inline` class now. The **paragraphs** that are **block elements** by **default changes** to **inline elements**.

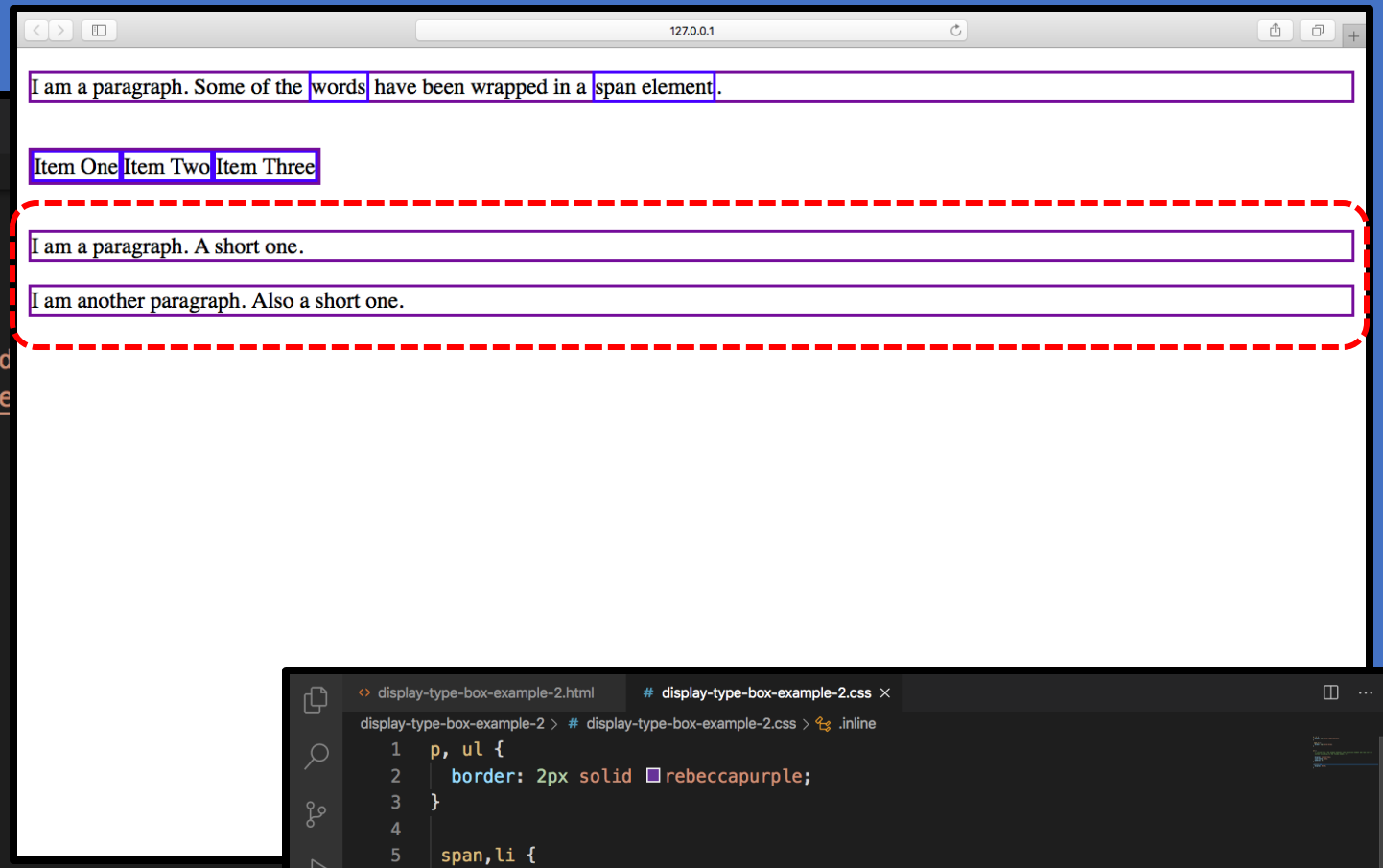


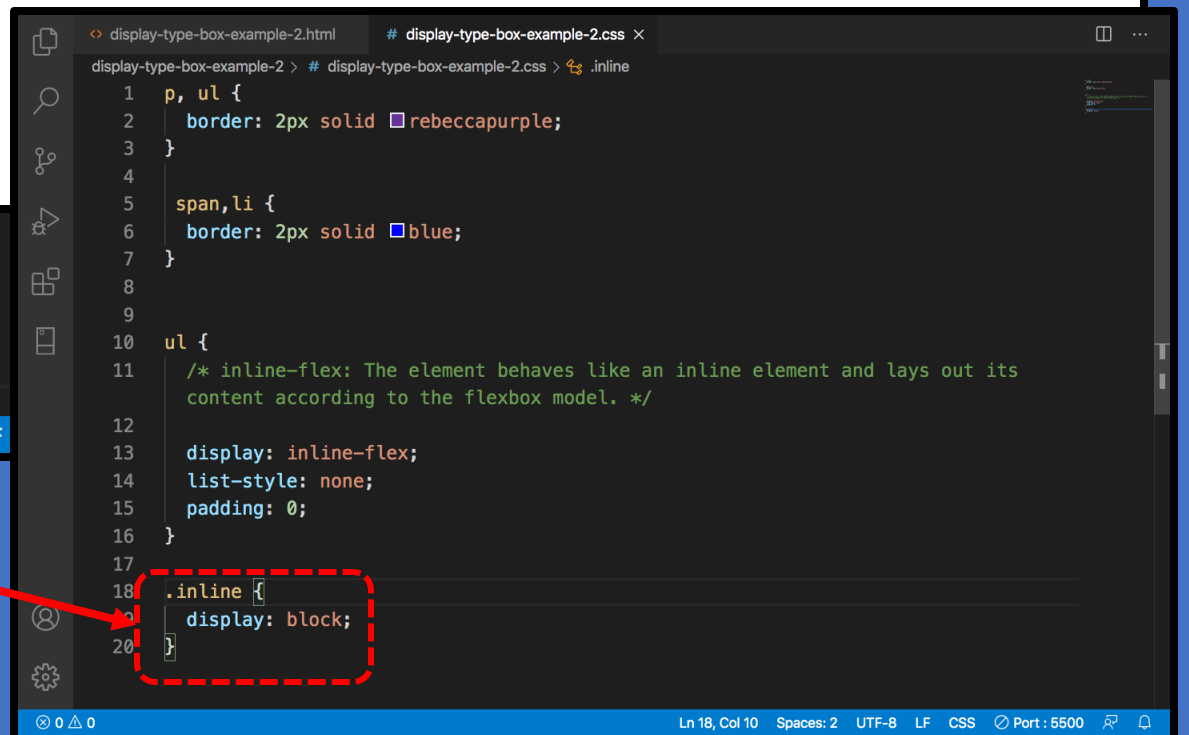




Focus on the **paragraphs** with the `inline` **class** now. You can **change** `display: inline` to `display: block` to **toggle between** these **display modes**.







Changed to `block`. The **default behavior** for `<p>` **elements**.

Flexbox

On this Page

- Why Flexbox?
- Introducing a simple example
- Specifying what elements to lay out as flexible boxes
- The flex model
- Columns or rows?
- Wrapping
- flex-flow shorthand
- Flexible sizing of flex items
- flex: shorthand versus longhand
- Horizontal and vertical alignment
- Ordering flex items
- Nested flex boxes
- Cross-browser compatibility
- Test your skills!
- Summary
- In this module

Related Topics

← Previous ↑ Overview: CSS layout Next →

Flexbox is a one-dimensional layout method for laying out items in rows or columns. Items flex to fill additional space and shrink to fit into smaller spaces. This article explains all the fundamentals.

Prerequisites:	HTML basics (study Introduction to HTML), and an idea of how CSS works (study Introduction to CSS.)
Objective:	To learn how to use the Flexbox layout system to create web layouts.

Why Flexbox?

For a long time, the only reliable cross browser-compatible tools available for creating CSS layouts were things like floats and positioning. These are fine, and they are also rather limiting and frustrating.

The following simple layout requirements are either difficult or impossible to achieve with such

https://mzl.la/32Bdzwj

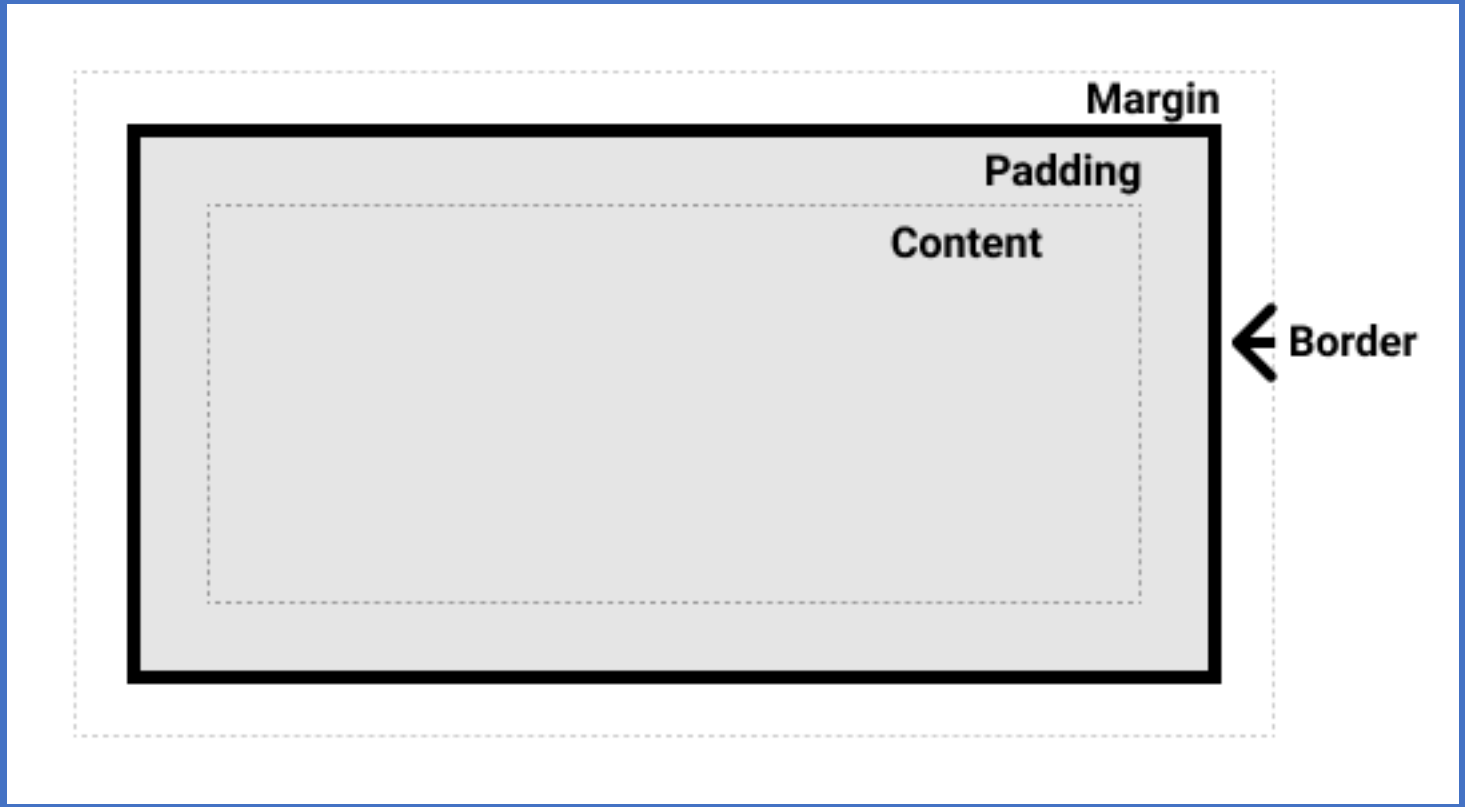
The **key thing to remember** is that **changing** the **value** of the `display` **property** can change whether the **outer display type** of a **box** is **block** or **inline**, which **changes** the **way** it **displays alongside** other **elements** in the **layout**.

What is the **CSS box model**?

What is the CSS box model?

- The **full CSS box model** applies to **block boxes**
  - ✓ **Inline boxes** only use **some** of **the behavior** defined in the **box model**.
- The **model defines how** the different parts of a box **work together** to **create** a **box** that you can **see** on the **page**.
  - ✓**`margin`**, **`border`**, **`padding`**, and **`content`**.
- To add **additional complexity**, there is a **standard** and an **alternate box model**.

Parts of a box



The standard CSS box model

- In the **standard box model**, if you give a **box** a `width` and a `height` **properties**, this defines the `width` and `height` of the **content box**.
  - ✓Any **`padding`** and **`border`** is then **added** to that **`width`** and **`height`** to get the **total size** taken up by the **box**.

```
1  .box {
2    width: 350px;
3    height: 150px;
4    margin: 10px;
5    padding: 25px;
6    border: 5px solid black;
7  }
```

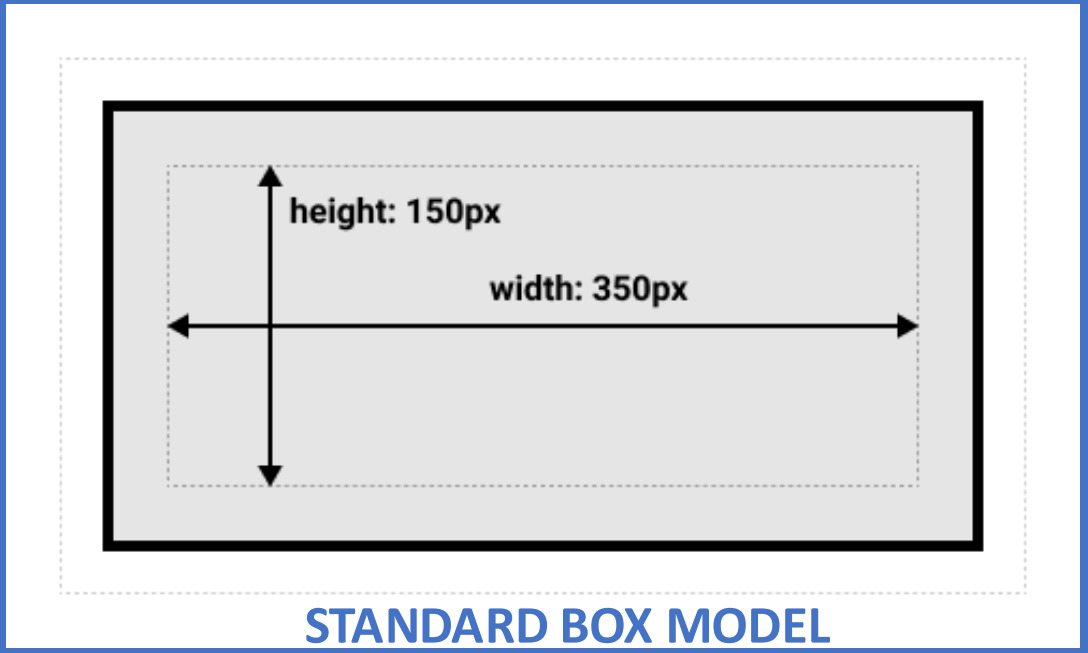


STANDARD BOX MODEL

The standard CSS box model

- The **space** taken up by our **box**:
  - ✓**Width**: 410px (350 + 25 + 25 + 5 + 5)
  - ✓**Height**: 210px (150 + 25 + 25 + 5 + 5)

```
1 .box {
2   width: 350px;
3   height: 150px;
4   margin: 10px;
5   padding: 25px;
6   border: 5px solid black;
7 }
```



Note: Does margin count?

Note: The **margin** is **NOT counted** towards the **actual size of the box.** it **affects** the **total space** that the **box takes up** on the **page**, but **only** the **space outside** the **box**. **The box's area stops at the border** — it **does not extend** into the **margin**.

The alternative CSS box model

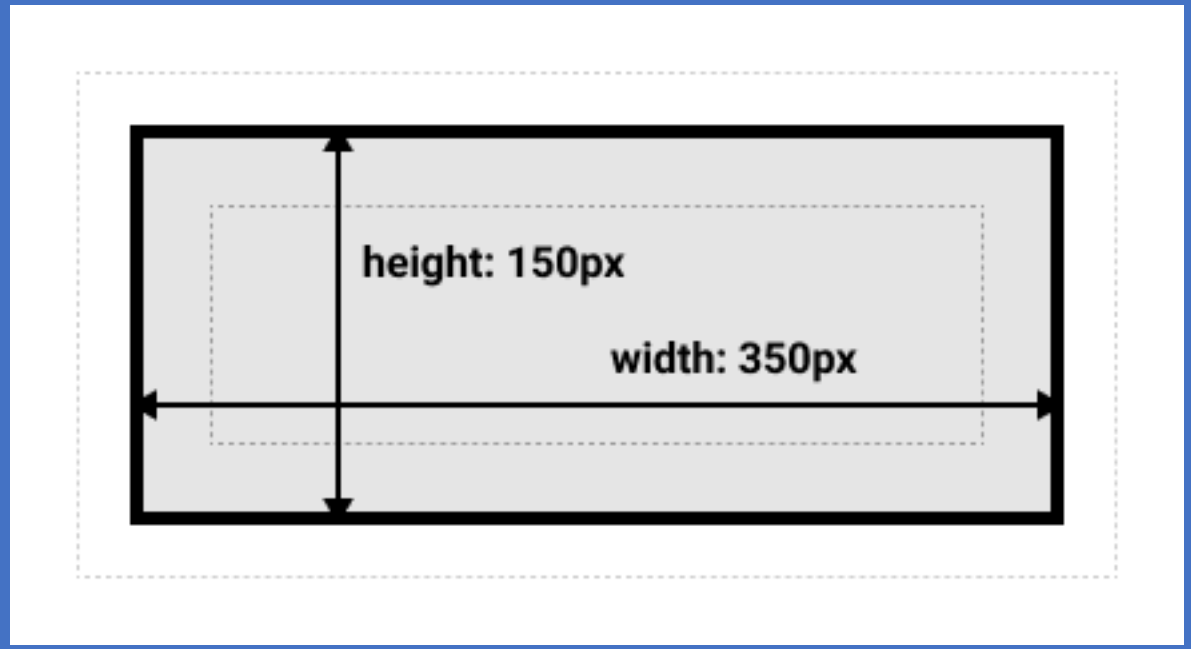
- You might think it is rather **inconvenient** to have to **add up** the **`border`** and **`padding`** to get the **real size** of the **box**.

- For this reason, **CSS** had an **alternative box model introduced** some time **after** the **standard box model**.
  - ✓Using the **alternative box model**, any **width** is the **width** of the **visible box** on the **page**.

The alternative CSS box model

- The **same CSS** used **previously** would give the below **result**:
  - ✓ `width = 350px, height = 150px.`

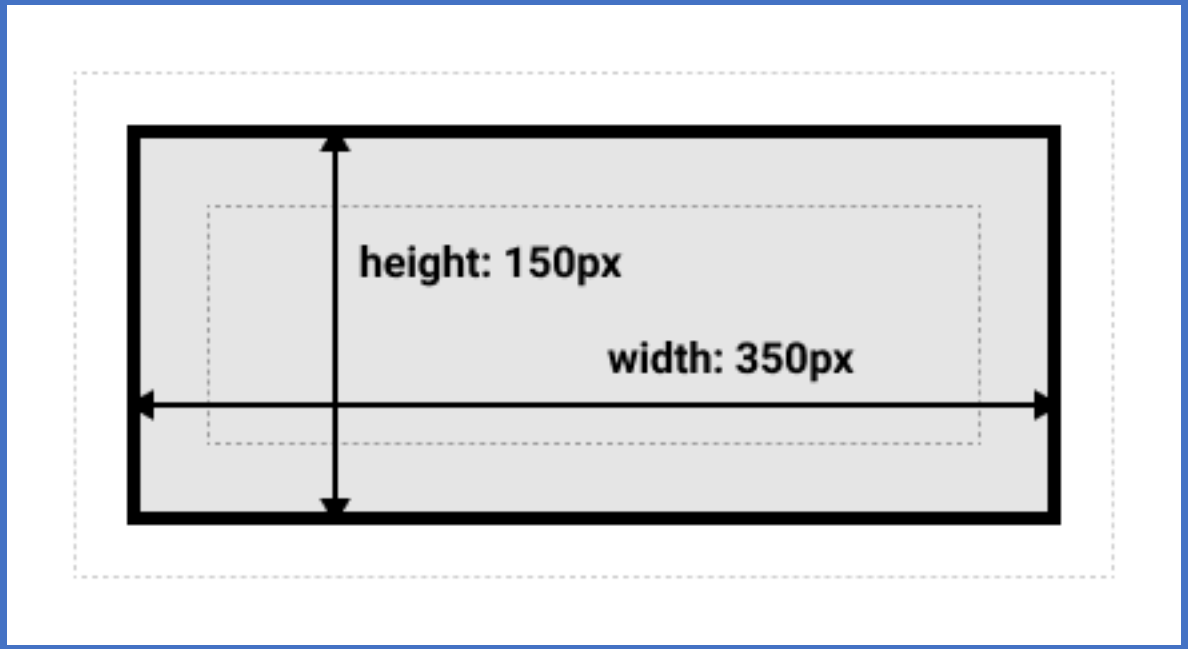
```
1  .box {
2    width: 350px;
3    height: 150px;
4    margin: 10px;
5    padding: 25px;
6    border: 5px solid black;
7  }
```



The alternative CSS box model

- By **default**, **browsers** use the **standard box model**.
- If you want to turn on the **alternative model** for an **element** you do so by **setting** `box-sizing: border-box` on it.

```
1 .box {
2   box-sizing: border-box;;
3 }
```

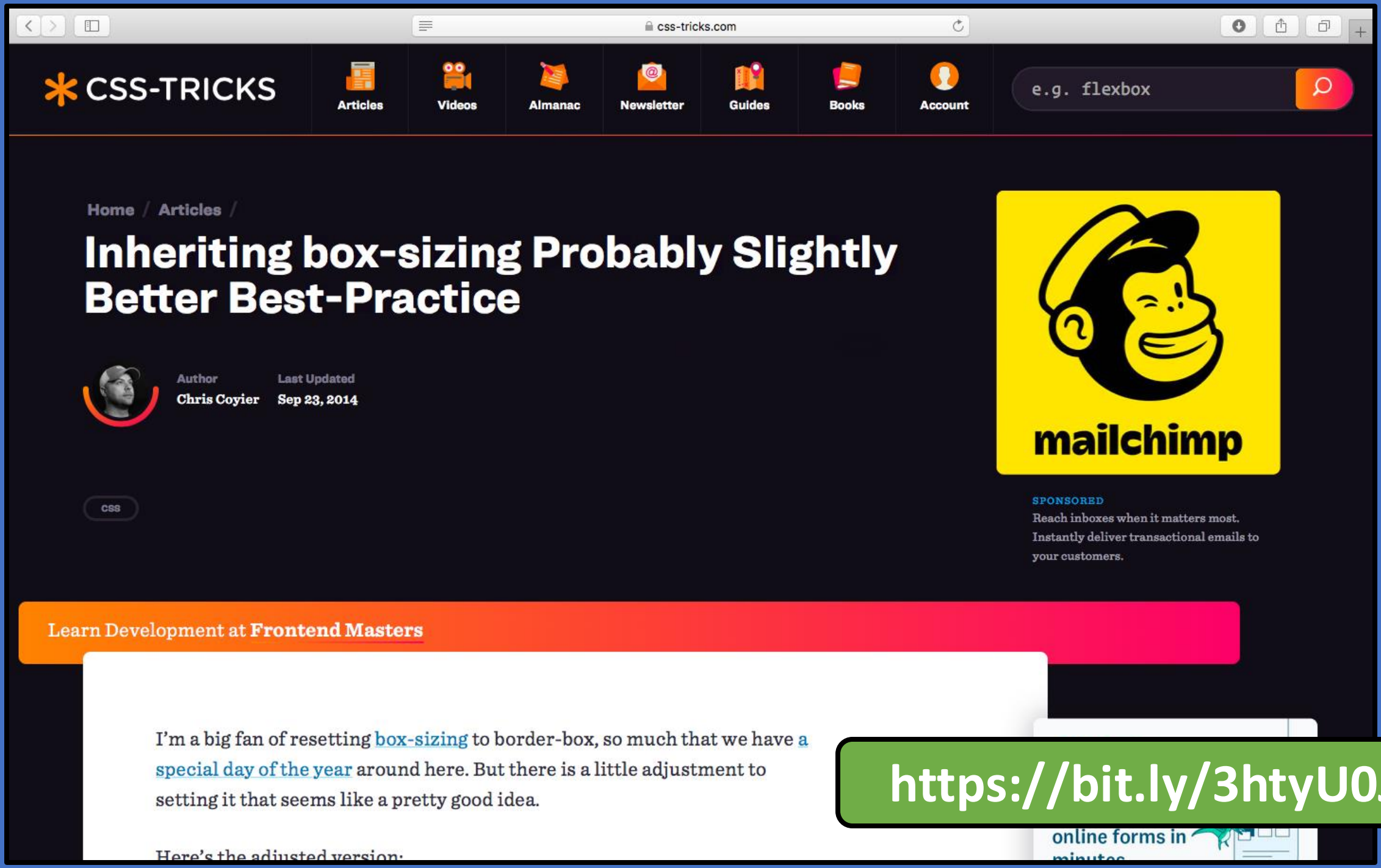


The alternative CSS box model

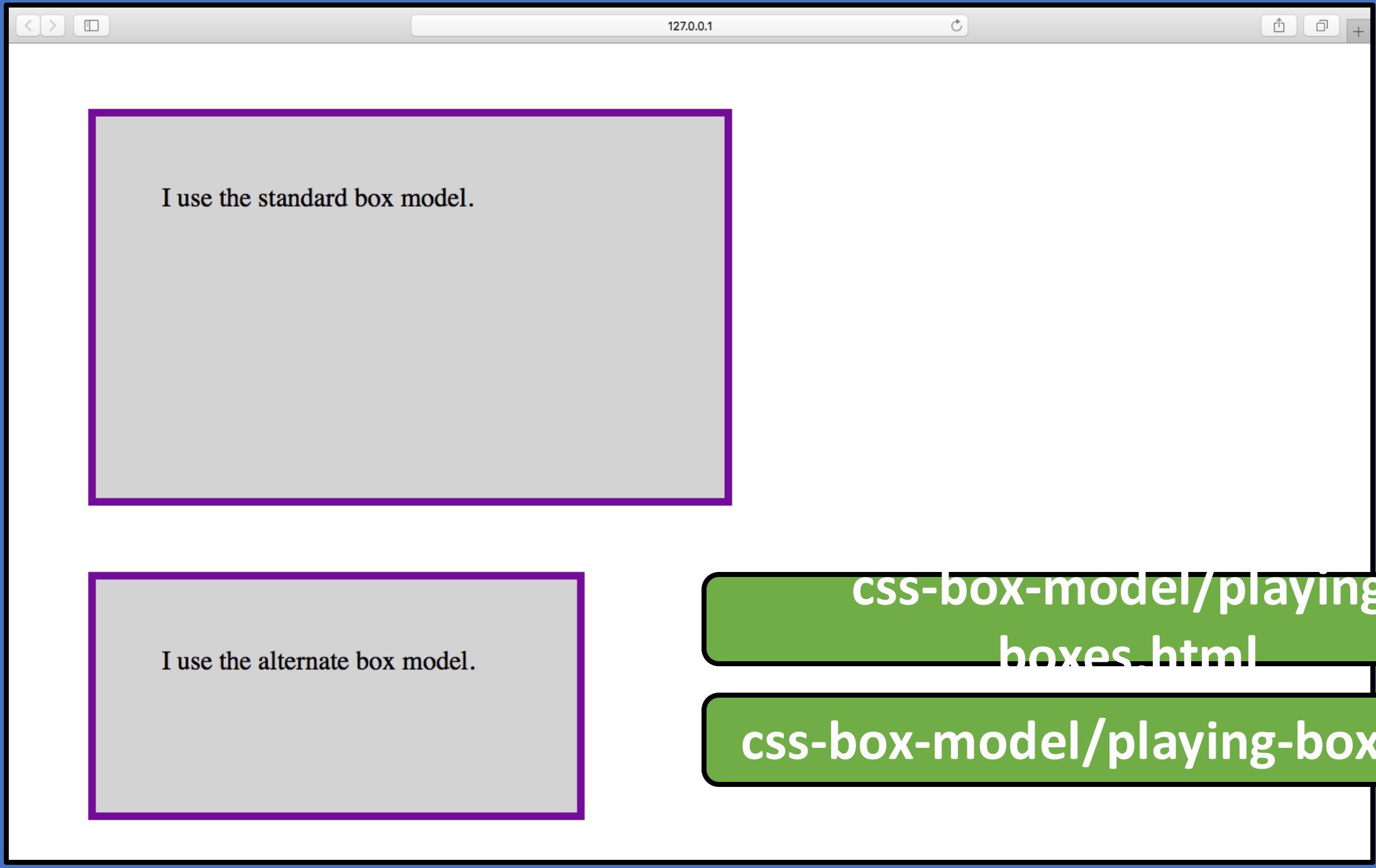
- If you want **all** elements to **use** the **alternative box model:**
  1. Set the `box-sizing property` on the `<html>` **element**
  2. Set **all** other **elements** to **inherit** that **value**.

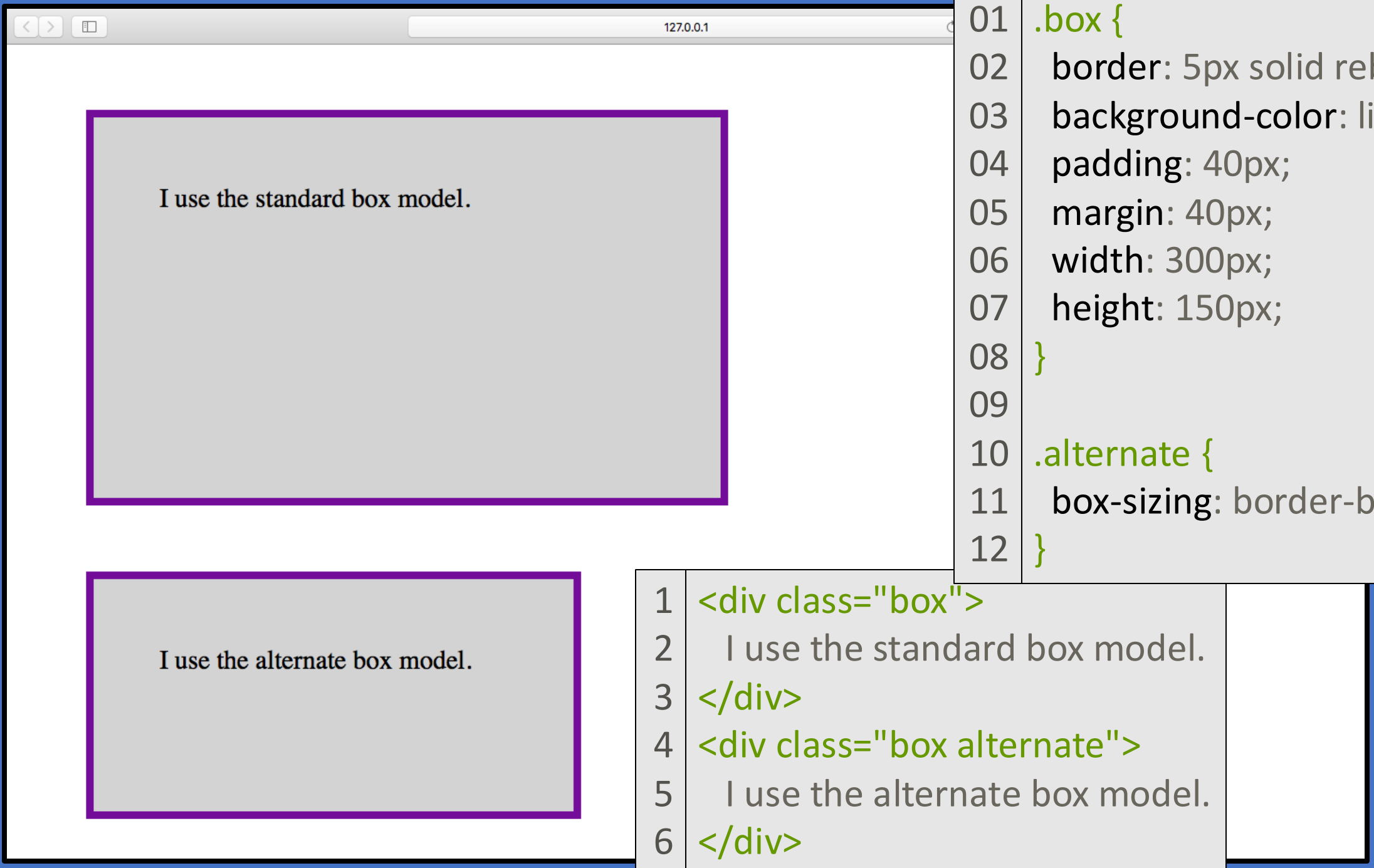
- **Common choice** among **developers**.

```
1 html {
2   box-sizing: border-box;
3 }
4 *, *::before, *::after {
5   box-sizing: inherit;
6 }
```


css-tricks.com
CSS-TRICKS
Articles
Videos
Almanac
Newsletter
Guides
Books
Account
e.g. flexbox
Home / Articles /
Inheriting box-sizing Probably Slightly Better Best-Practice
Author
Chris Coyier
Last Updated
Sep 23, 2014
CSS
mailchimp
SPONSORED
Reach inboxes when it matters most. Instantly deliver transactional emails to your customers.
Learn Development at Frontend Masters
I'm a big fan of resetting box-sizing to border-box, so much that we have a special day of the year around here. But there is a little adjustment to setting it that seems like a pretty good idea.
Here's the adjusted version:
online forms in
https://bit.ly/3htyU0J

Playing with box models

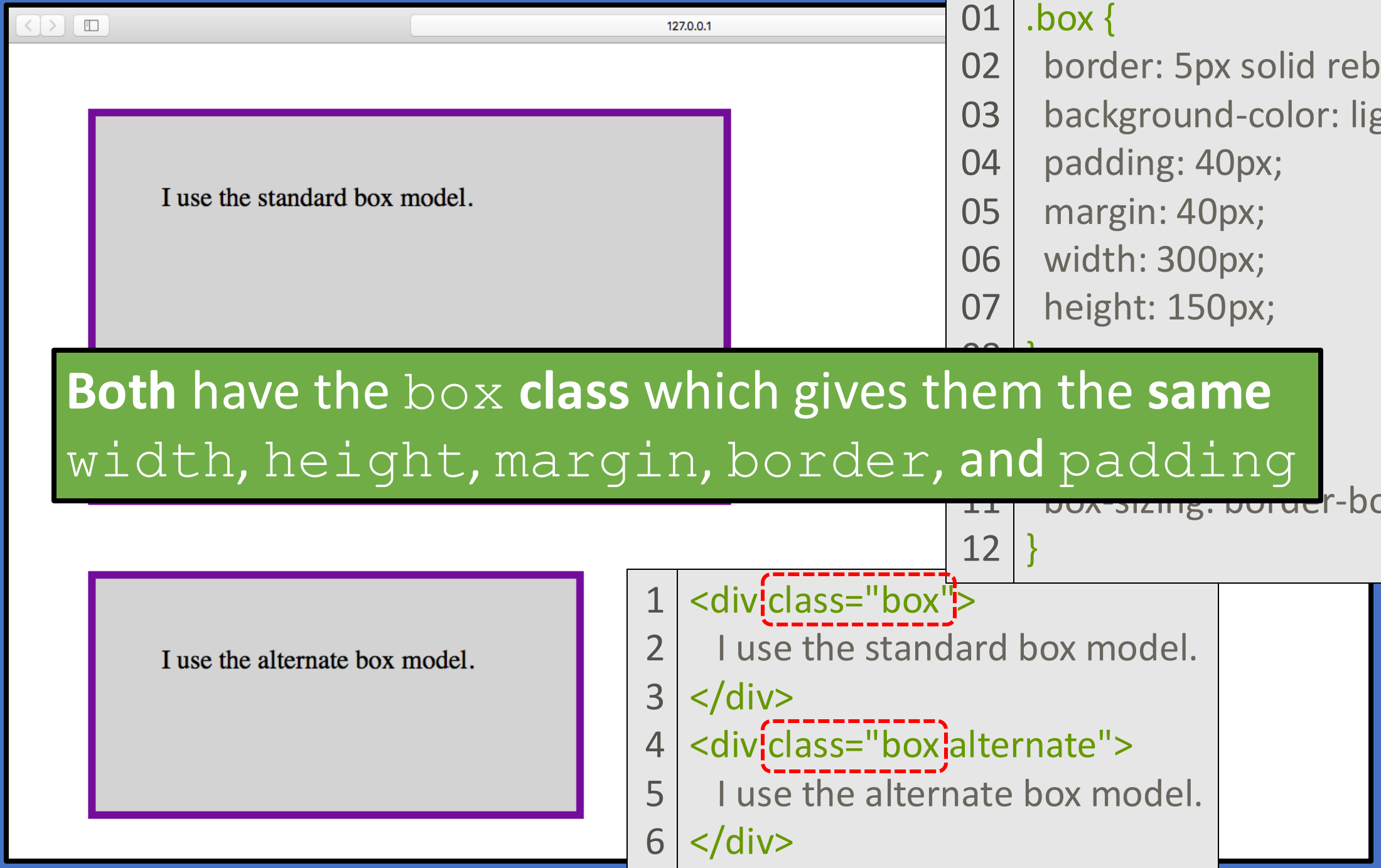

127.0.0.1
I use the standard box model.
I use the alternate box model.
css-box-model/playing-boxes.html
css-box-model/playing-boxes.css




```css
.box {
  border: 5px solid rebeccapurple;
  background-color: lightgray;
  padding: 40px;
  margin: 40px;
  width: 300px;
  height: 150px;
}

.alternate {
  box-sizing: border-box;
}
```

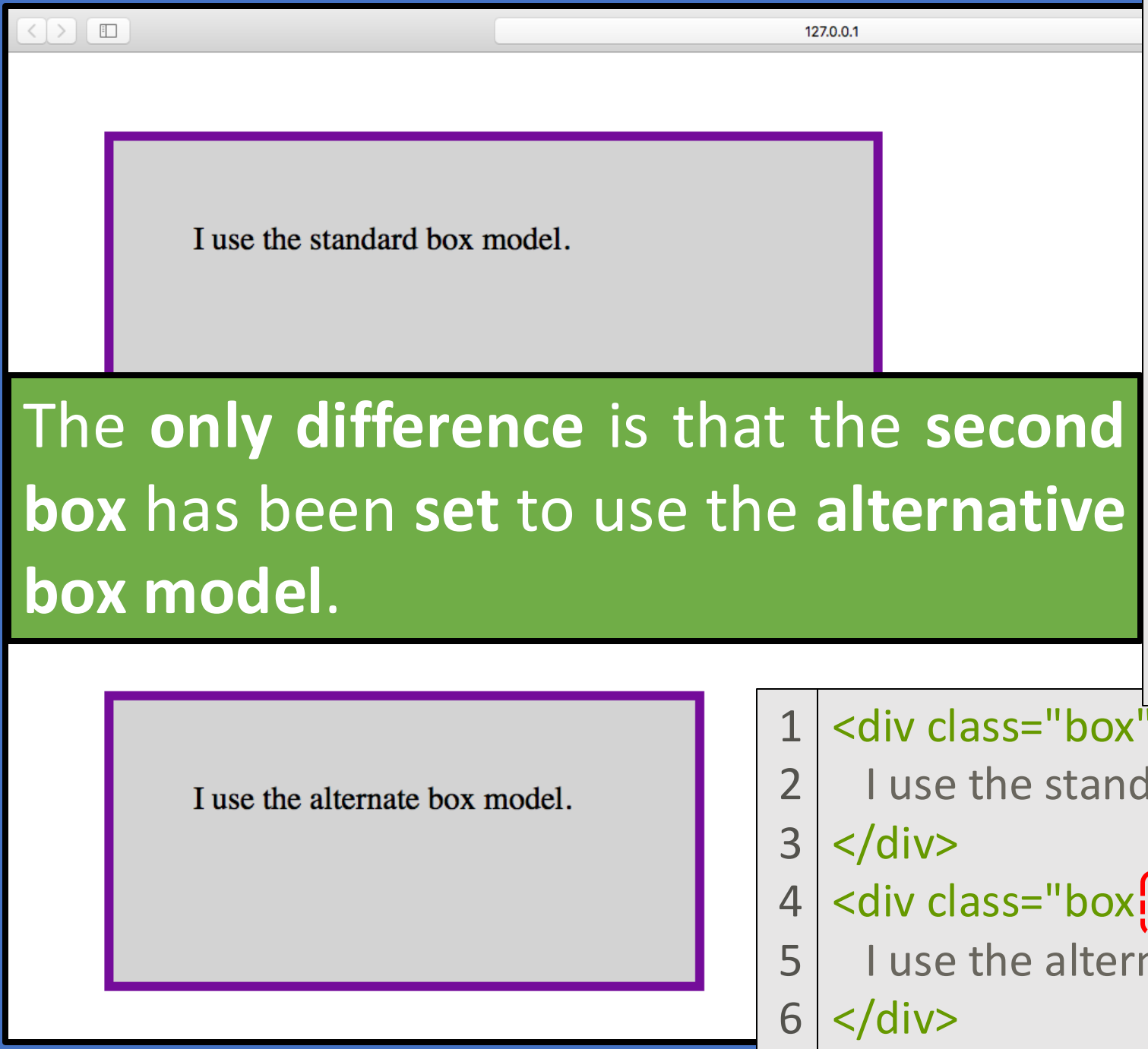
```html
<div class="box">
  I use the standard box model.
</div>
<div class="box alternate">
  I use the alternate box model.
</div>
```




```
01 .box {
02   border: 5px solid rebeccapurple;
03   background-color: lightgray;
04   padding: 40px;
05   margin: 40px;
06   width: 300px;
07   height: 150px;
11   box-sizing: border-box;
12 }
```

Both have the `box` **class** which gives them the **same** `width`, `height`, `margin`, `border`, **and** `padding`

```
1 <div class="box">
2   I use the standard box model.
3 </div>
4 <div class="box alternate">
5   I use the alternate box model.
6 </div>
```

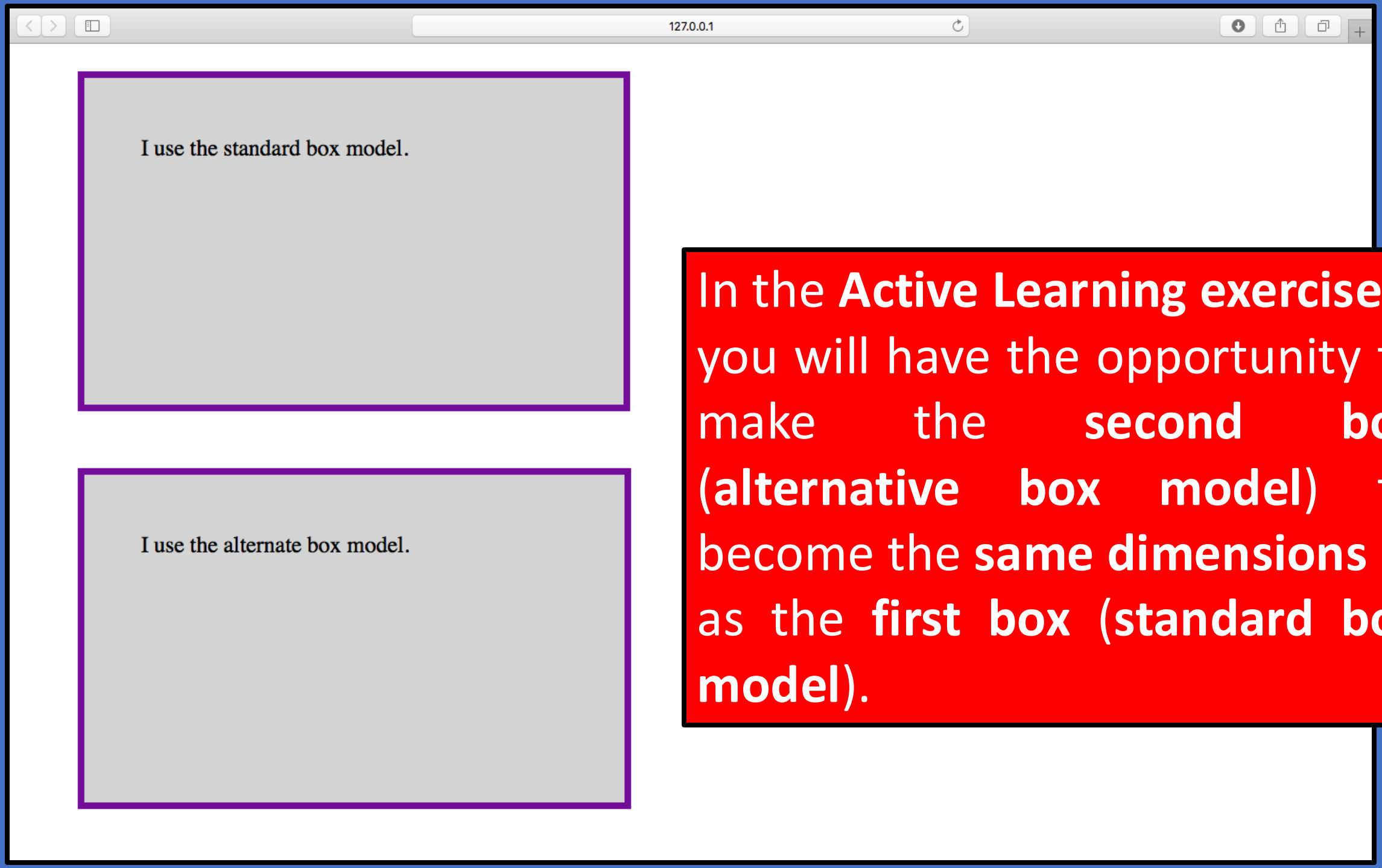


The **only difference** is that the **second box** has been **set** to use the **alternative box model**.

```
.box {
  border: 5px solid rebeccapurple;
  background-color: lightgray;
  padding: 40px;
  margin: 40px;
  width: 300px;
  height: 150px;
}

.alternate {
  box-sizing: border-box;
}
```

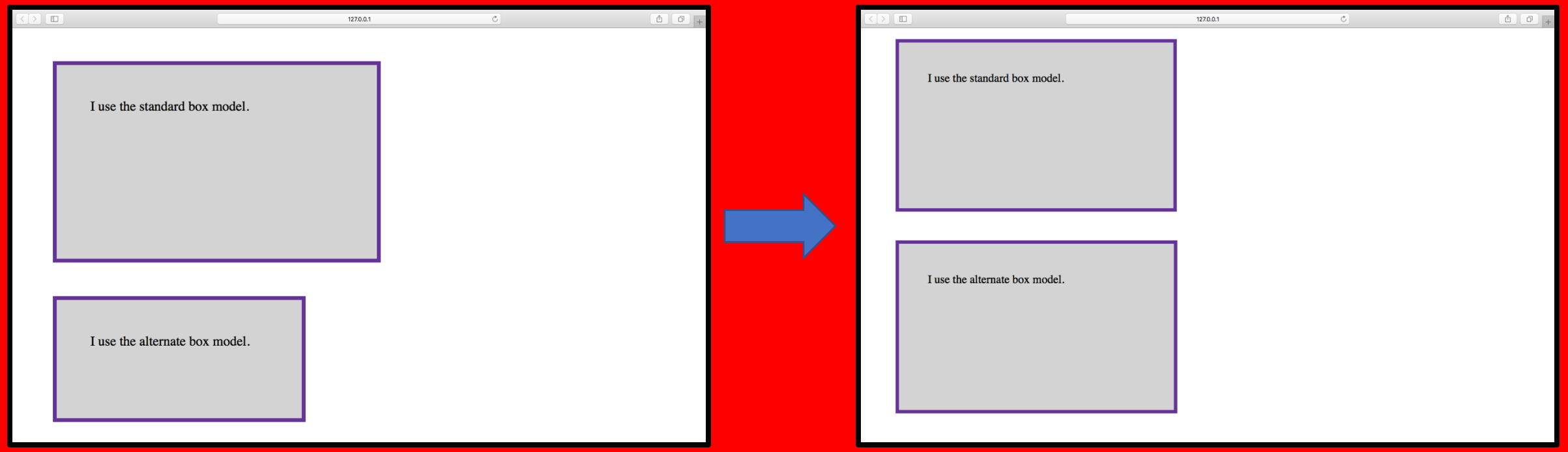
```
<div class="box">
  I use the standard box model.
</div>
<div class="box alternate">
  I use the alternate box model.
</div>
```



In the **Active Learning exercises**, you will have the opportunity to make the **second box (alternative box model)** to become the **same dimensions** as the **first box (standard box model)**.

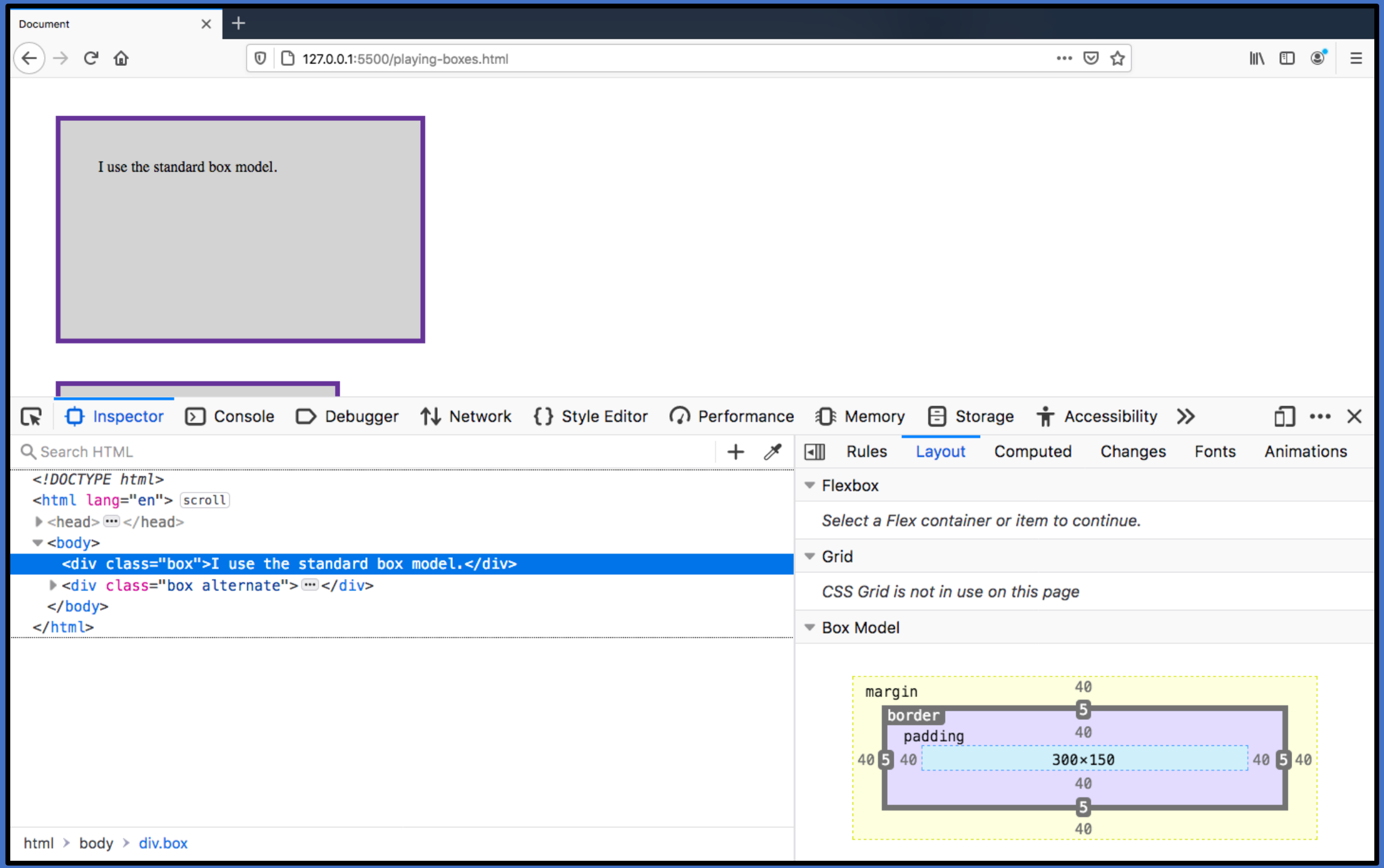
Active learning:
Playing with **box models**

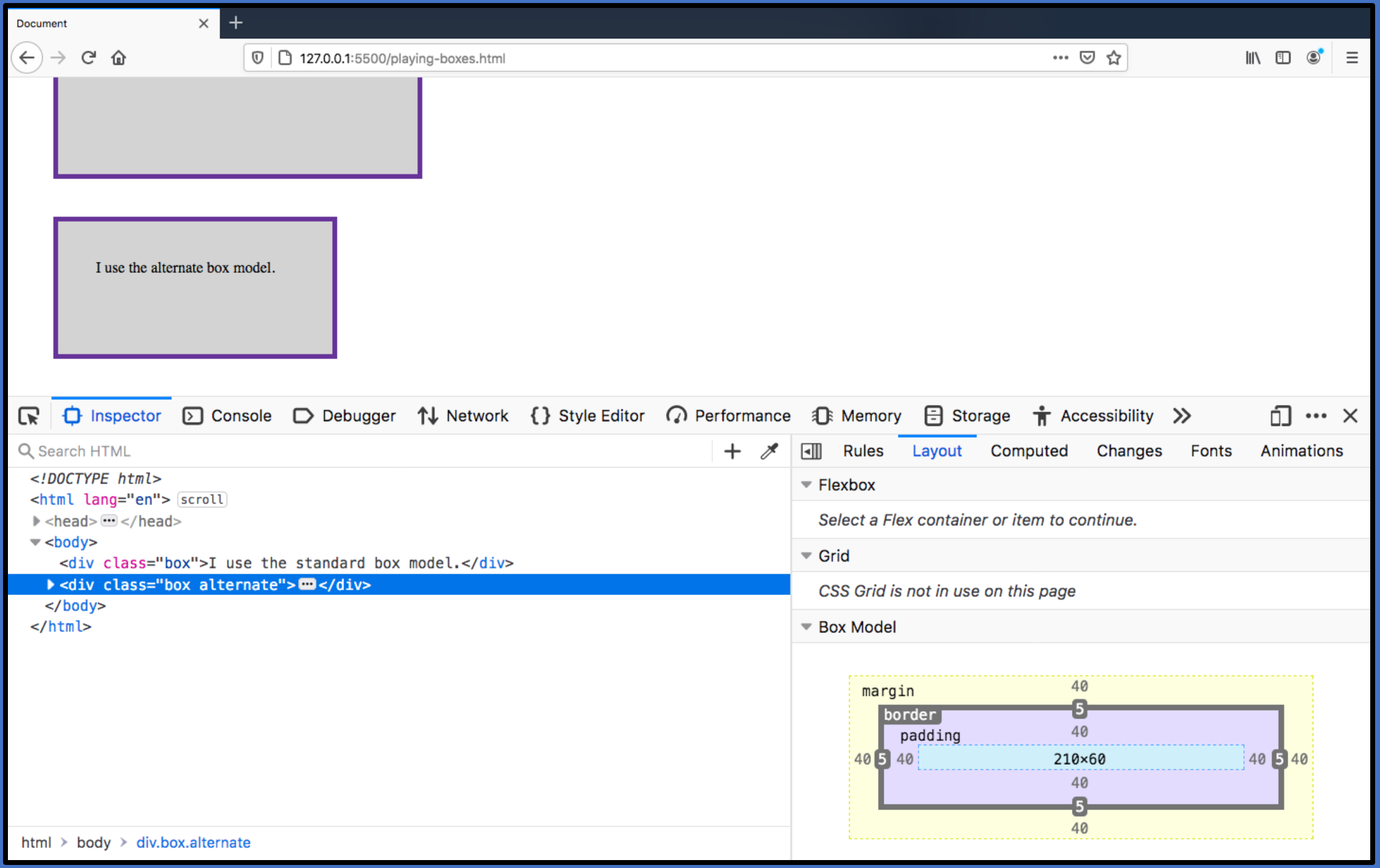
Can you **change** the **size** of the **second box** (by **adding CSS** to the `.alternate` **class**) to make it **match** the **first box** in **`width`** and **`height`**?



https://mzl.la/2Cqg0J1

Use **browser developer tools** to **view** the **box model**


Document
127.0.0.1:5500/playing-boxes.html
I use the standard box model.
Inspector
Console
Debugger
Network
Style Editor
Performance
Memory
Storage
Accessibility
Search HTML
<!DOCTYPE html>
<html lang="en"> scroll
<head> ··· </head>
<body>
I use the standard box model.
 ···
</body>
</html>
Rules
Layout
Computed
Changes
Fonts
Animations
Flexbox
Select a Flex container or item to continue.
Grid
CSS Grid is not in use on this page
Box Model
margin
40
border
5
padding
40
40 5 40
300×150
40 5 40
40
5
40
html
body
div.box


Document
127.0.0.1:5500/playing-boxes.html
I use the alternate box model.
Inspector
Console
Debugger
Network
Style Editor
Performance
Memory
Storage
Accessibility
Search HTML
<!DOCTYPE html>
<html lang="en"> scroll
<head> ··· </head>
<body>
I use the standard box model.
 ···
</body>
</html>
Rules
Layout
Computed
Changes
Fonts
Animations
Flexbox
Select a Flex container or item to continue.
Grid
CSS Grid is not in use on this page
Box Model
margin
40
border
5
padding
40
40 5 40
210×60
40 5 40
40
5
40
html > body > div.box.alternate

Margins, **padding** and **borders**

Shorthands

- The **box properties** used are **shorthands** and allow us to **set** all **four sides** of the **box** at once.
  - ✓`padding`, `border`, `margin`.

- These **shorthands** also have **equivalent longhand properties.**
  - ✓The **longhand versions allow control** over the different **sides** of the **box individually**.

CSS property: `margin`

- The **`margin`** is an **invisible space around** your **box**.
- It **pushes** other **elements away** from the **box**.
- **Margins** can have **positive** or **negative values**.

CSS property: `margin`

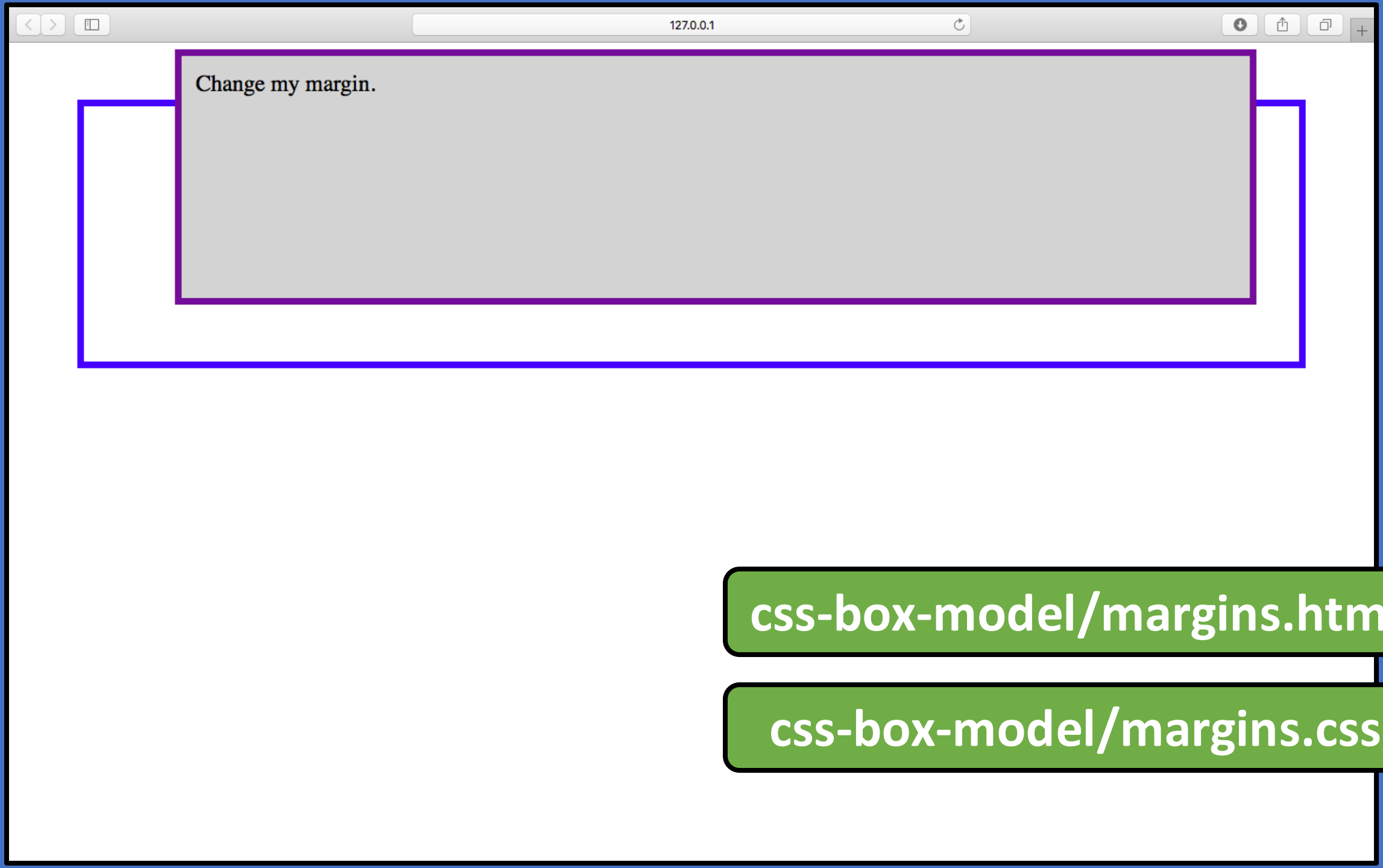
- Setting a **negative margin** on **one side** of your **box** can **cause** it to **overlap** other **things** on the **page**.

- The **margin** is **always added AFTER** the **size** of the **visible box** has been **calculated**.
  - ✓**Whether** you are using the **standard** or **alternative box model**.

CSS property: `margin`

- We can **control all margins** of an **element** at **once** using the `margin` **property**.

- To **control each side** individually using the equivalent **longhand properties**:
  - ✓`margin-top`
  - ✓`margin-right`
  - ✓`margin-bottom`
  - ✓`margin-left`



css-box-model/margins.html

css-box-model/margins.css

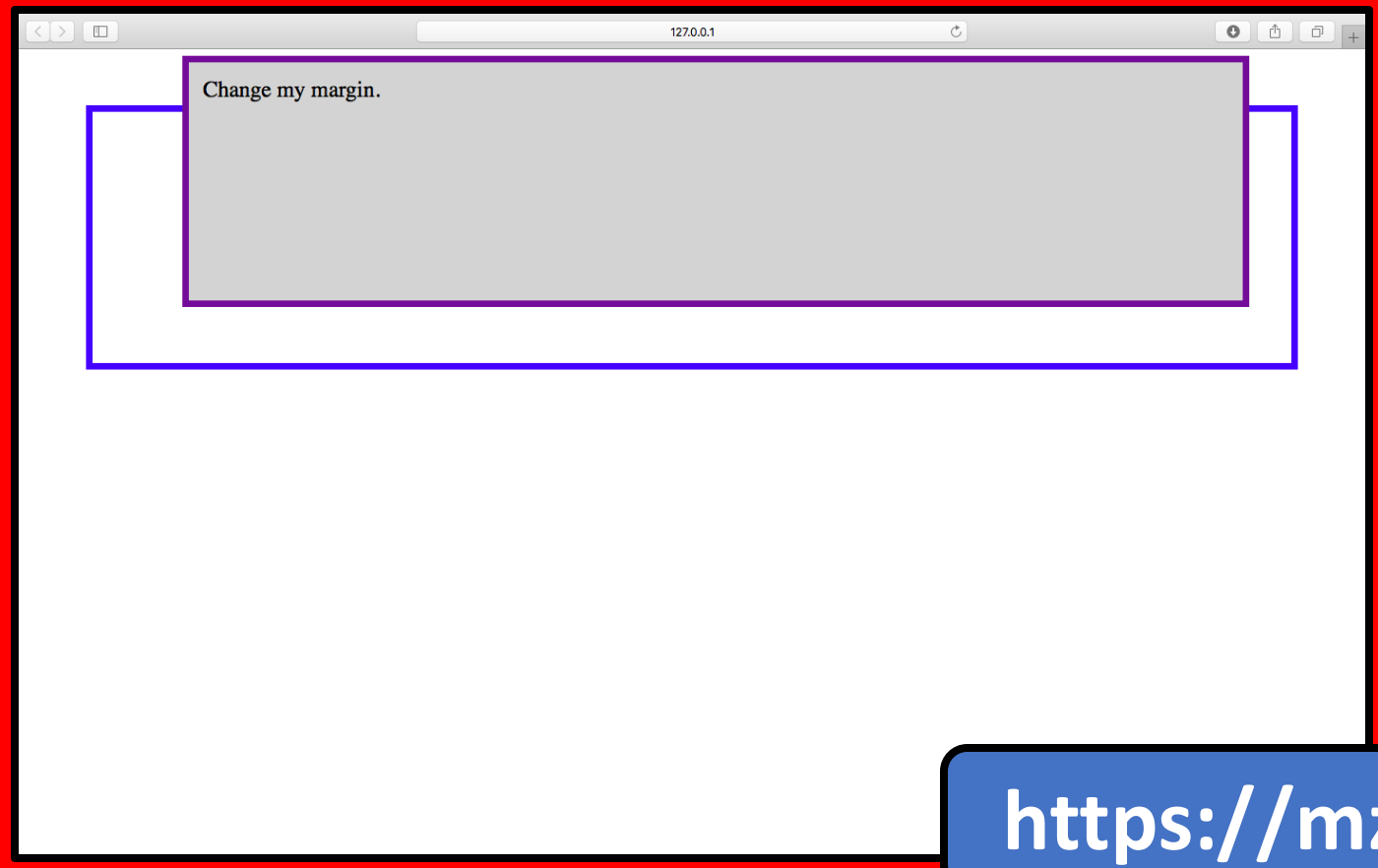



```
01 .container {
02   border: 5px solid blue;
03   margin: 40px;
04 }
05
06 .box {
07   margin-top: -40px;
08   margin-right: 30px;
09   margin-bottom: 40px;
10   margin-left: 4em;
11   border: 5px solid rebeccapurple;
12   background-color: lightgray;
13   padding: 10px;
14   height: 150px;
15 }
```

```
1 <div class="container">
2     <div class="box">Change my margin.</div>
3 </div>
```

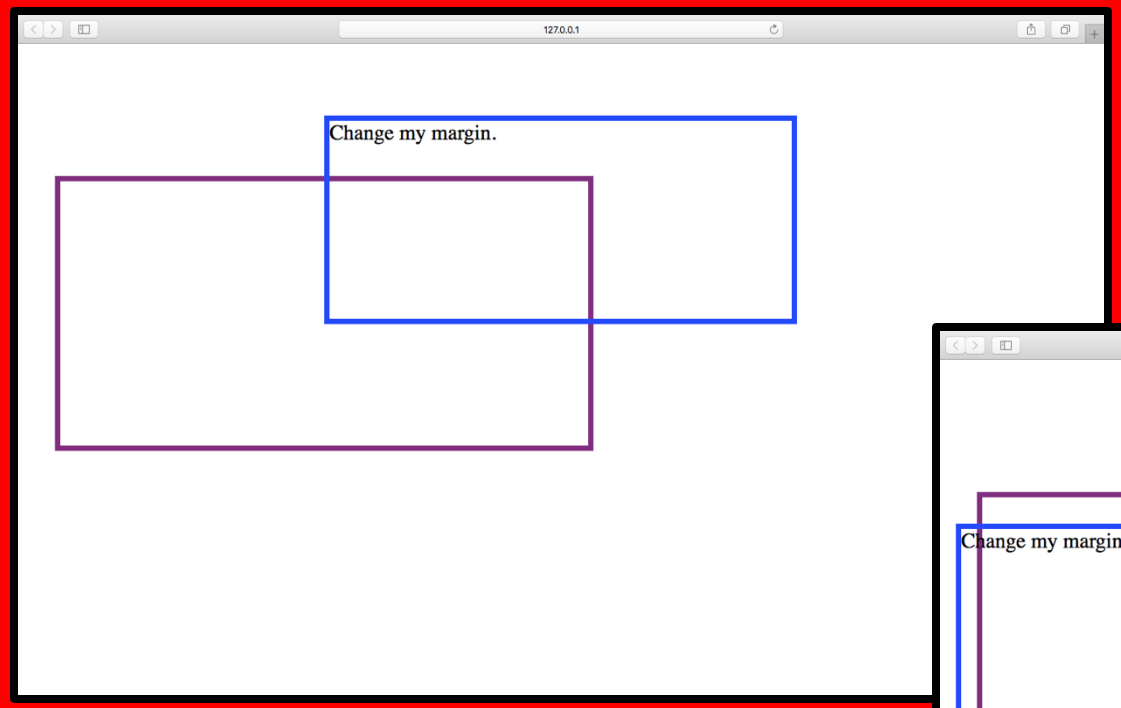
Active learning:
Playing with **margins**

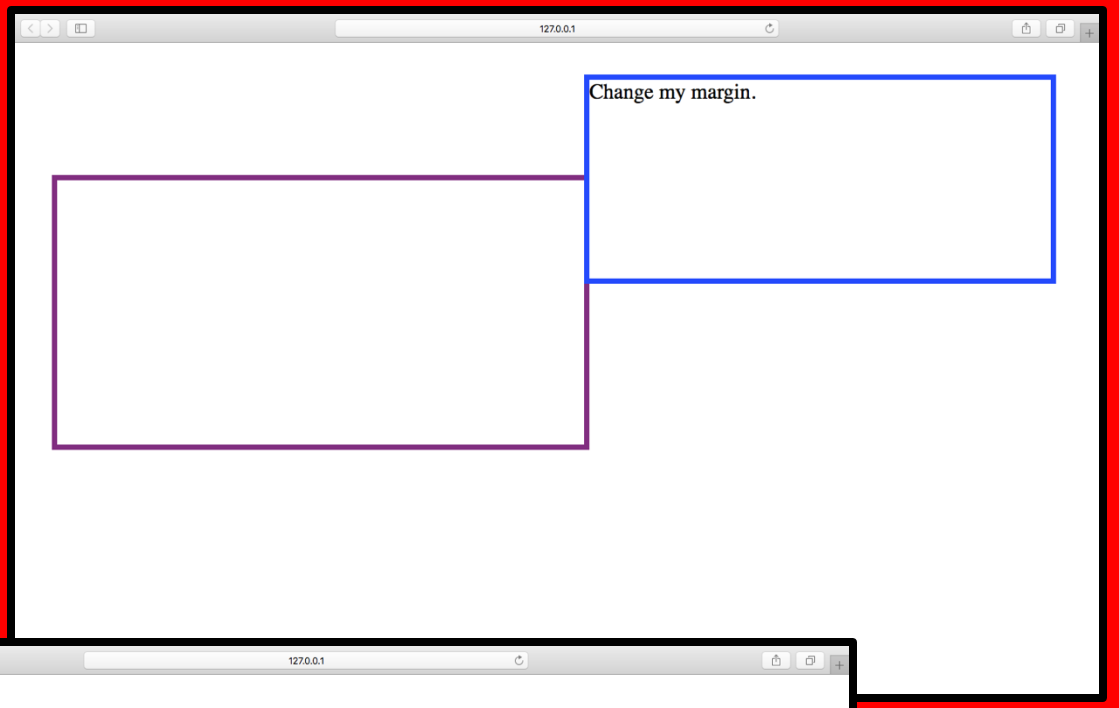
Try changing the **margin values** to see **how** the **box** is **pushed around due** to the **margin creating** or **removing space** (if it is a **negative margin**) **between** this **element** and the **containing element.**

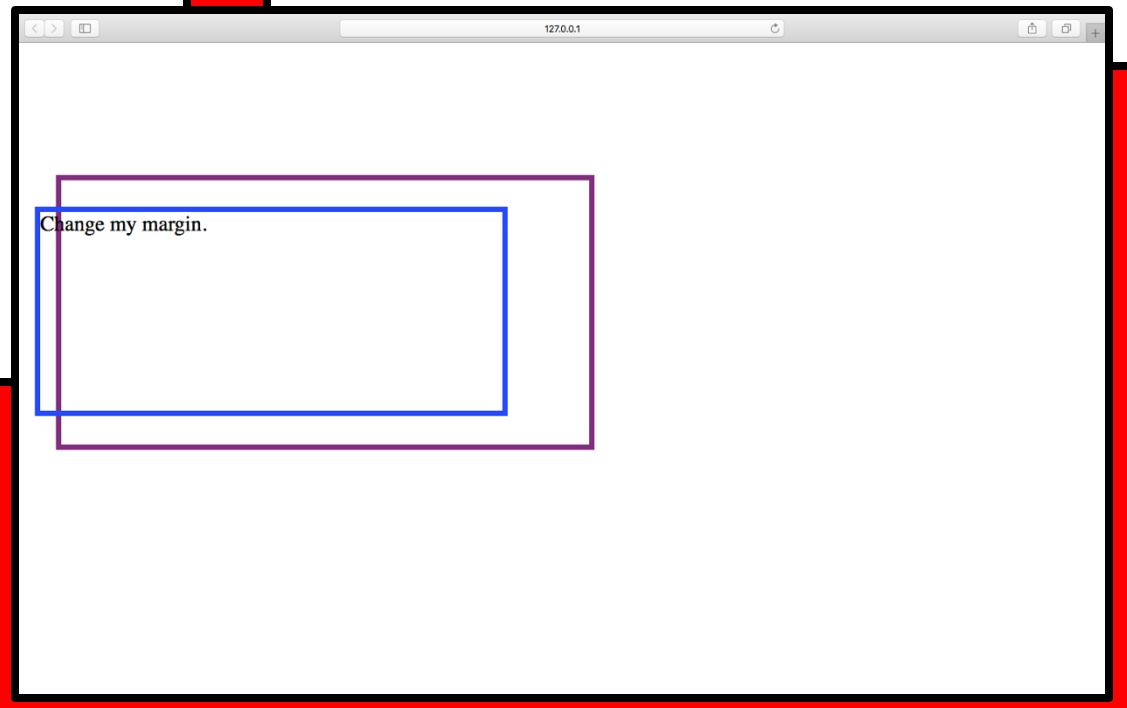


https://mzl.la/31Gn5ij

Possible solutions





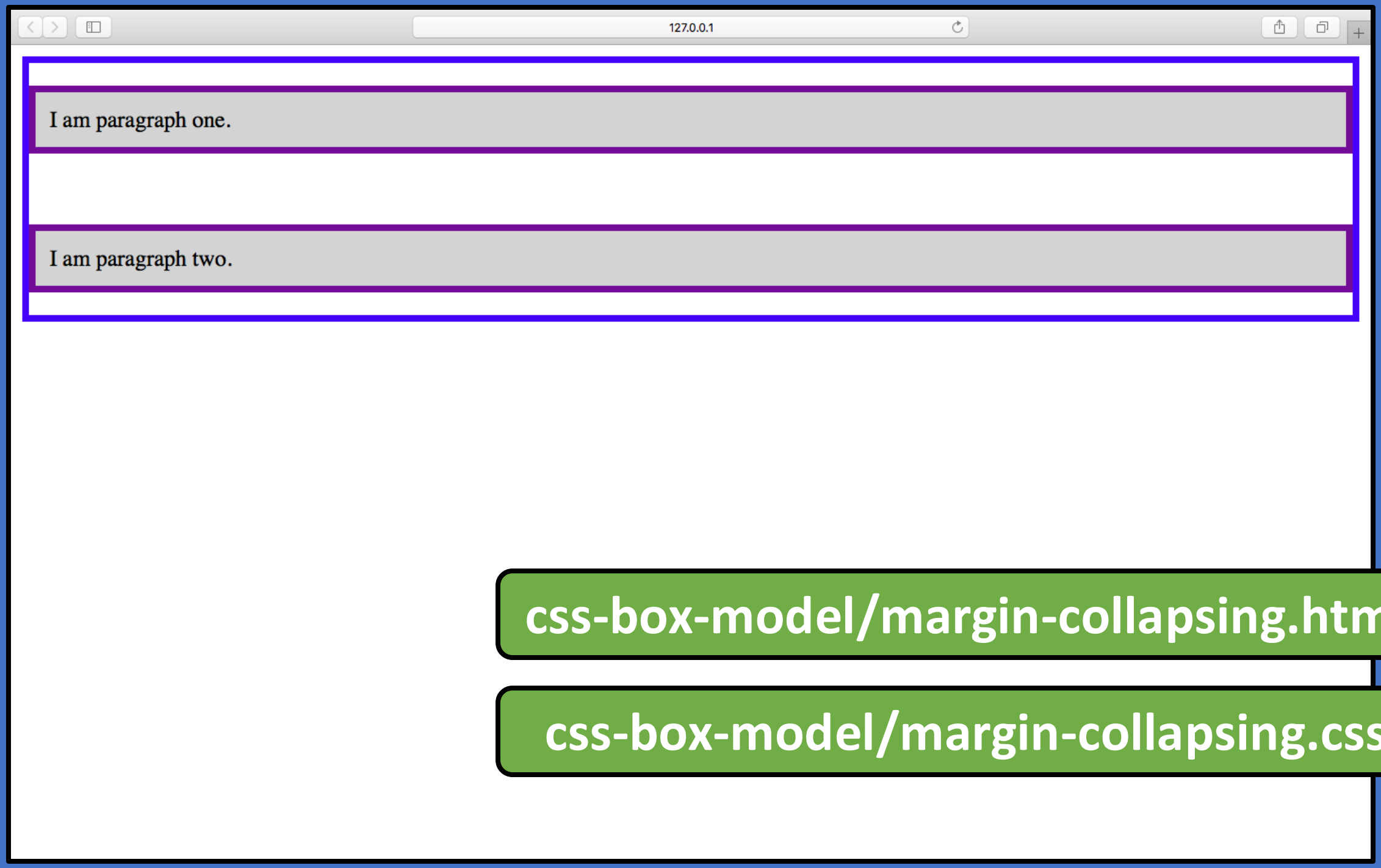


Marge colappsing

Marging collapsing

- **Key thing** to **understand about margins**.

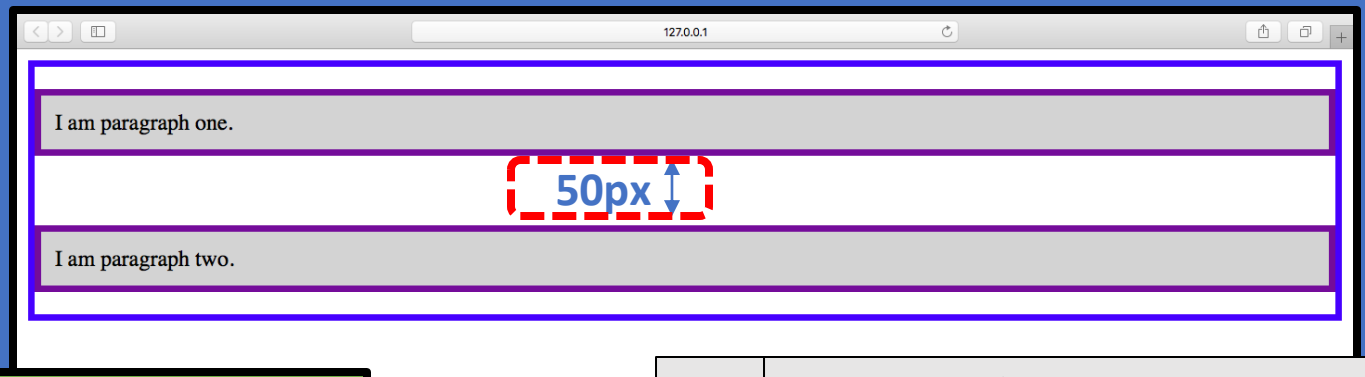
- **Algorithm**:
  1. If you have **two elements** whose **margins touch**, and **both margins** are **positive**, those **margins** will **combine** to **become** one **margin**, which is the **size** of the **largest individual margin**.
  2. If **one margin** is **negative**, the **amount** of **negative value** will **subtract from** the **positive**.
  3. If **both margins** are **negative,** the **margin** that **collapses** is the bigger **negative** of the **two** rather **than the one** that is **closest** to being **positive**.



css-box-model/margin-collapsing.html

css-box-model/margin-collapsing.css

Marging collapsing



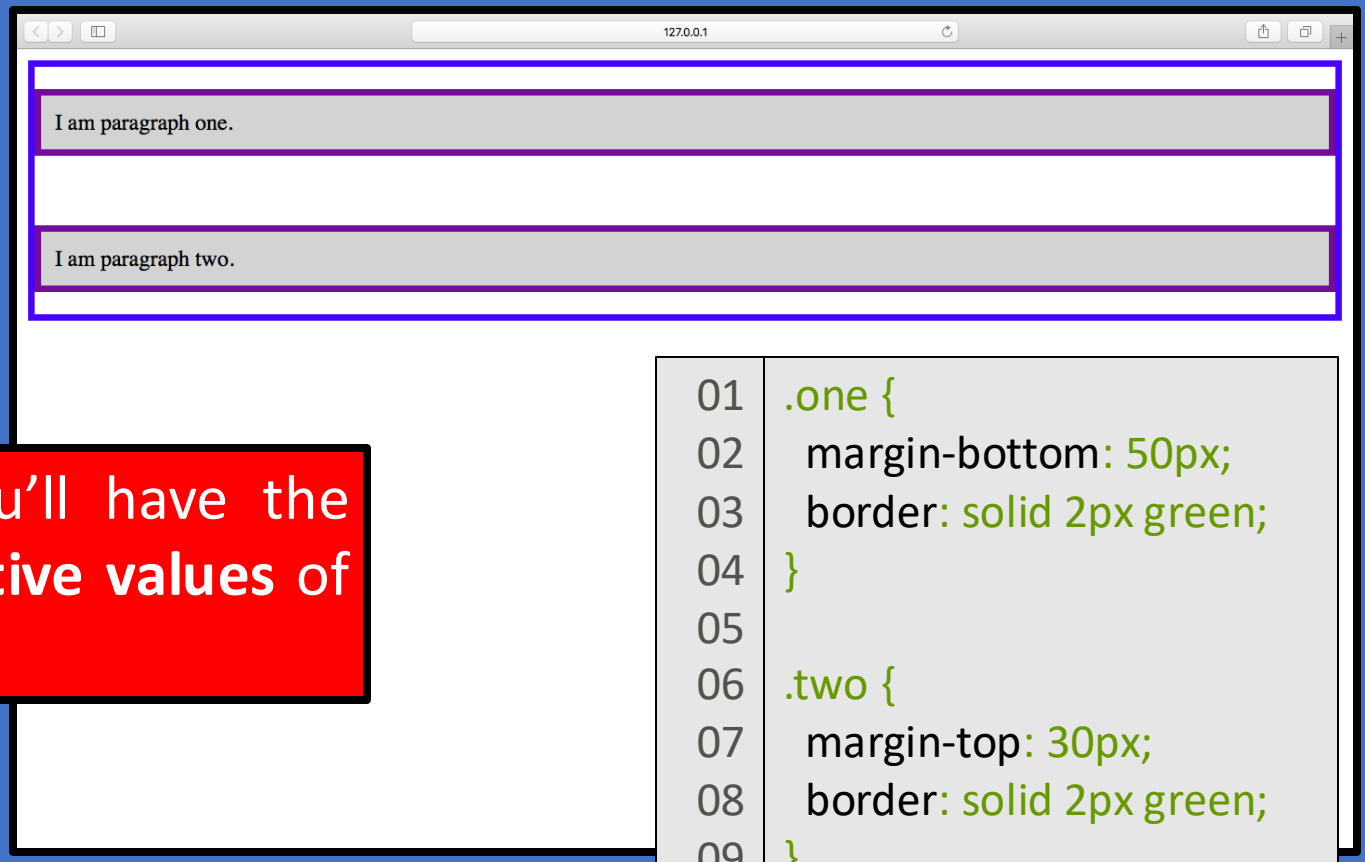
- The **top paragraph** has a `margin-bottom` of **50** pixels.
- The **second paragraph** has a `margin-top` of **30** pixels.
- The **margins** have **collapsed** together so the **actual margin between** the **boxes** is **50 pixels** and not the **total** of the **two margins**.

```
01 .container {
02   border : solid 5px blue;
03 }
04
05 .one {
06   margin-bottom: 50px;
07 }
08
09 .two {
10   margin-top: 30px;
11 }
12
13 p {
14   border: 5px solid rebeccapurple;
15   background-color: lightgray;
16   padding: 10px;
17 }
```

```
01 <body>
02    <div class="container">
03       <p class="one">I am paragraph one.</p>
04       <p class="two">I am paragraph two.</p>
05     </div>
06 </body>
```

Marging collapsing

In the **Active Learning** exercises, you'll have the **opportunity** to **experiment** with **negative values** of **margins**.

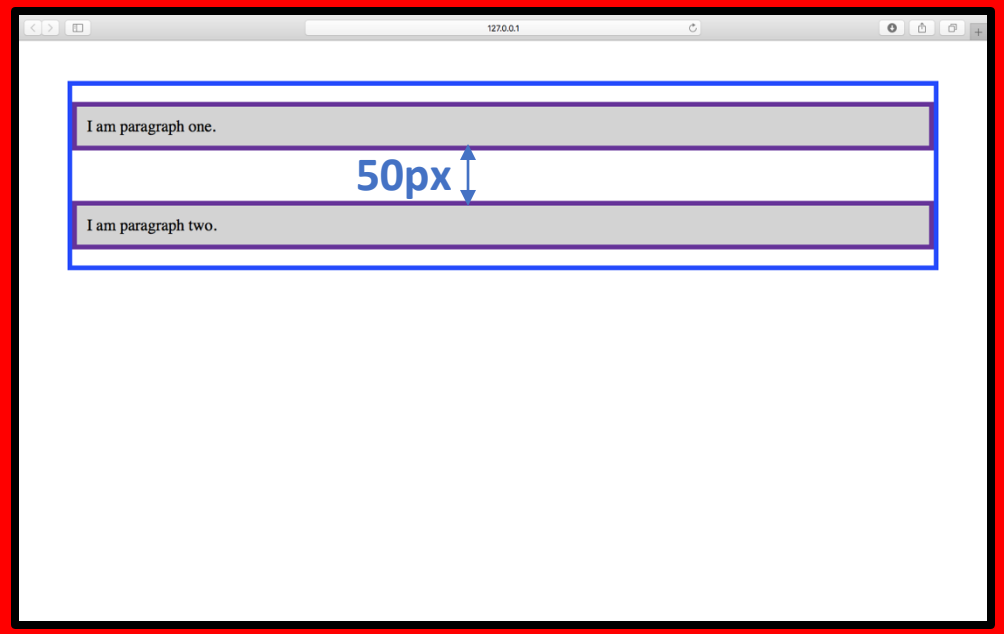


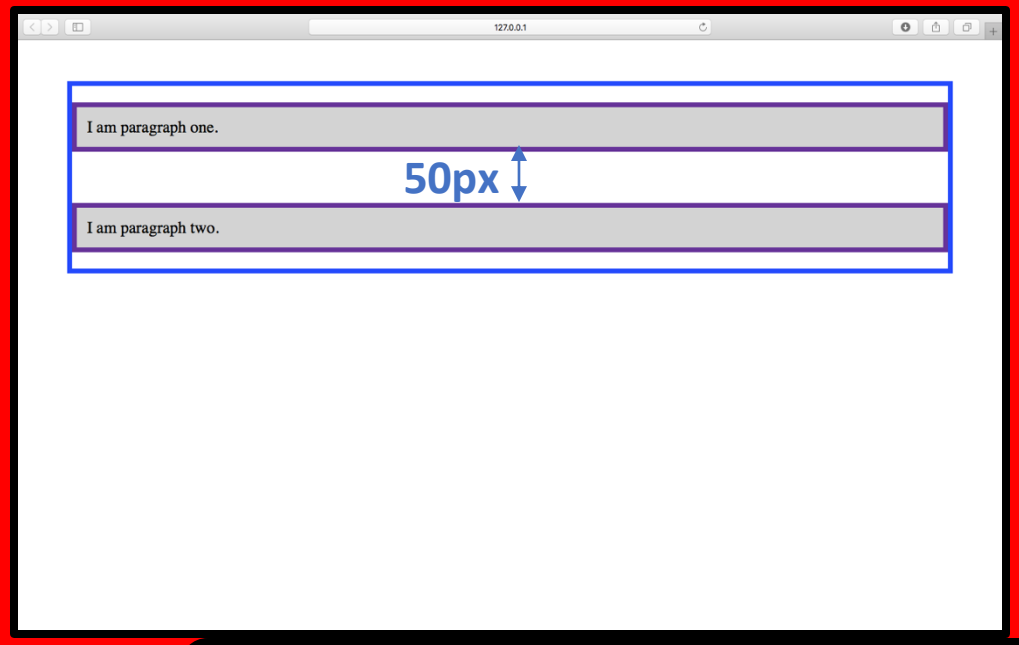

```
01  .one {
02    margin-bottom: 50px;
03    border: solid 2px green;
04  }
05
06  .two {
07    margin-top: 30px;
08    border: solid 2px green;
09  }
10  ...
11  ...
```

```
01  <body>
02    <div class="container">
03      <p class="one">I am paragraph one.</p>
04      <p class="two">I am paragraph two.</p>
05    </div>
06  </body>
```

Active learning:
Margin collapsing

We can test margin collapsing algorithm:
Set the `margin-top` of paragraph two to 0. The visible margin between the two paragraphs will not change — it retains the 50 pixels set in the bottom-margin of paragraph one

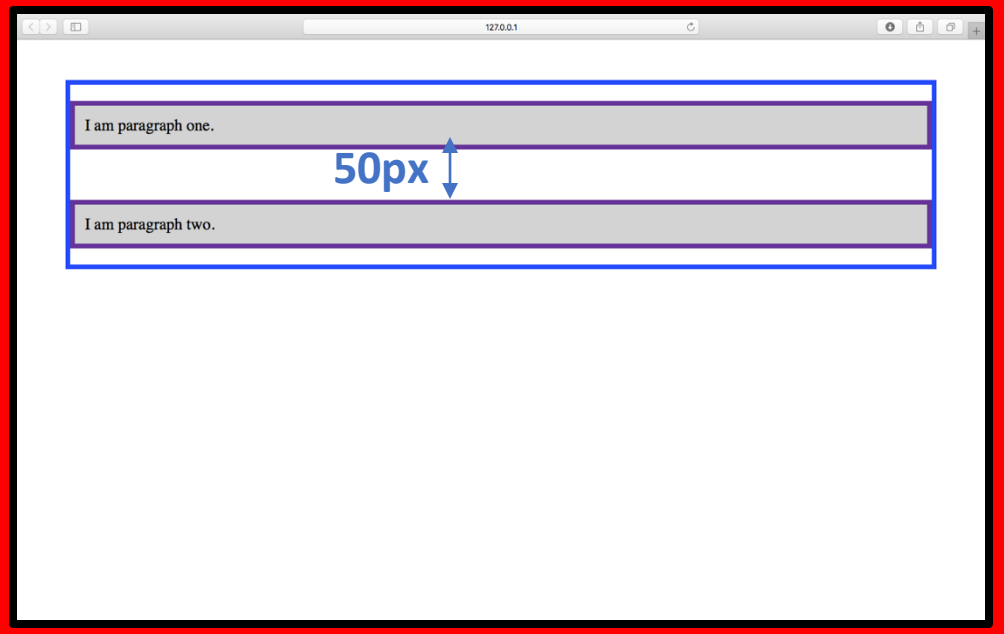


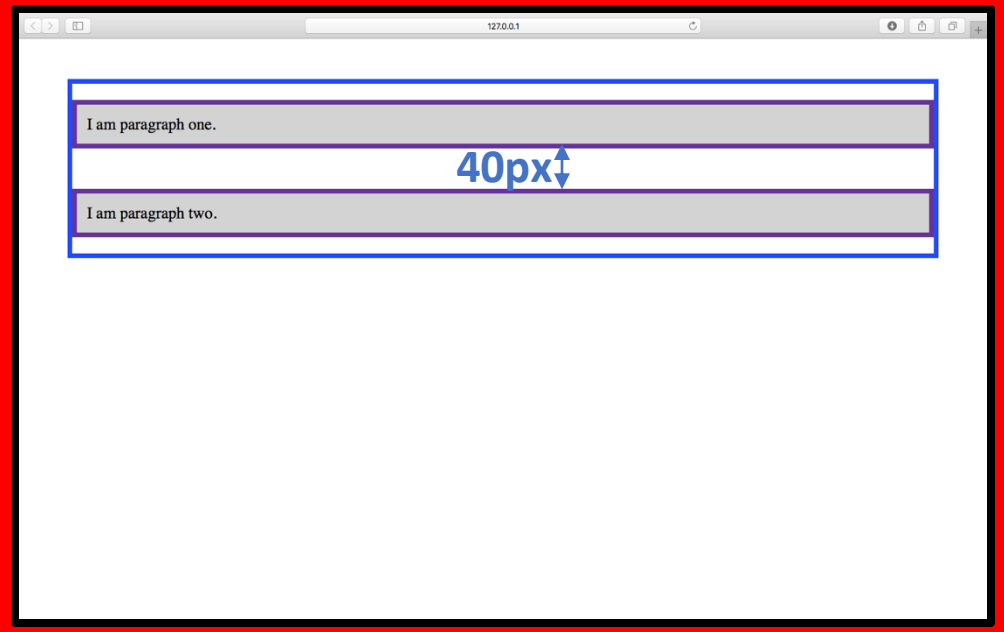


https://mzl.la/2PPpXTt

We can test margin collapsing algorithm:

Set the `margin-top` of **paragraph two** to **-10px,** you'll see that the **overall margin** becomes **40px — it subtracts** from the **50px.**





Borders

Borders

- The **border** is **drawn between** the **margin** and the **padding** of a **box**.
- If you are **using** the **standard box model**, the **size** of the **border** is **added** to the `width` and `height` of the **box**.
- If you are **using** the **alternative box model** then the **size** of the **border** makes the **content box smaller** as it takes up some of that available **width** and **height**.

Properties for styling boders

- A **box** has **four borders** and **each border** has a **`style`**, **`width`** and **`color`** that we might want to **manipulate**.

- You can set the `width`, `style`, or `color` of **all four** borders at **once** using the `border` **property**.

Properties for styling boders

- To **set** the **properties** of **each side** individually, you can use:
  - ✓`border-top`
  - ✓`border-right`
  - ✓`border-bottom`
  - ✓`border-left`

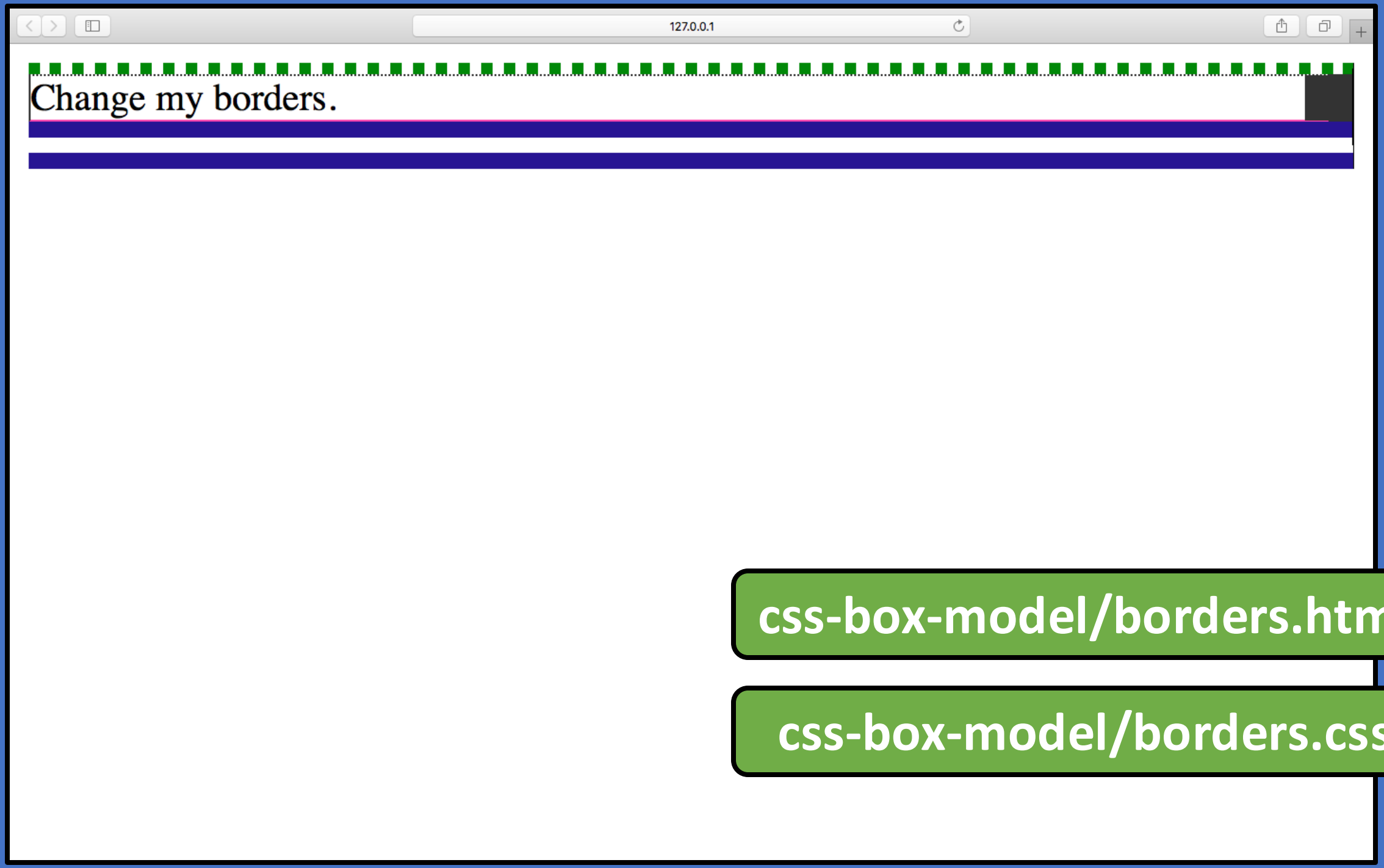
Properties for styling boders

- To **set** the `width`, `style`, or `color` of all **sides**, use the **following**:
  - ✓`border-width`
  - ✓`border-style`
  - ✓`border-color`

Properties for styling boders

- To set the `width`, `style`, or `color` of a **single side**, you can **use one** of the most **granular longhand properties**:

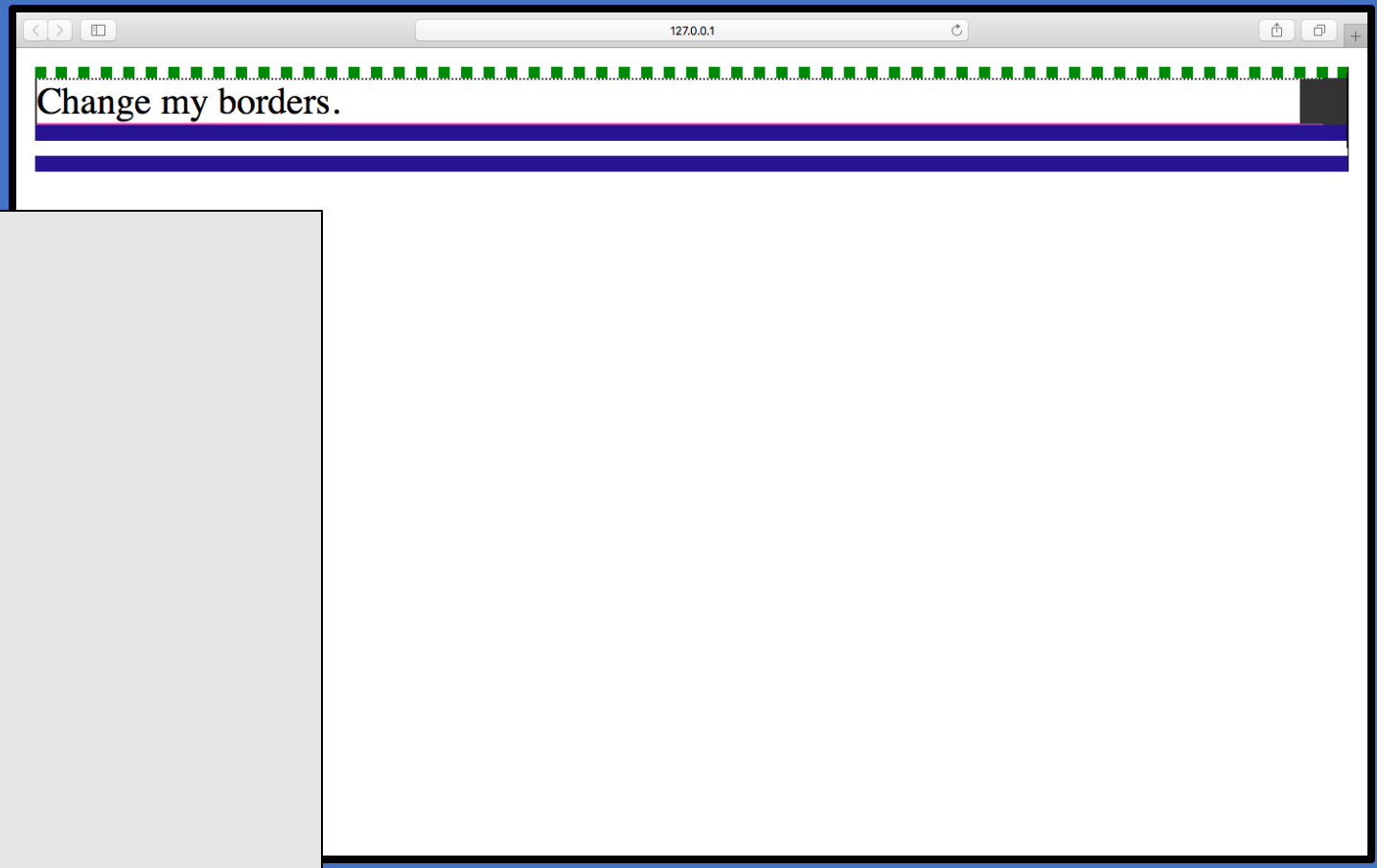
✓`border-top-width`
✓`border-top-style`
✓`border-top-color`
✓`border-right-width`
✓`border-right-style`
✓`border-right-color`
✓`border-bottom-width`
✓`border-bottom-style`
✓`border-bottom-color`
✓`border-left-width`
✓`border-left-style`
✓`border-left-color`



css-box-model/borders.html

css-box-model/borders.css

Borders




```css
.container {
  border-top: 5px dotted green;
  border-right: 1px solid black;
  border-bottom: 20px double rgb(23,45,145);
}

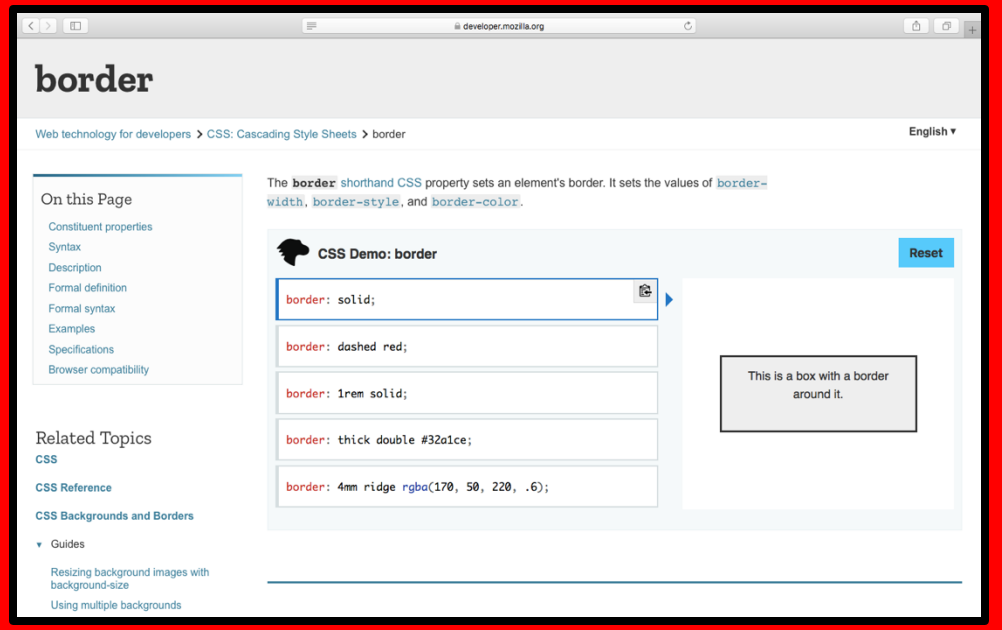
.box {
  border: 1px solid #333333;
  border-top-style: dotted;
  border-right-width: 20px;
  border-bottom-color: hotpink;
}
```

```html
<body>
  <div class="container">
    <div class="box">Change my borders.</div>
  </div>
</body>
```

Active learning:
Shorthand and **longhand** syntax for **borders**

Have a **play around** with the different border **properties** to **check** that you **understand how** they work**. The MDN** pages for the **border properties** give you information about the **different styles** of **border** you can **choose from.**

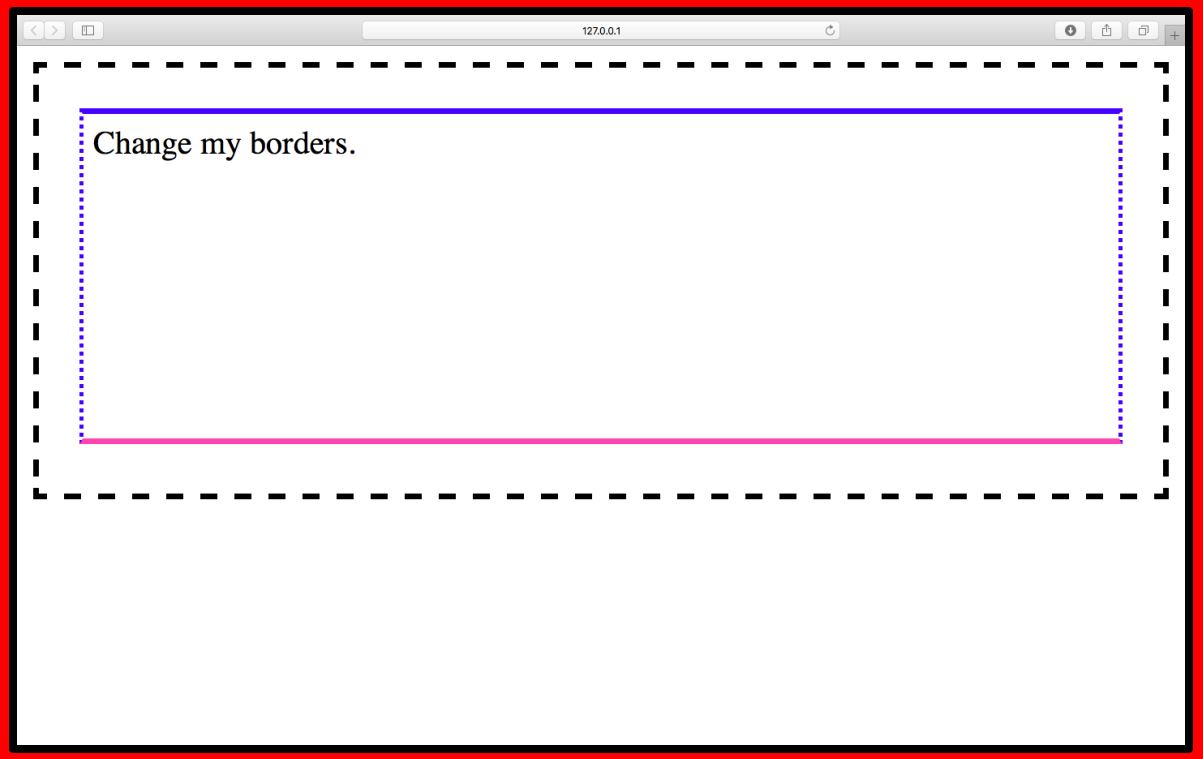




https://mzl.la/3kLiyT8

https://mzl.la/31GIyrf

Possible solutions





Padding

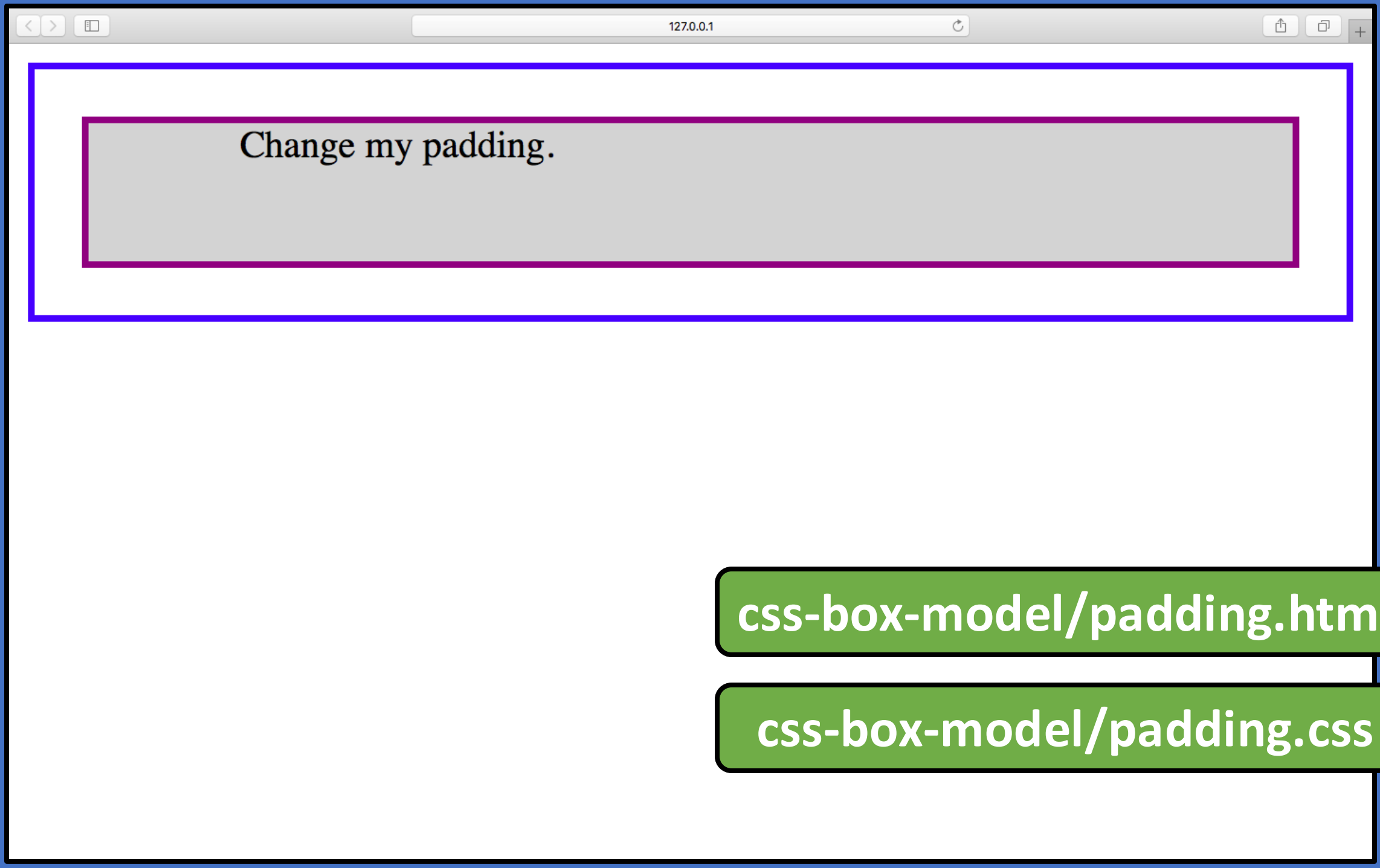
Padding

- The **padding sits between** the **border** and the **content area**.
- **Unlike margins** you cannot have **negative amounts** of **padding**.
- **Any background applied** to your **element** will **display behind** the **`padding`**, and it is **typically** used to **push** the **content away** from **the `border`**.

Padding

- We can set the **`padding`** of **all four** sides of a **box** at **once** using the `padding` **property**.

- We can **control** the **padding** on **each side** of an **element individually** using the `padding` **longhand properties**:
  - ✓`padding-top`
  - ✓`padding-right`
  - ✓`padding-bottom`
  - ✓`padding-left`



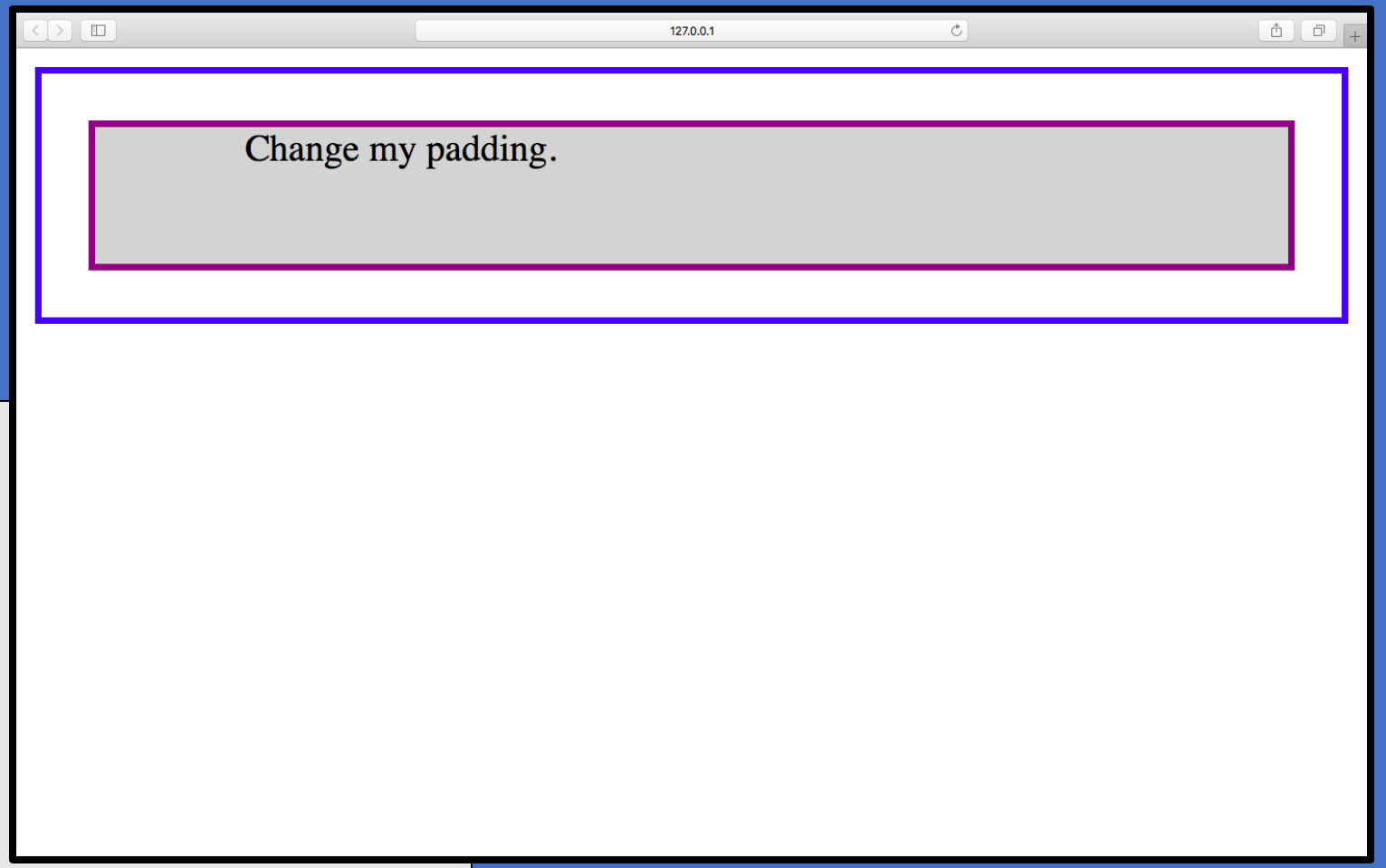
css-box-model/padding.html

css-box-model/padding.css

Padding

```css
.container {
  padding: 20px;
  border: blue 5px solid;
}

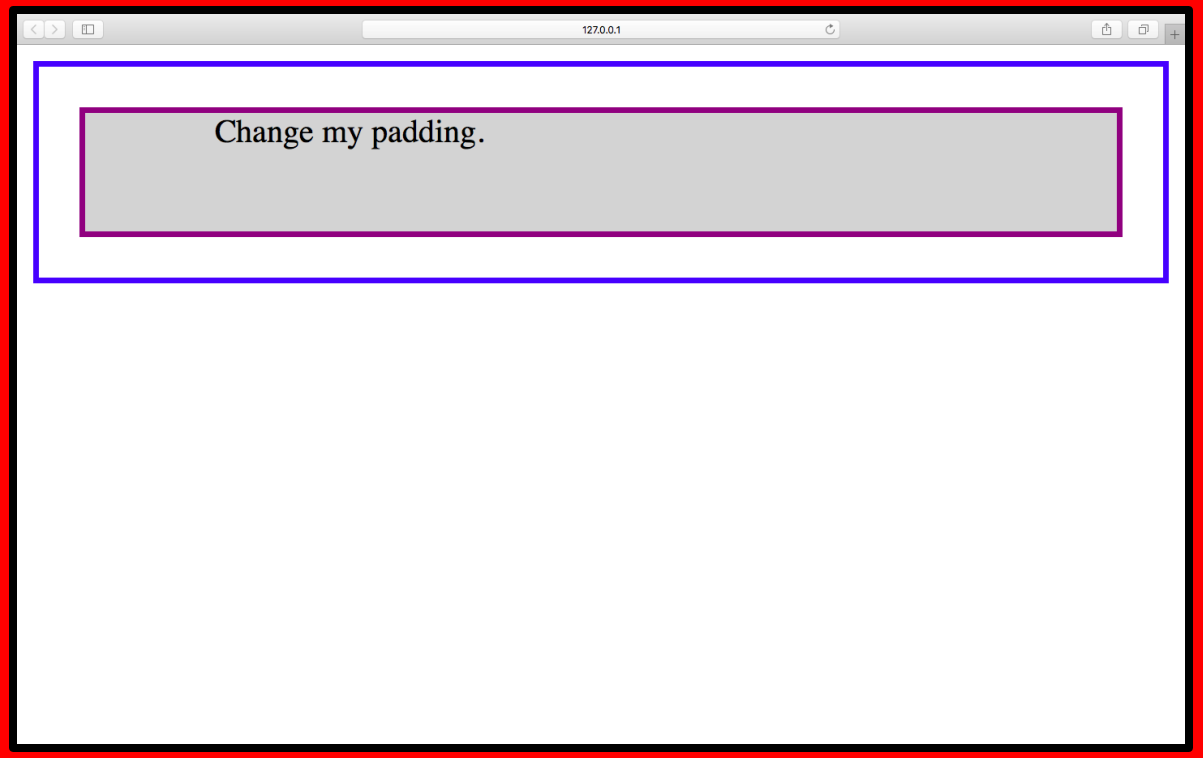
.box {
  padding-top: 0;
  padding-right: 30px;
  padding-bottom: 40px;
  padding-left: 4em;
  background-color: lightgray;
  border: purple 5px solid;
}
```

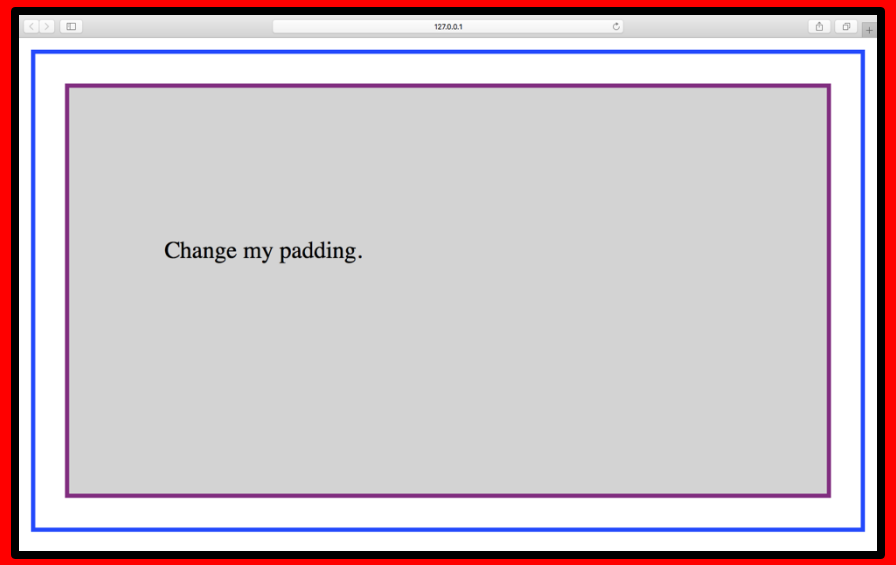



```html
<div class="container">
  <div class="box">Change my padding.</div>
</div>
```

Active learning: `Padding`

Change the **values** for `padding` on the **class** `.box` in the example to **see** that this **changes where** the **text begins** in **relation** to the **box**.

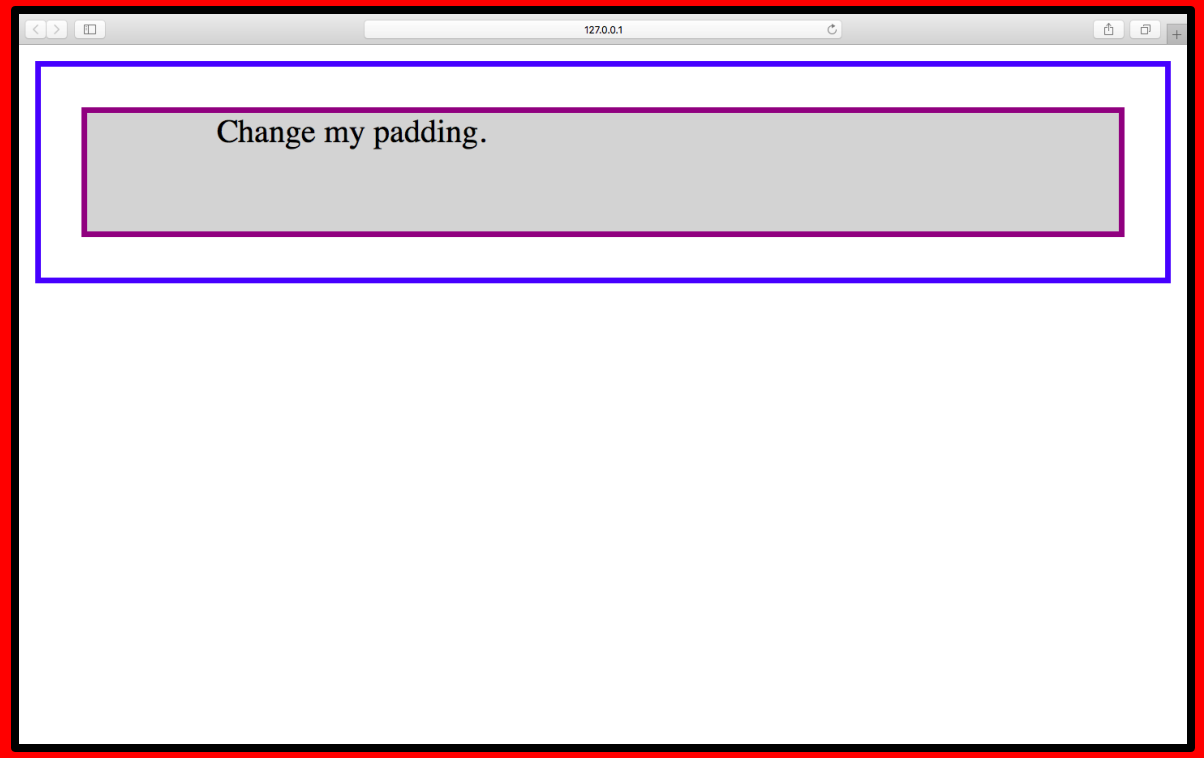


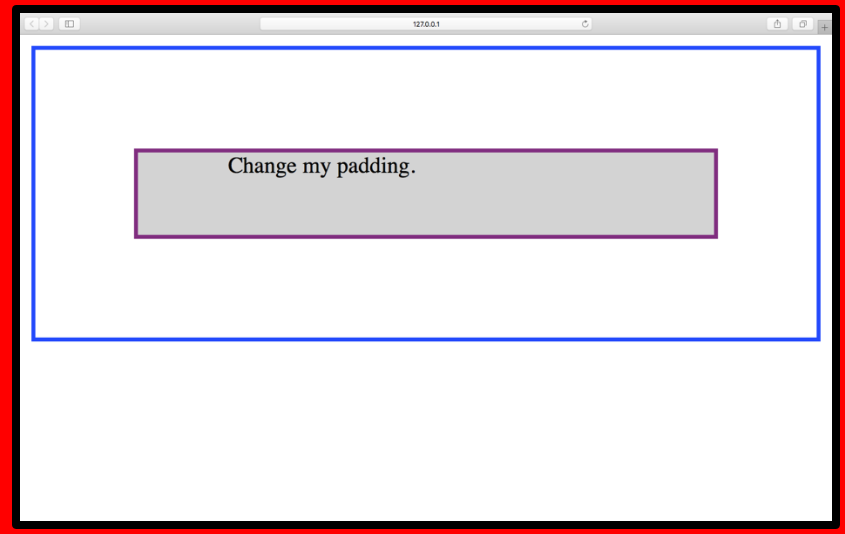


Possible solution

https://mzl.la/3aLWqDE

Change the **padding** on the **class** `.container`, which will **make space between** the **container** and the **box. Padding** can be **changed** on **any element** and **will make space between** its **border** and whatever is **inside** the **element.**



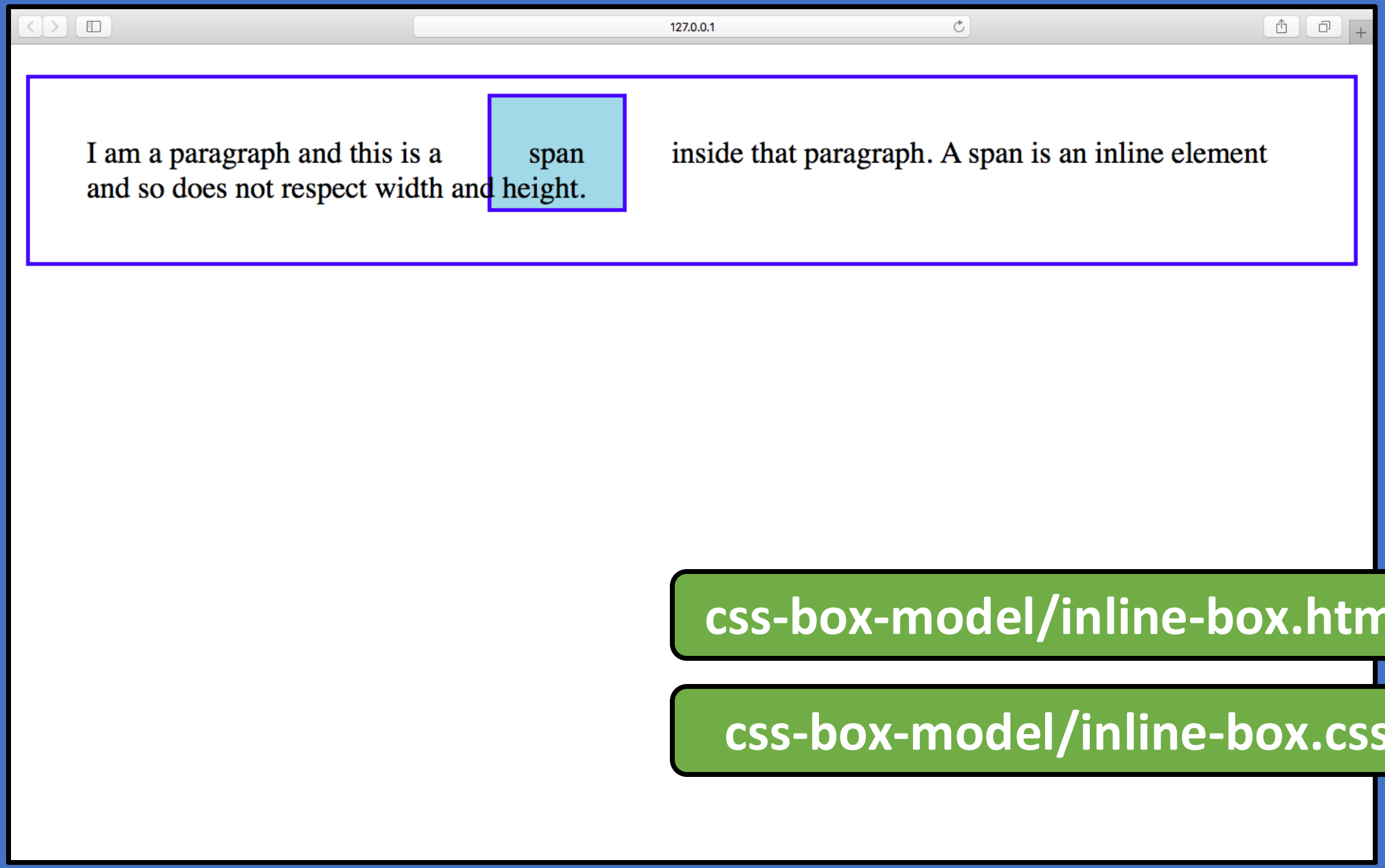


Possible solution

The box model & inline boxes

The box model & inline boxes

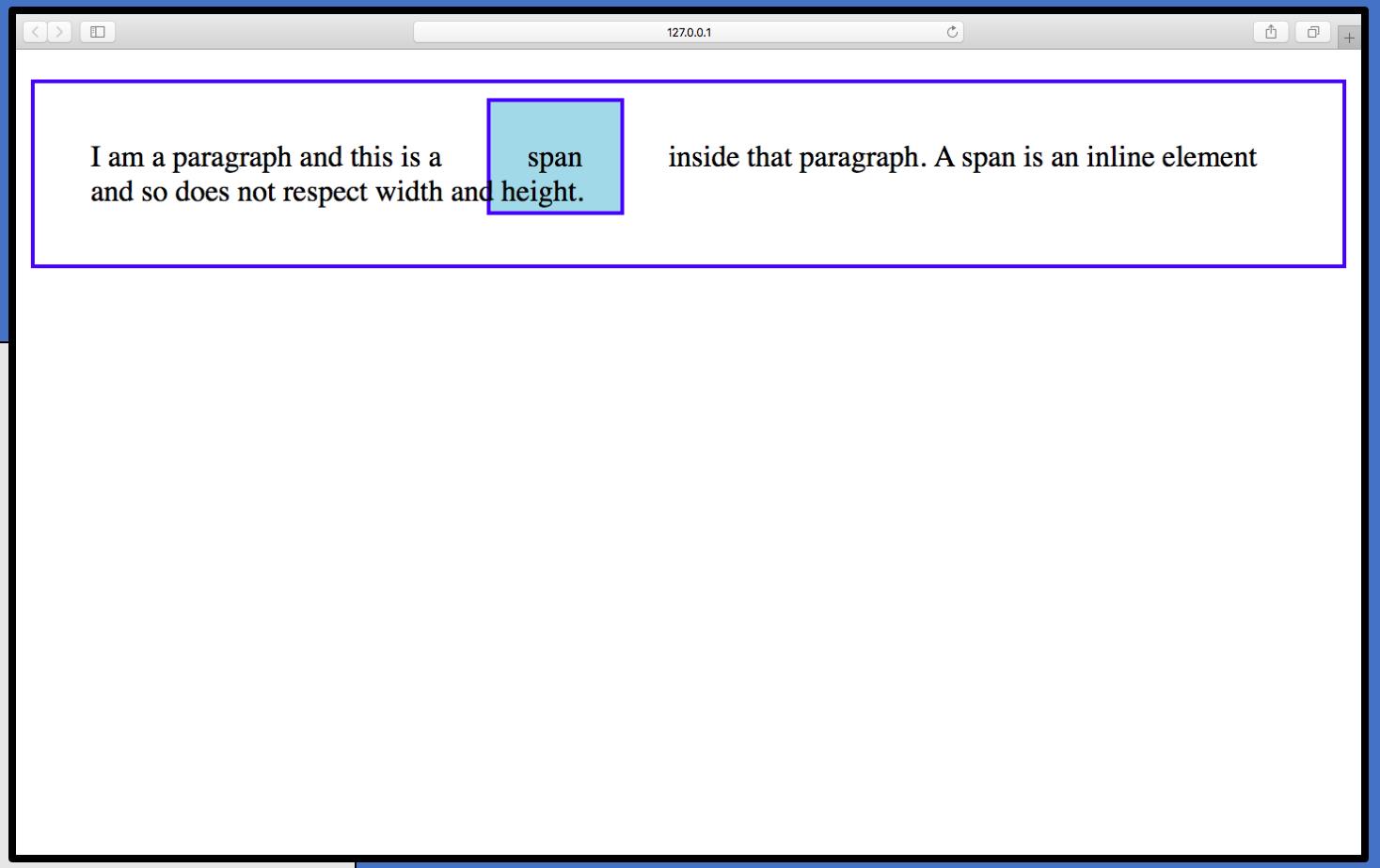
- **All properties** discussed so far in this lecture **apply** to **block boxes**.
- **Some** of the **properties** can **apply** to **inline boxes** too
  - ✓Such as those created by a `<span>` **element**.


127.0.0.1
I am a paragraph and this is a span inside that paragraph. A span is an inline element and so does not respect width and height.
css-box-model/inline-box.html
css-box-model/inline-box.css

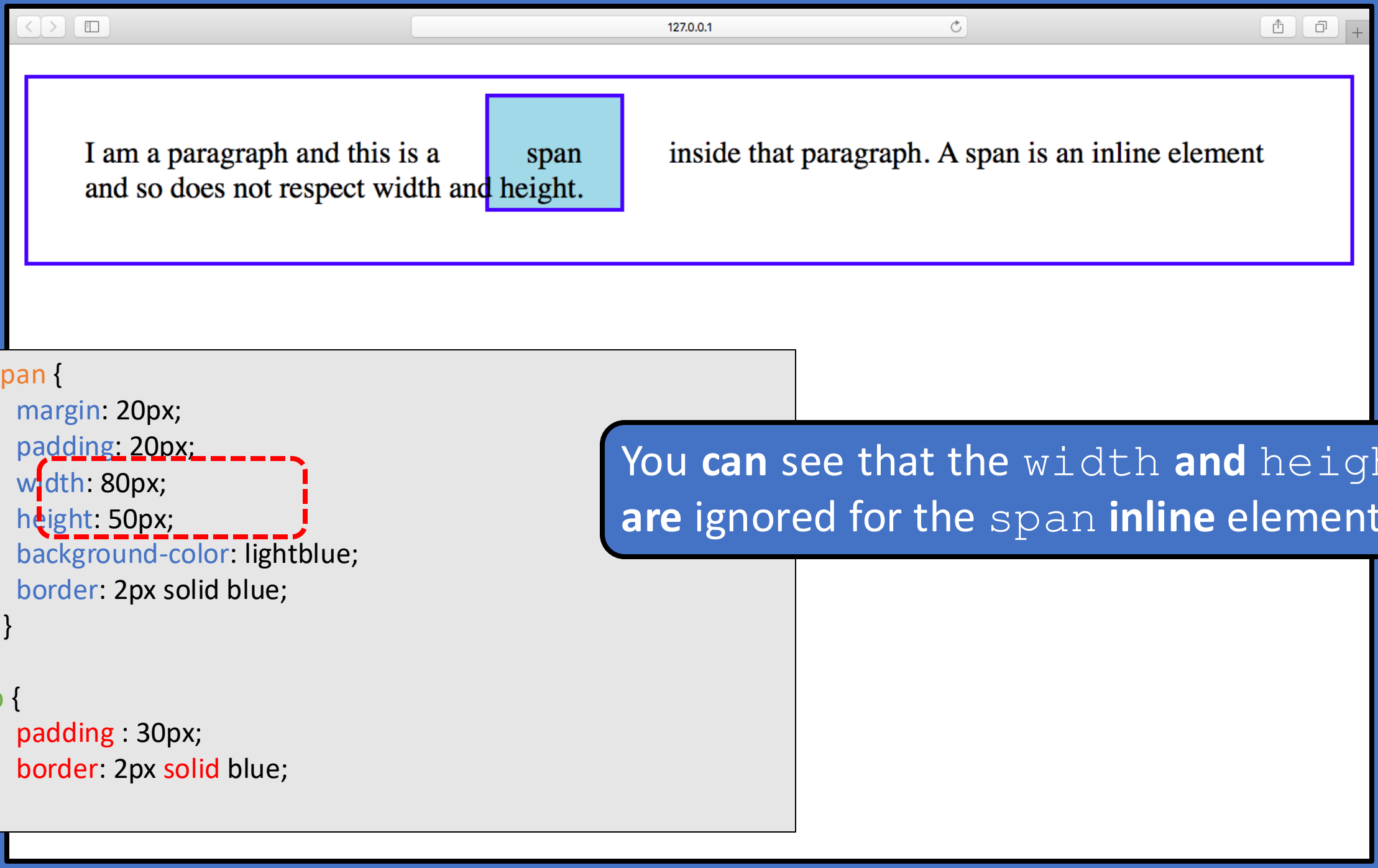
Inline boxes

```css
span {
    margin: 20px;
    padding: 20px;
    width: 80px;
    height: 50px;
    background-color: lightblue;
    border: 2px solid blue;
}

p {
    padding : 30px;
    border: 2px solid blue;
}
```

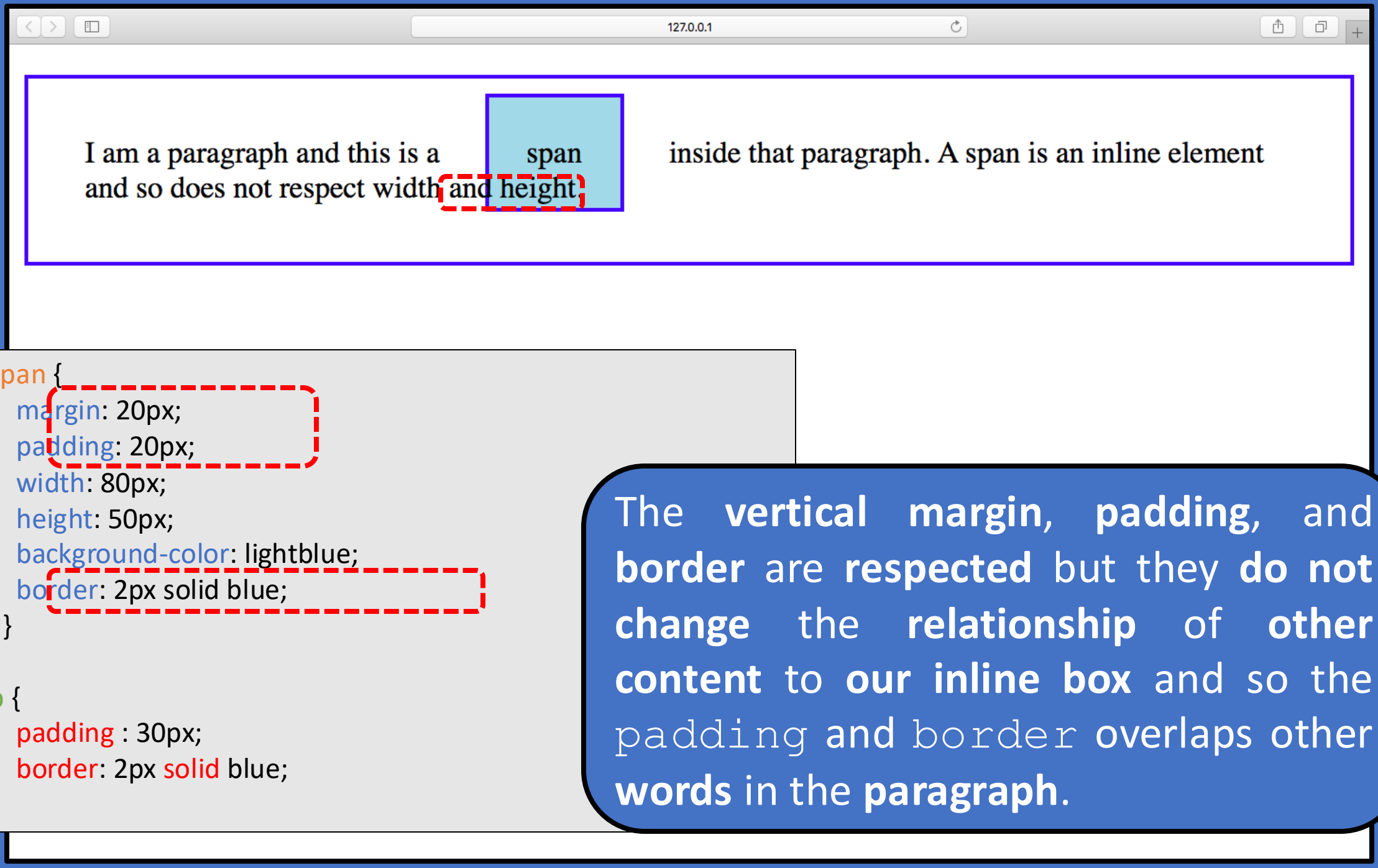



```html
<p>I am a paragraph and this is a <span>span</span> inside that paragraph. A span is an inline element and so does not respect width and height.</p>
```



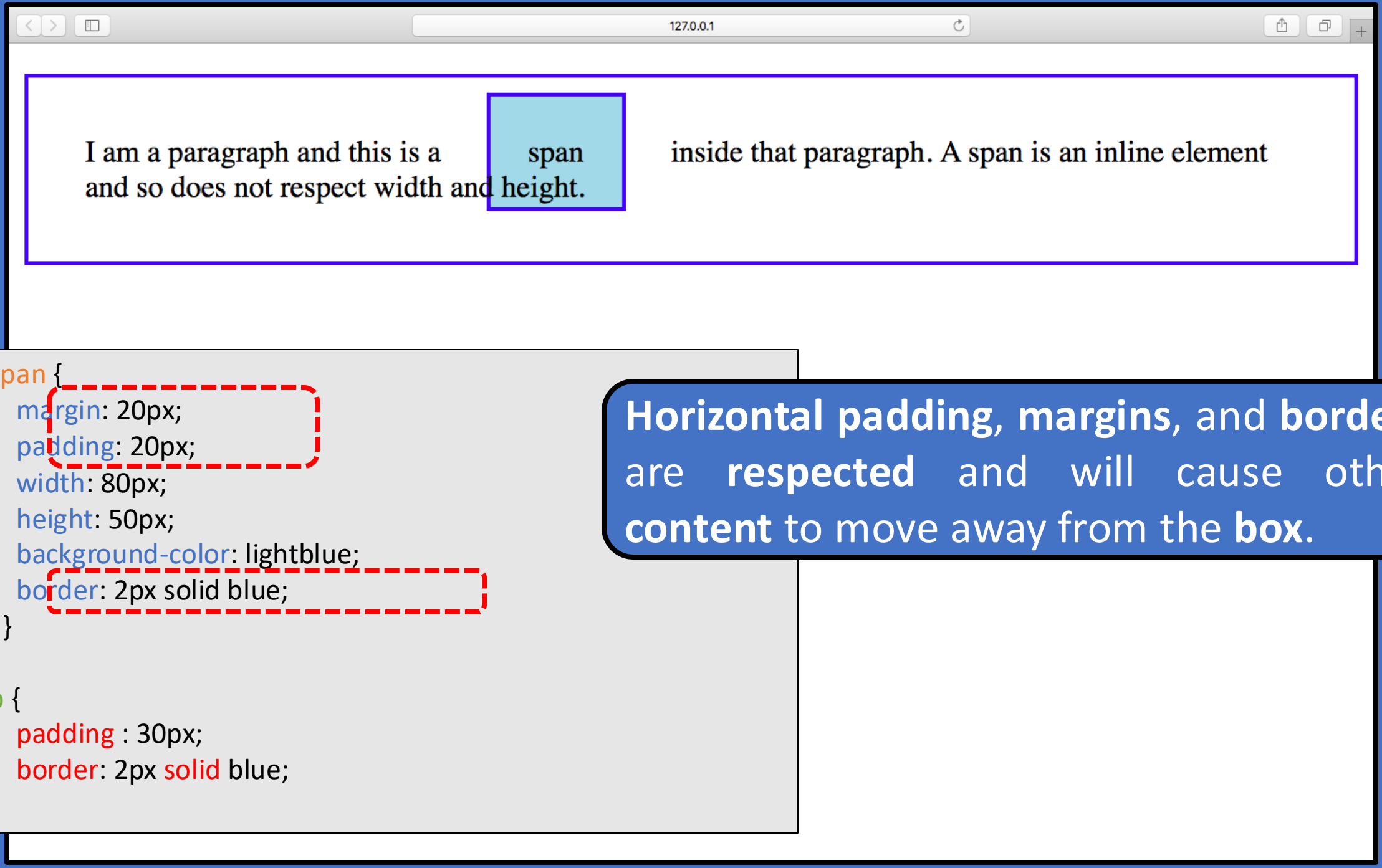

```
01  span {
02      margin: 20px;
03      padding: 20px;
04      width: 80px;
05      height: 50px;
06      background-color: lightblue;
07      border: 2px solid blue;
08    }
09
10  p {
11      padding : 30px;
12      border: 2px solid blue;
13  }
```

You **can** see that the `width` **and** `height` **are** ignored for the `span` **inline** element.




```
01  span {
02      margin: 20px;
03      padding: 20px;
04      width: 80px;
05      height: 50px;
06      background-color: lightblue;
07      border: 2px solid blue;
08    }
09
10  p {
11      padding : 30px;
12      border: 2px solid blue;
13  }
```

The **vertical margin**, **padding**, and **border** are **respected** but they **do not change** the **relationship** of **other content** to **our inline box** and so the `padding` and `border` overlaps other **words** in the **paragraph**.




```
01  span {
02      margin: 20px;
03      padding: 20px;
04      width: 80px;
05      height: 50px;
06      background-color: lightblue;
07      border: 2px solid blue;
08    }
09
10  p {
11      padding : 30px;
12      border: 2px solid blue;
13  }
```

Horizontal padding, **margins**, and **borders** are **respected** and will cause other **content** to move away from the **box**.

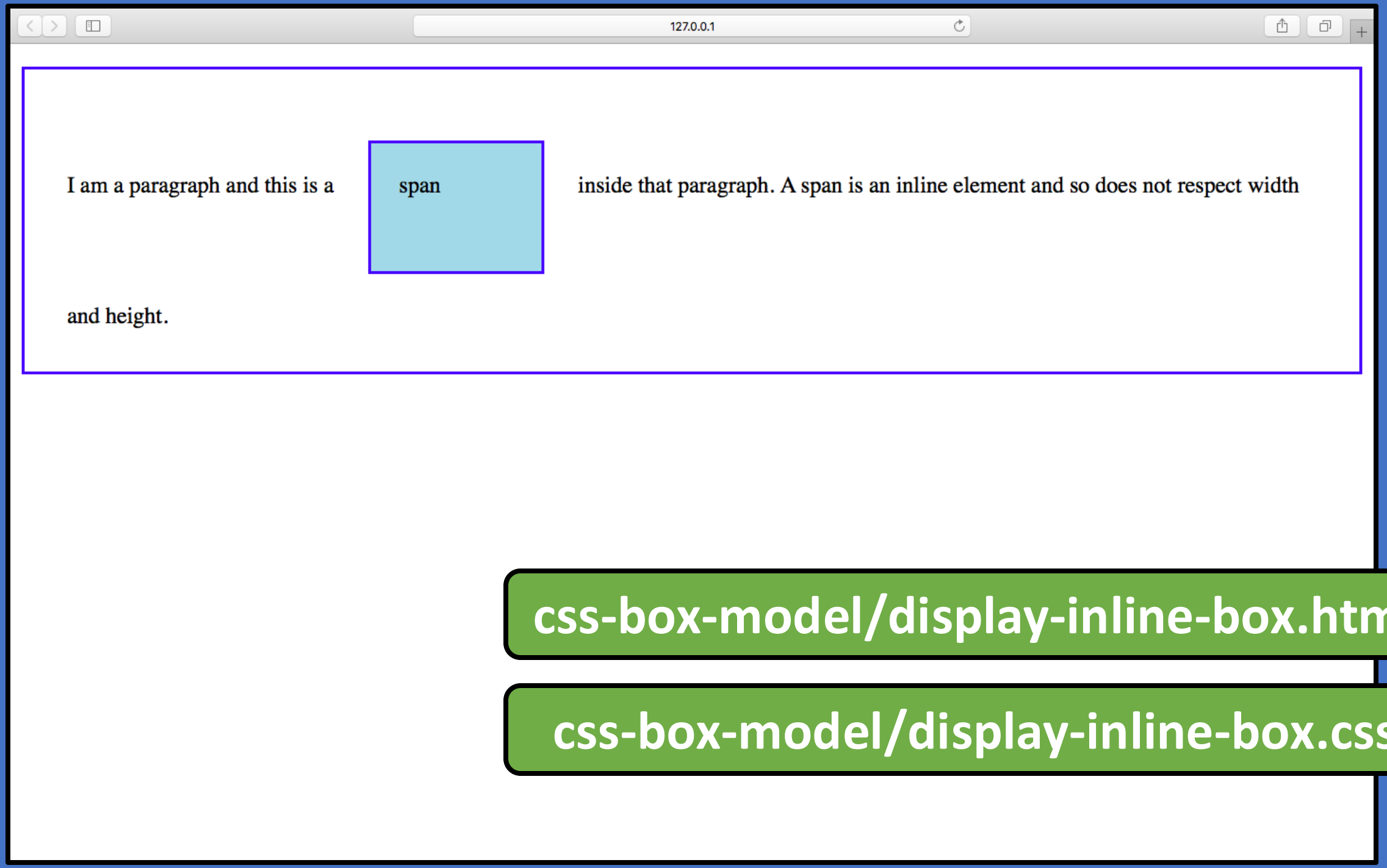
Using `display:inline-block`

Using `display:inline-block`

- There is a **special value** of `display`, which **provides** a **middle ground** between `inline` and `block`.
- **Useful** when you **do not want** an **item** to **break** onto a **new line**, but do **want** it to **respect** `width` and `height` and **avoid** the **overlapping** seen in the **previous example**.

Using `display:inline-block`

- An element with `display: inline-block` does a **subset** of the **block things** we **already** know **about**:
  - ✓The `width` and `height` properties are **respected**.
  - ✓`padding`, `margin`, and `border` will **cause** other **elements** to be **pushed away** from the **box**.
- It **does not**, however, **break onto** a **new line**, and will **only become larger** than its **content** if you **explicitly** add `width` and `height` properties.



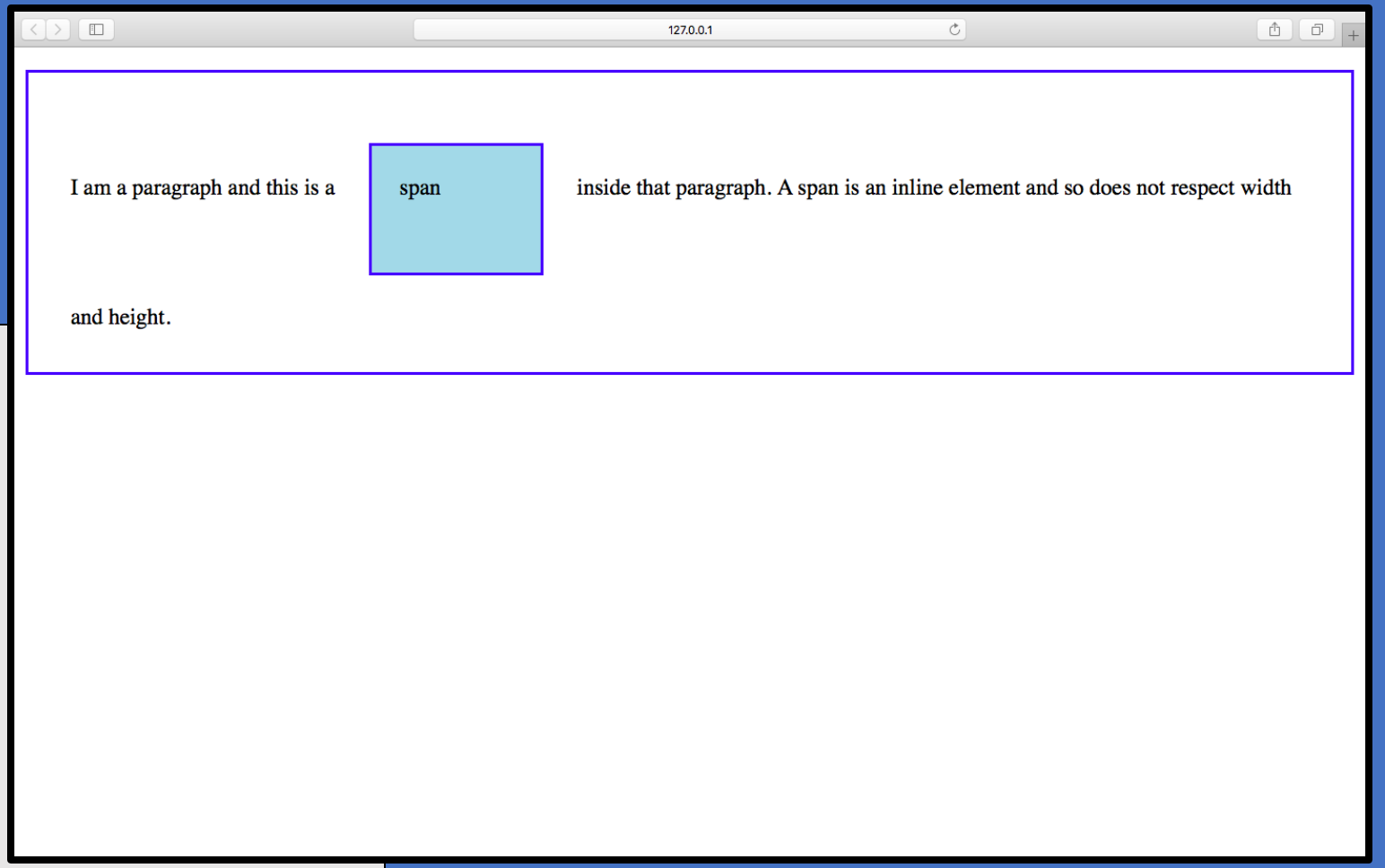
css-box-model/display-inline-box.html

css-box-model/display-inline-box.css

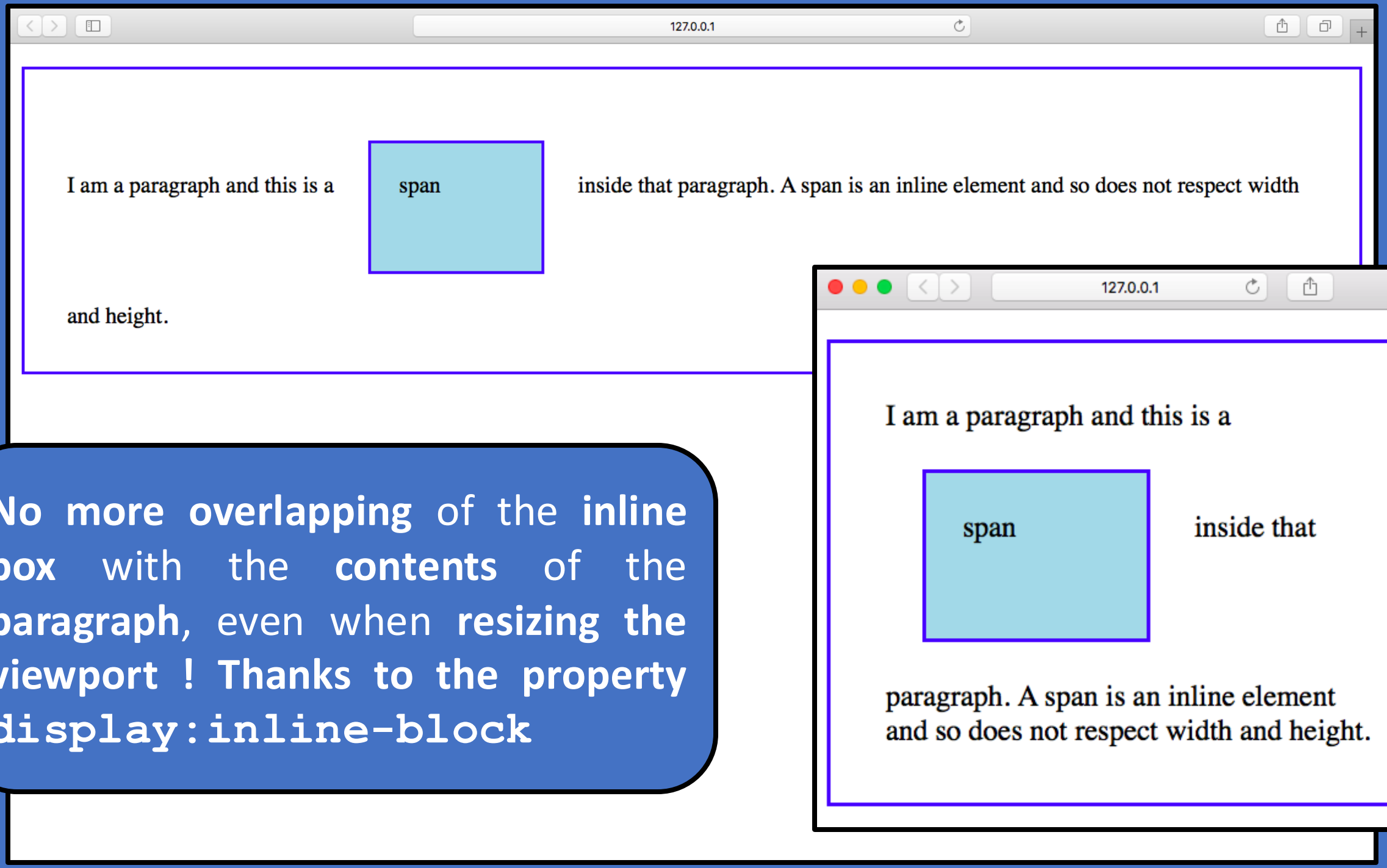
display: inline-block

```css
span {
  margin: 20px;
  padding: 20px;
  width: 80px;
  height: 50px;
  background-color: lightblue;
  border: 2px solid blue;
  display: inline-block;
}

p {
  border: 2px solid blue;
}
```



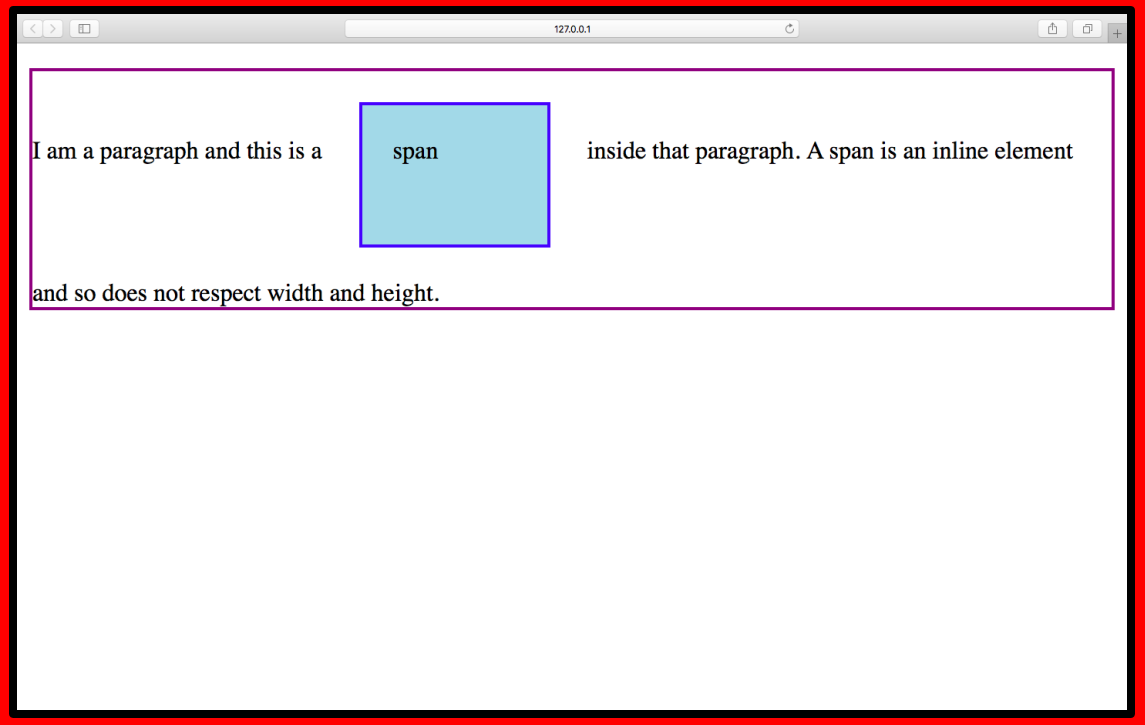

```html
<p>I am a paragraph and this is a <span>span</span> inside that
paragraph. A span is an inline element and so does not respect width
and height.</p>
```

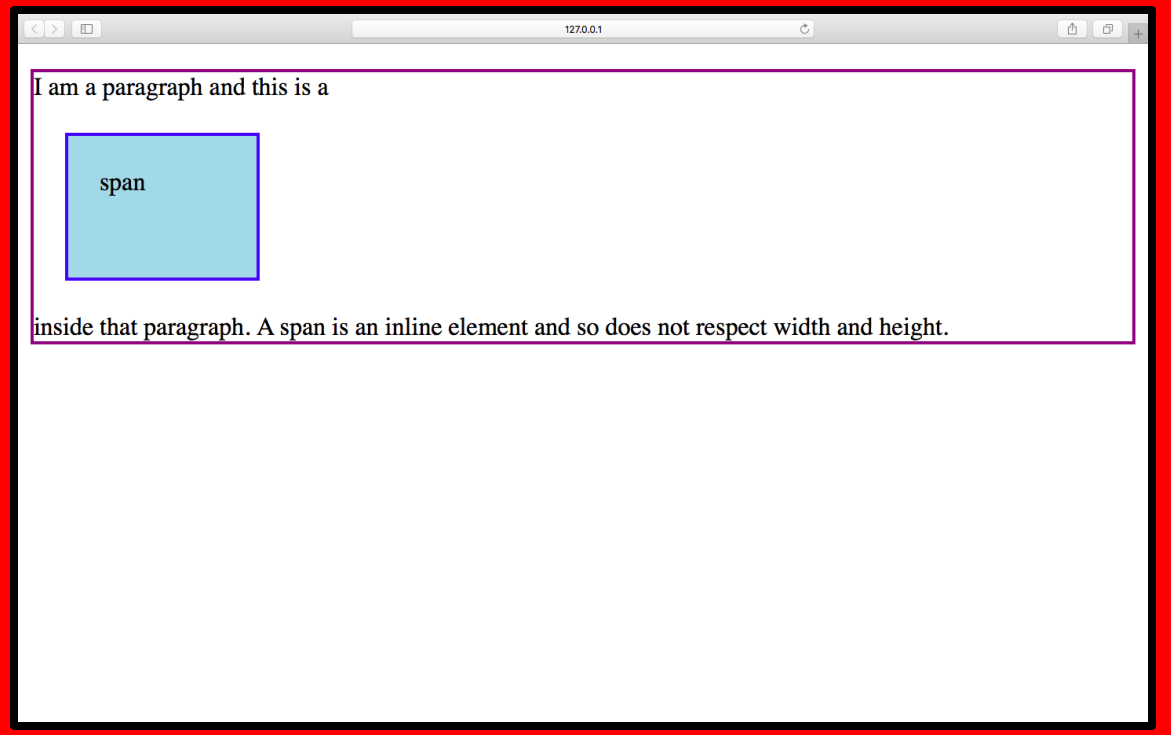


No more overlapping of the **inline box** with the **contents** of the **paragraph**, even when **resizing the viewport ! Thanks to the property** `display:inline-block`

Active learning:
The **box model** and **inline boxes**

We have added **`display: inline-block`** to our **`<span>` element.** Try **changing** this to **`display: block`** or **removing** the **line** completely to **see** the **difference** in **display models**

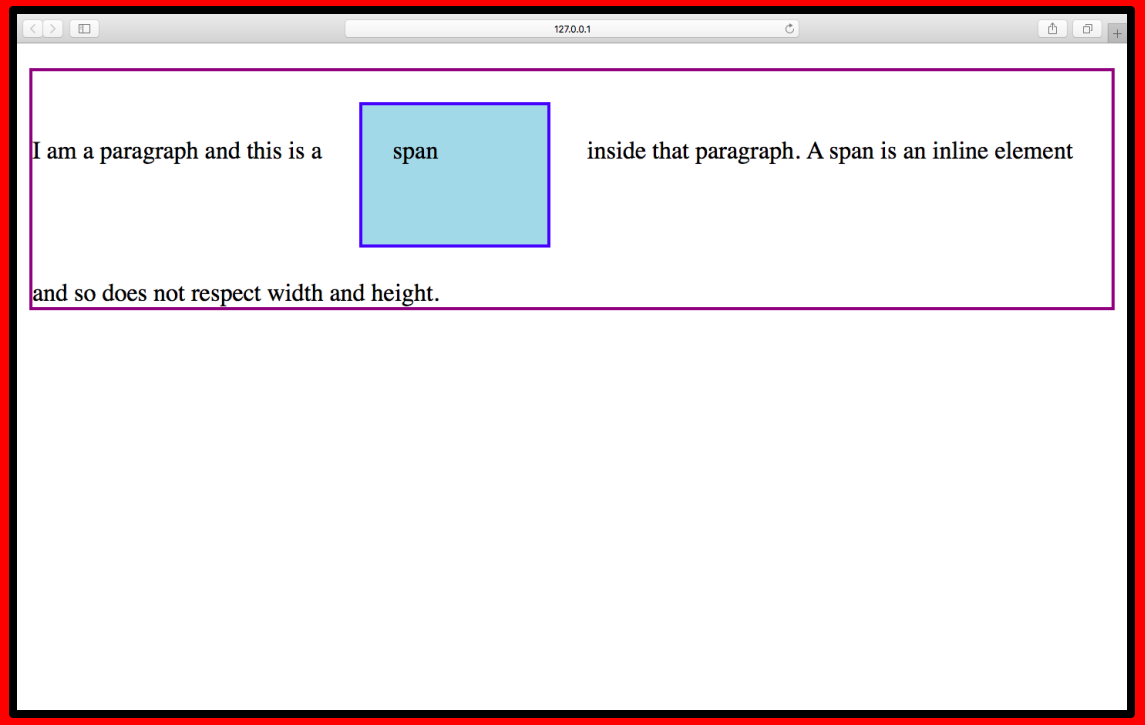


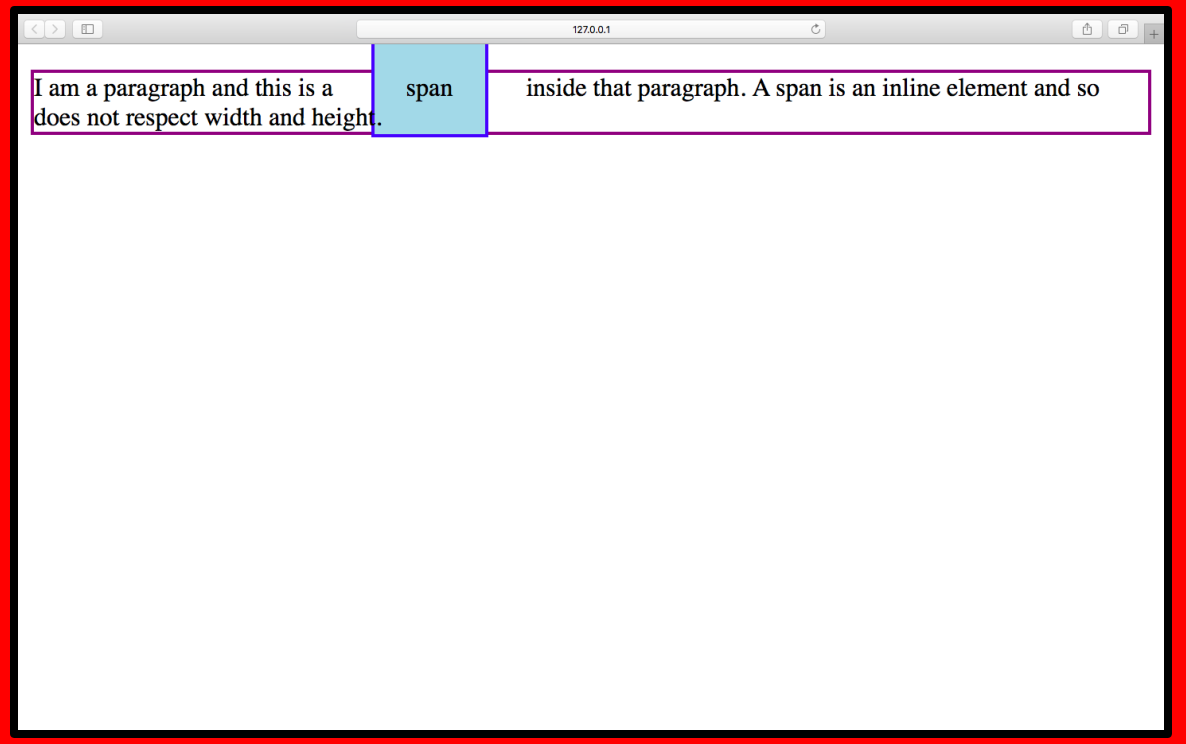


https://mzl.la/3IsfgXY

`display:block`

We have added **`display: inline-block`** to our **`<span>`** **element.** Try **changing** this to **`display: block`** or **removing** the **line** completely to **see** the **difference** in **display models**





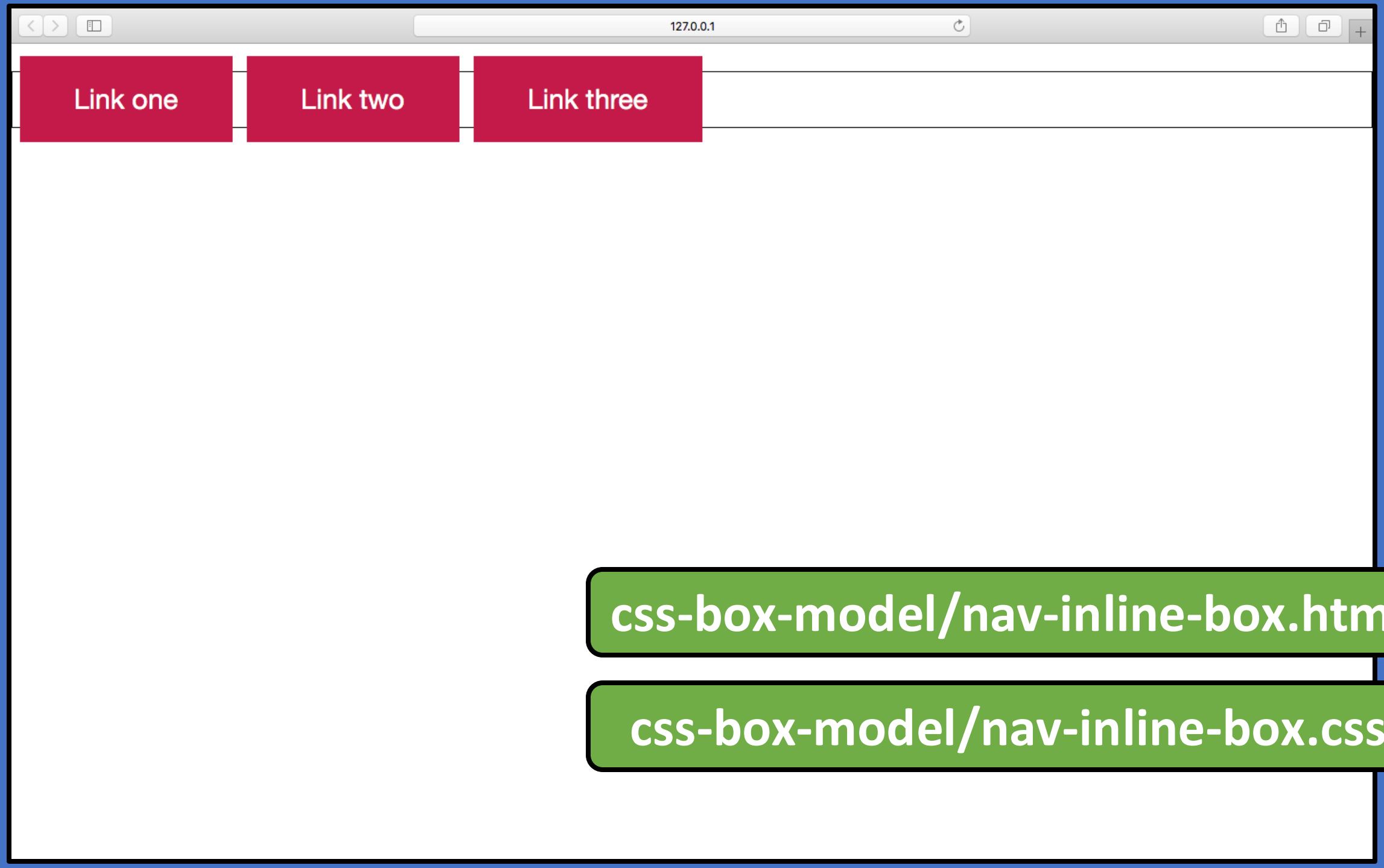
Property removed

Using `display:inline-block`

- **Useful when** you **want** to **give** a **link** to a **larger hit area** by **adding padding**.

- `<a>` is an **inline element** like `<span>`;
  - ✓You can use `display: inline-block` to allow **padding** to be **set** on it, making it **easier** for a **user** to **click** the **link**.

- You see this **fairly frequently** in **navigation bars**.



css-box-model/nav-inline-box.html

css-box-model/nav-inline-box.css

display:inline-block

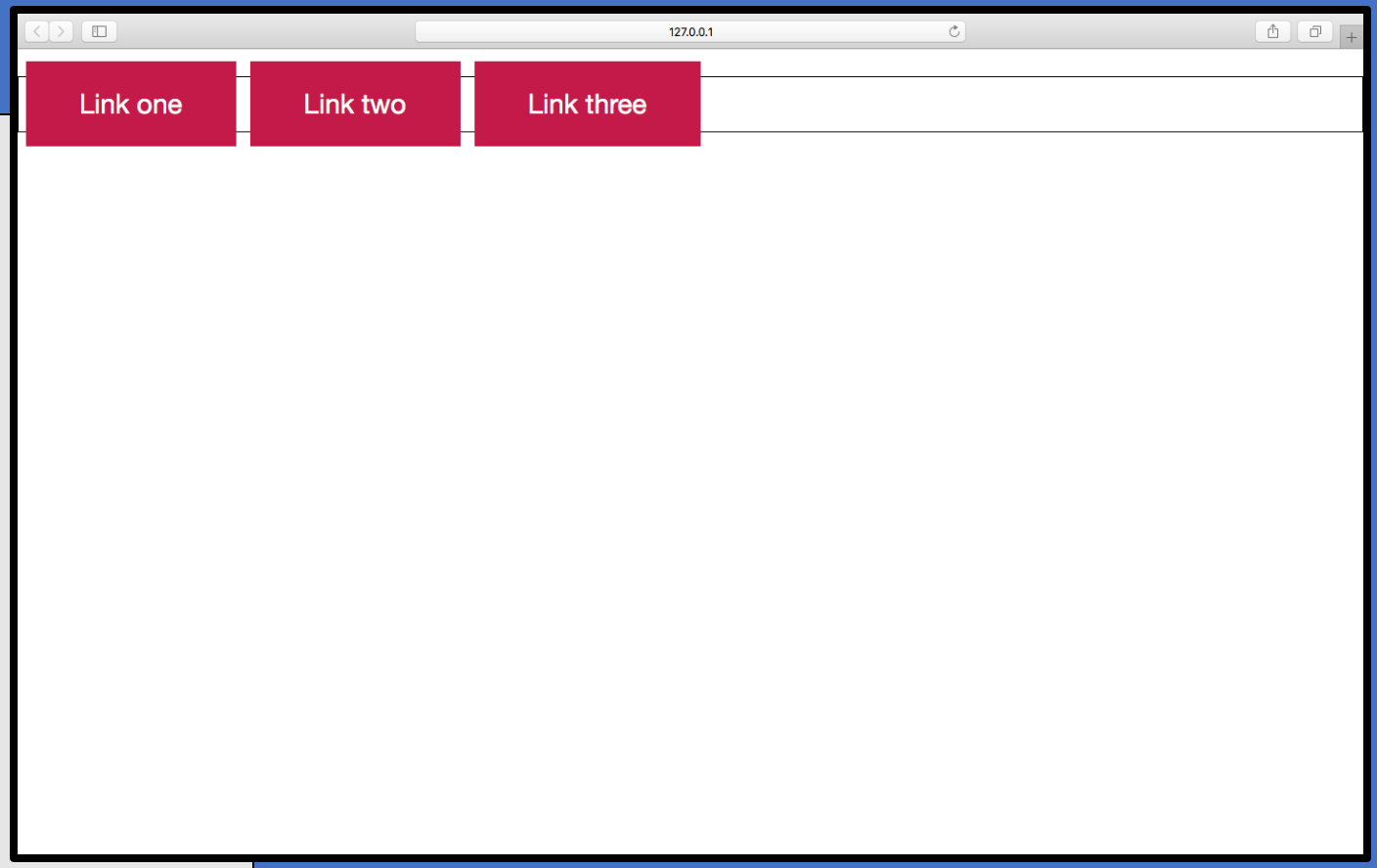
```css
.links-list a  {
  background-color: rgb(179,57,81);
  padding: 1em 2em;
  text-decoration: none;
  display: inline-block;
}

.links-list a:hover {
  background-color: rgb(66, 28, 40);
  color: #fff;
}

ul {
  display: flex;
  ...
}

li {
  list-style: none;
}

```

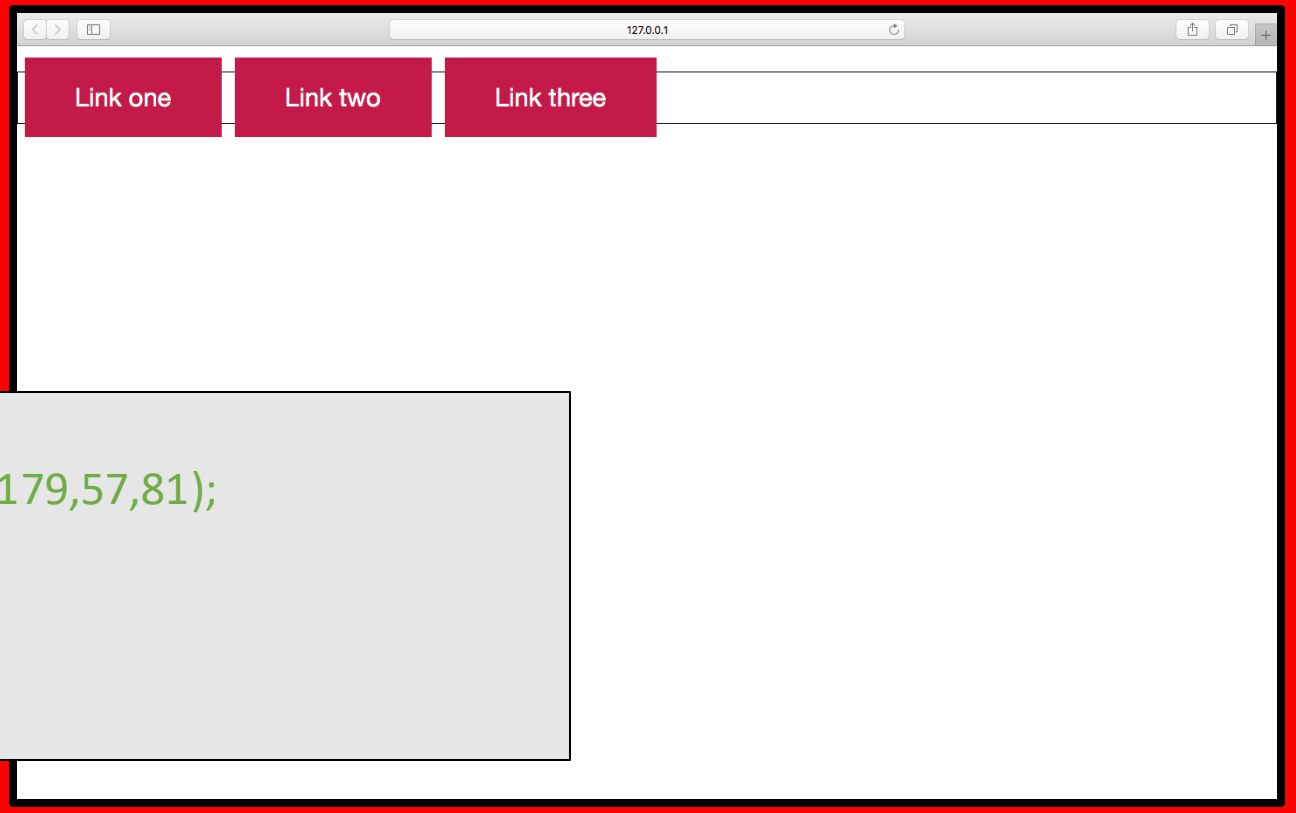



```html
<nav>
  <ul class="links-list">
    <li><a href="">Link one</a></li>
    <li><a href="">Link two</a></li>
    <li><a href="">Link three</a></li>
  </ul>
</nav>
```

Active learning:

`display:inline-block`

Try **adding** **`width`** and **`height`** **properties** and **changing** the `display` values to **see** **difference** in the **output**




```
.links-list a  {
  background-color: rgb(179,57,81);
  padding: 1em 2em;
  text-decoration: none;
  display: inline-block;
}
```

Test your **skills**: the **box model**

Table of contents

The Box Model One
The Box Model Two
The Box Model Three
Assessment or further help

Related Topics

Complete beginners start here!

▶ Getting started with the Web

HTML — Structuring the Web

▶ Introduction to HTML

▶ Multimedia and embedding

▶ HTML tables

CSS — Styling the Web

▶ CSS first steps

▼ CSS building blocks

CSS building blocks overview

Cascade and inheritance

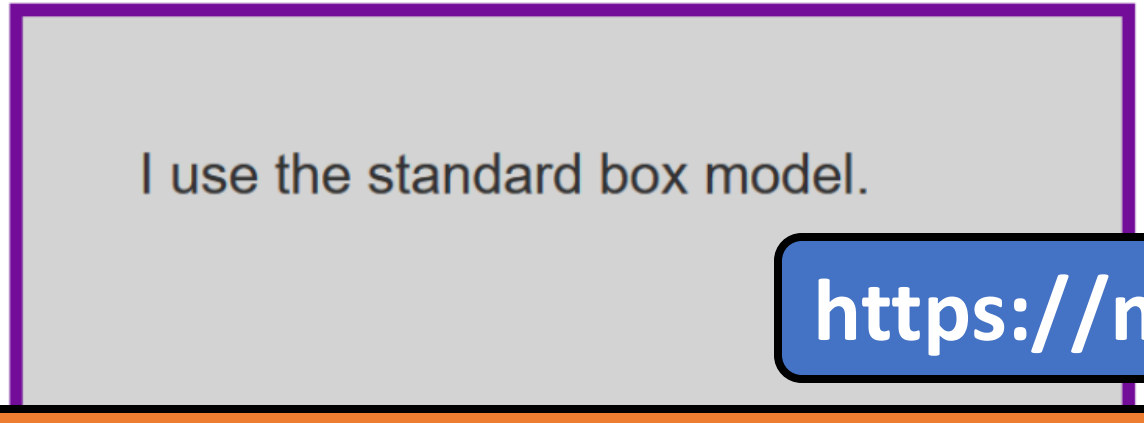
Test your skills: The Box Model

The aim of this task is to help you check your understanding of the CSS Box Model.

> **Note**: You can try out solutions in the interactive editors below, however, it may be helpful to download the code and use an online tool such as CodePen, jsFiddle, or Glitch to work on the tasks.
>
> If you get stuck, then ask us for help — see the Assessment or further help section at the bottom of this page.

The Box Model One

With the two boxes below, one is using the standard box model, the other the alternate box model. Change the width of the second box by adding rules to the .alternate class, so that it matches the visual width of the first box. Your final result will look like the image below.



https://mzl.la/3zZZOGm

Questions?